

STM32 单片机

目录

第 1 章 物联网通信之 CAN 通讯	4
1.1 CAN 通讯介绍	4
1.1.1 物理层	4
1.1.2 协议层	4
1.2 STM32 的 CAN 外设	6
1.2.1 CAN 外设 (CAN 控制器) 介绍	6
1.2.2 CAN 控制器的 3 种工作模式	7
1.2.3 CAN 控制器的 3 种测试模式	7
1.2.4 功能框图	8
1.3 CAN 通讯案例 1: 环回静默模式测试	11
1.3.1 需求描述	11
1.3.2 软件设计 (寄存器)	12
1.3.3 软件设计 (HAL 库)	18
1.4 CAN 通讯实验 2: 双击测试: 1 发 1 收	23
1.4.1 需求描述	23
1.4.2 硬件设计	23
1.4.3 软件设计 (寄存器)	23
1.4.4 软件设计 (HAL 库)	24
第 2 章 物联网通信之以太网通讯	27
2.1 以太网通讯基础知识	27
2.1.1 什么是以太网	27
2.1.2 互联网和以太网的区别	27
2.1.3 以太网的层次	28
2.1.4 OSI 7 层模型	29
2.1.5 TCP/IP 4 层模型	29
2.1.6 一些常见的网络协议	30
2.2 W5500 芯片	31
2.2.1 W5500 芯片介绍	31
2.2.2 W5500 芯片特点	31
2.2.3 应用目标	32
2.2.4 接入框图	32
2.2.5 主控芯片与 W5500 交互	33
2.3 以太网通讯案例 1: 网络搭建	35
2.3.1 需求描述	35
2.3.2 硬件电路设计	35
2.3.3 W5500 官方库的移植	36
2.3.4 软件设计	39
2.4 以太网通讯案例 2: 搭建 TCP 服务端	44
2.4.1 需求描述	44
2.4.2 软件设计	45
2.5 以太网通讯案例 3: 搭建 TCP 客户端	49
2.5.1 需求描述	49
2.5.2 软件设计	49
2.6 以太网通讯案例 4: UDP 通讯	52
2.6.1 需求描述	52

2.6.2 软件设计.....	53
2.7 以太网通讯案例 5: 搭建简易版 web 服务器	55
2.7.1 需求描述.....	55
2.7.2 软件设计.....	56
第 3 章 物联网通讯之 WIFI 通讯	61
3.1 WIFI 基础知识	61
3.1.1 WiFi 由来.....	61
3.1.2 WiFi 和 WLAN.....	61
3.1.3 WiFi 的发展.....	61
3.1.4 IEEE 802.11	62
3.1.5 AP.....	62
3.1.6 WiFi 的频段 5G 和 2.4G	63
3.1.7 WiFi 的信道.....	63
3.1.8 2.4G 的信道.....	63
3.1.9 5G Wi-Fi 信道	63
3.1.10 信道的穿墙能力.....	64
3.1.11 一些其他术语	64
3.2 WIFI 模块	65
3.2.1 ESP32-C3 模块介绍	65
3.2.2 ESP32-C3 模块的特点	65
3.2.3 ESP32-C3 的使用	66
3.3 WiFi 案例 1: 测试 AT 指令.....	71
3.3.1 需求描述.....	71
3.3.2 软件设计.....	71
3.4 WiFi 案例 2: TCP 通讯	75
3.4.1 ESP32 支持的 WIFI 工作模式	75
3.4.2 需求描述.....	75
3.4.3 软件设计.....	76
第 4 章 物联网通讯之蓝牙通讯.....	82
4.1 蓝牙基础知识.....	82
4.1.1 蓝牙概述.....	82
4.1.2 蓝牙产生背景.....	82
4.1.3 蓝牙发展历程.....	82
4.1.4 蓝牙技术类型.....	82
4.1.5 市场上常见蓝牙架构.....	83
4.1.6 蓝牙协议栈.....	85
4.2 ESP32-C3 中的蓝牙功能.....	88
4.2.1 BLE 角色划分	88
4.2.2 BLE 的地址	88
4.2.3 广播.....	89
4.2.4 扫描.....	90
4.2.5 通讯.....	90
4.3 蓝牙通讯案例: 透传模式下收发数据	91
4.3.1 需求描述.....	91
4.3.2 软件设计.....	91
4.3.3 测试.....	96
第 5 章 物联网通讯之 LoRa 通讯	100
5.1 LoRa 通讯基础知识.....	100
5.1.1 常见的 3 种通讯协议.....	100
5.1.2 低功耗广域网	100
5.1.3 LoRa 简介	102

5.1.4 LoRa 的特点.....	102
5.1.5 LoRa 应用.....	103
5.1.6 LoRa 组网架构.....	103
5.2 LoRa 模块 E220-400M22S	104
5.2.1 简介.....	104
5.2.2 应用场景.....	105
5.2.3 引脚说明.....	105
5.2.4 LoRa 通讯的几个关键参数.....	106
5.2.5 硬件电路设计.....	107
5.3 LoRa 通讯实验 1：LoRa 普通节点	108
5.3.1 需求描述.....	108
5.3.2 硬件电路设计.....	108
5.3.3 软件设计.....	109
5.4 LoRa 通讯案例 2：LoRa 网关	121
5.4.1 软件设计.....	121
5.4.2 LoRa 普通节点给网关发送数据测试.....	124

第 1 章 物联网通信之 CAN 通讯

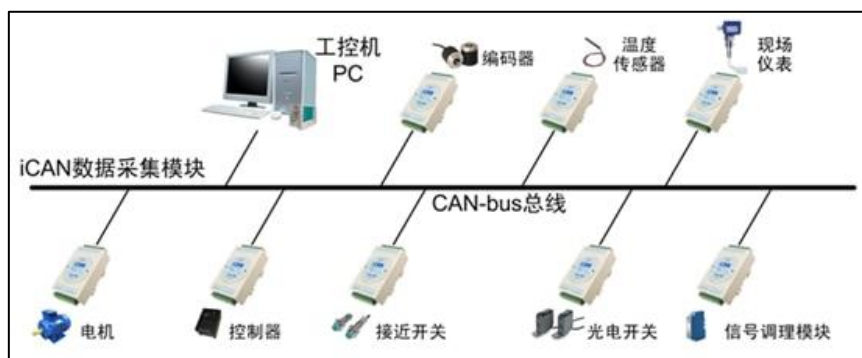
1.1 CAN 通讯介绍

参考：<https://www.kvaser.cn/about-can/can-protocol-tutorial/>

CAN（Controller Area Network 控制器局域网，简称 CAN 或者 CAN bus）是一种功能丰富的车用总线标准。被设计用于在不需要主机（Host）的情况下，允许网络上的单片机和仪器相互通信。

它基于消息传递协议，设计之初在车辆上复用通信线缆，以降低铜线使用量，后来也被其他行业所使用。

CAN 拥有了良好的弹性调整能力，可以在现有网络中增加节点而不用在软、硬件上做出调整。除此之外，消息的传递不基于特殊种类的节点，增加了升级网络的便利性。



1.1.1 物理层

物理层

CAN网络节点

一个CAN控制器

一般MCU提供。STM32内部提供了1个CAN控制器

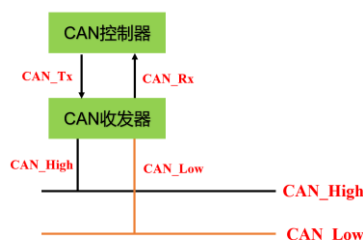
一个CAN收发器

收发器一般需要专门芯片提供。我们开发用的是XL1050

收发器芯片

控制器与收发器之间通过 CAN_Tx及CAN_Rx信号线相连，

收发器与CAN总线之间使用CAN_High及CAN_Low 信号线相连



1.1.2 协议层

1.1.2.1 CAN 的帧（报文）种类

CAN 总线是广播类型的总线。这意味着所有节点都可以侦听到所有传输的报文。无法将报文单独发送给指定节点；所有节点都将始终捕获所有报文。但是 CAN 硬件能够提供本地过滤功能，让每个节点对报文有选择性地做出响应。

CAN 使用短报文 – 最大实用负载是 94 位。报文中没有任何明确的地址；相反，可以认为报文是通过内容寻址，也就是说，报文的内容隐式地确定其地址。

CAN 总线上有 5 种不同的报文类型（或“帧”）：数据帧，远程帧，错误帧，过载帧和帧间隔。

1) 数据帧

数据帧是最常见的报文类型，用于发送单元向接收单元发送数据。

2) 远程帧（遥控帧）

远程帧用于接收单元向具有相同 id 的发送单元请求发送数据。

3) 错误帧

错误帧当检测出错误时向其他单元通知错误的帧。

4) 过载帧

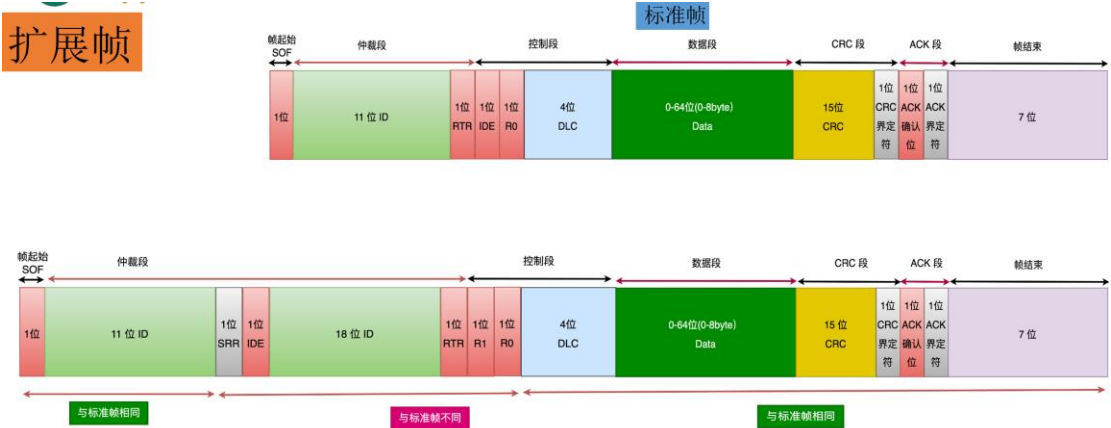
过载帧并不常用，因为当今的 CAN 控制器会非常智能化地避免使用过载帧。

5) 帧间隔

用于将数据帧及遥控帧与前面的帧分离开来的帧

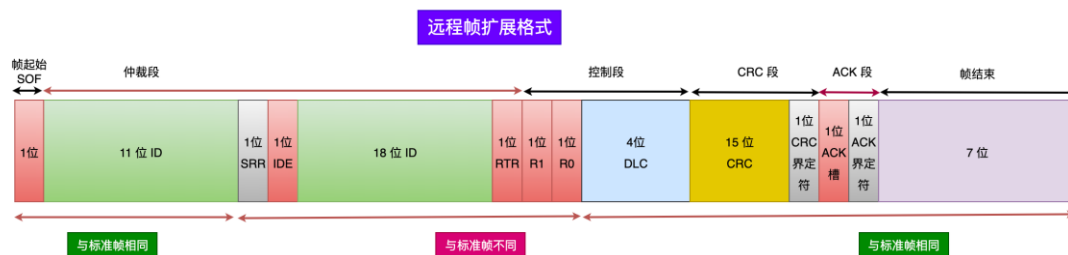
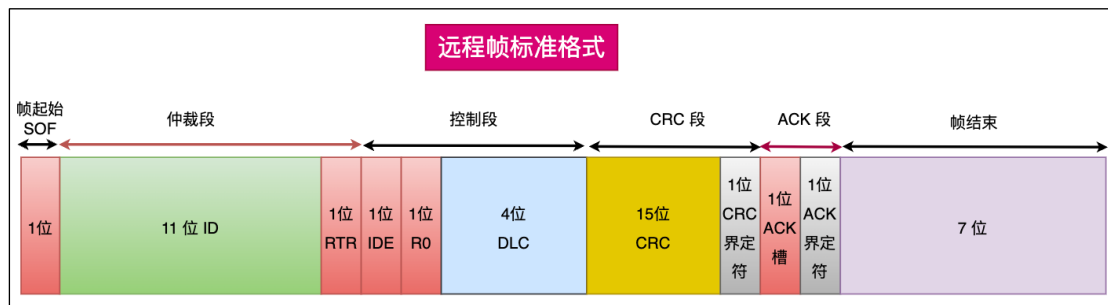
其中错误帧、过载帧、帧间隔都是由硬件自动完成的，没有办法用软件来控制。对于一般使用者来说，只需要掌握数据帧与遥控帧。数据帧和遥控帧有标准格式与扩展格式。标准格式有 11 位标识符，扩展格式有 29 位标识符。

1.1.2.2 数据帧介绍



1.1.2.3 远程帧介绍

与数据帧相比没有数据段。



1.1.2.4 CAN 总线仲裁

CAN 总线处于空闲状态的时候，最先发送消息的单元获得发送权。

多个单元同时开始发送时，从仲裁段（报文 id）的第一位开始进行仲裁。连续输出显性电平最多的单元可以继续发送，即首先出现隐性电平的单元失去总线的占有权变为接收。（即报文 id 小的优先级高）。

竞争失败，会自动检测总线空闲，在第一时间再次尝试发送。

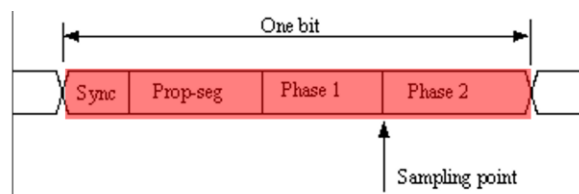
1.1.2.5 CAN 的位时序

位同步

CAN中提出了**位同步**的方式来**确保通讯时序**。

一帧中包含了很多个位

CAN把每**1位**分为**4段**



同步段（SS: Synchronization Segment）

传播时间段（PTS: Propagation Time Segment）

相位缓冲段 1（PBS1: Phase Buffer Segment 1）

相位缓冲段 2（PBS2: Phase Buffer Segment 2）

4段的总时间段构成了位时间（Bit Time），就是传输一个位所需的总时间。

位时间通常被分为若干等长的时间单元，称为时间量化器（Time Quanta, **Tq**）。

一个Tq的长度可以**根据传输速率的需要**设置。

在STM32的CAN外设中，通过设置**波特率分频器**的值来确定**Tq**的大小。

1.2 STM32 的 CAN 外设

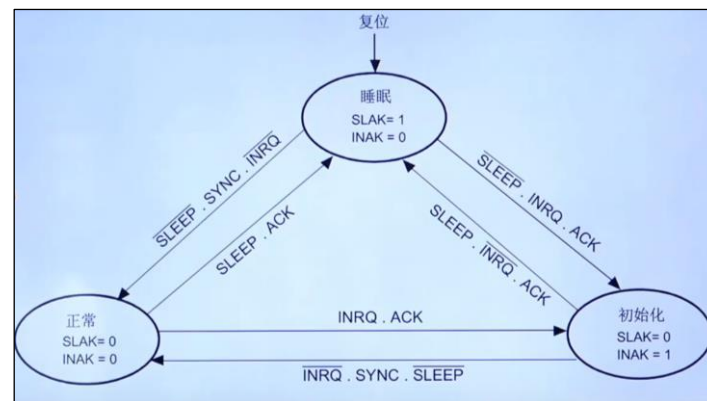
1.2.1 CAN 外设（CAN 控制器）介绍

STM32 的芯片中具有 **bxCAN** 控制器（Basic Extended CAN），它支持 CAN 协议 2.0A 和 2.0B Active 标准。（CAN2.0A 只能处理标准数据帧且扩展帧的内容会识别错误。而 CAN2.0 B Active 可以处理标准数据帧和扩展数据帧。CAN2.0 B Passive 只能处理标准数据帧而扩展帧的内容会被忽略）。

该 CAN 控制器支持最高的通讯速率为 1Mb/s；可以自动地接收和发送 CAN 报文，支持使用标准 ID 和扩展 ID 的报文；外设中具有 3 个发送邮箱，发送报文的优先级可以使用软件控制，还可以记录发送的时间；具有 2 个 3 级深度的接收 FIFO，可使用过滤功能只接收或不接收某些 ID 号的报文；可配置成自动重发；**不支持使用 DMA 进行数据收发**。

1.2.2 CAN 控制器的 3 种工作模式

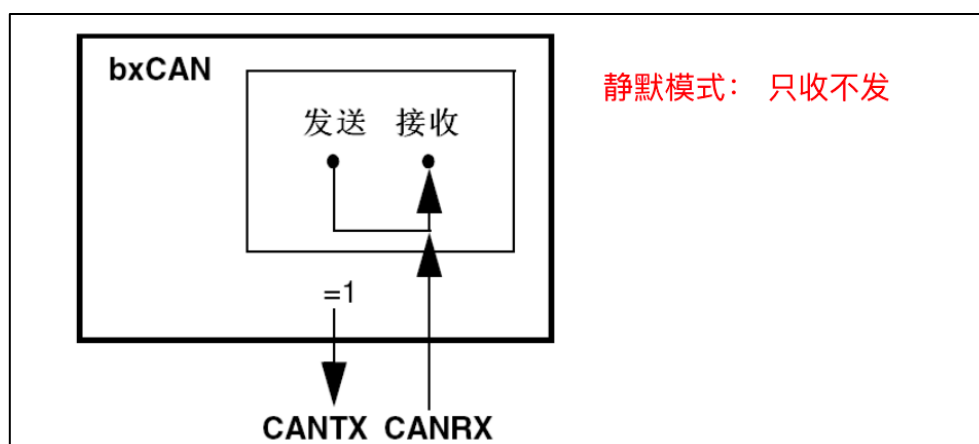
CAN 控制器有 3 种工作模式：初始化模式，正常模式，睡眠模式。

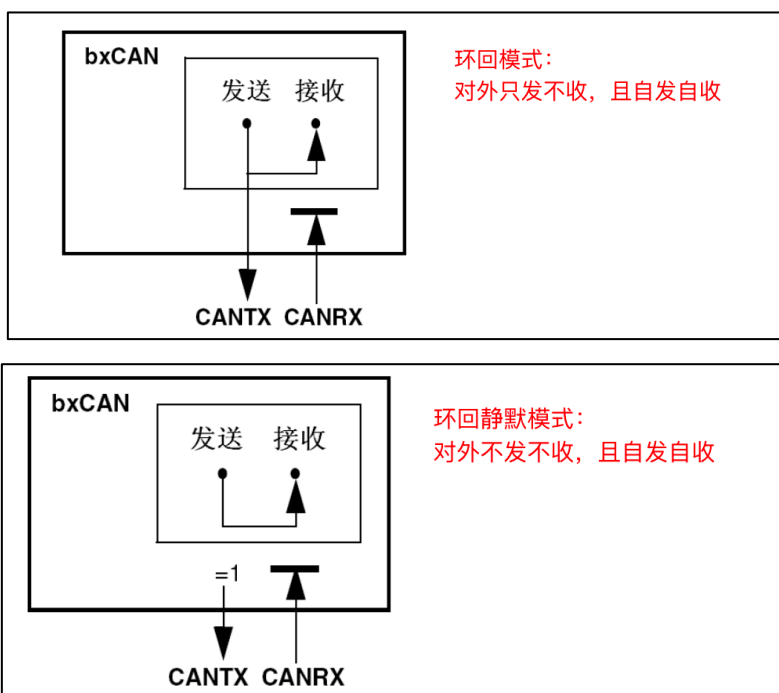


上电复位后 CAN 控制器默认会进入睡眠模式，作用是降低功耗。当需要将进行初始的时候（配置寄存器），会进入初始化模式。当需要通讯的时候，就进入正常模式。

1.2.3 CAN 控制器的 3 种测试模式

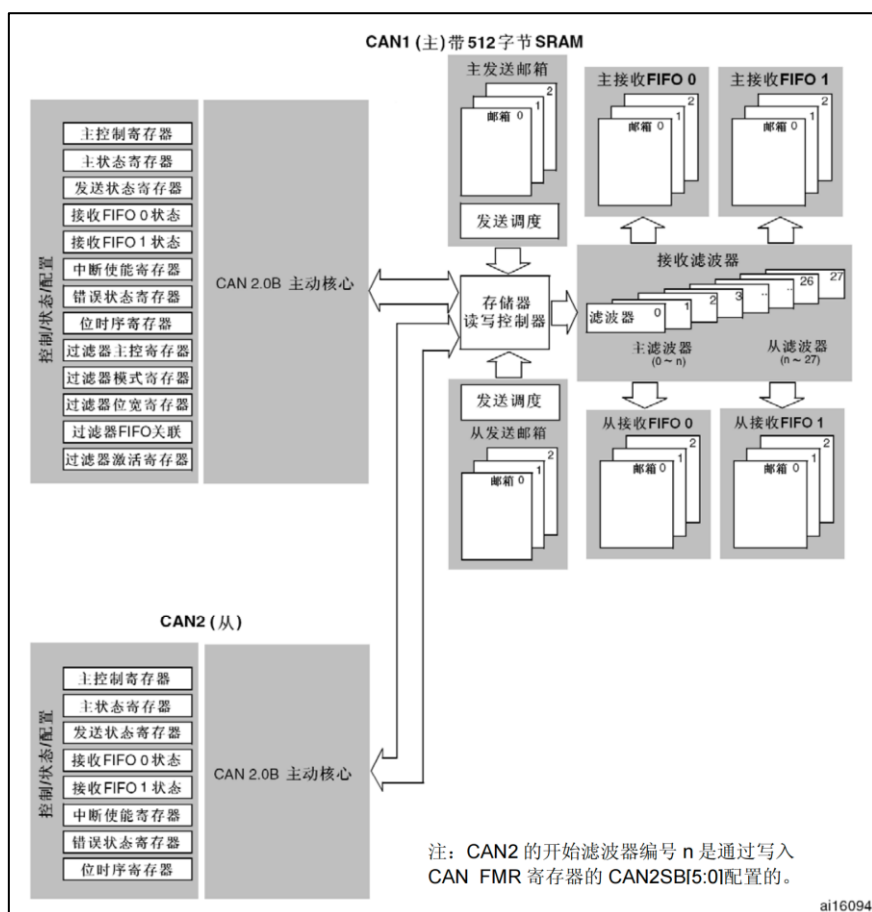
有 3 种测试模式：静默模式、环回模式、环回静默模式。当控制器进入初始化模式的时候才可以配置测试模式。





静默模式可以用于检测总线的数据流量。环回模式可以用于自检（影响总线）。环回静默也是用于自检，不会影响到总线。

1.2.4 功能框图



1.2.4.1 主动内核

含各种控制/状态/配置寄存器，可以配置模式、波特率等。在 STM32CubeMx 中可以非常方便的配置。

1.2.4.2 发送邮箱

用来缓存待发送的报文，最多可以缓存 3 个报文。发送调度决定报文的发送顺序。

1.2.4.3 接收 FIFO

共有 2 个接收 FIFO，每个 FIFO 都可以存放 3 个完整的报文。它们完全由硬件来管理。从而节省了 CPU 的处理负荷，简化了软件并保证了数据的一致性。应用程序只能通过读取 FIFO 输出邮箱，来读取 FIFO 中最先收到的报文。

1.2.4.4 接收滤波器（过滤器）

做用：对接到的报文进行过滤。最后放入 FIFO 0 或 FIFO 1。

当总线上报文数据量很大时，总线上的设备会频繁获取报文，占用 CPU。过滤器的存在，选择性接收有效报文，减轻系统负担。

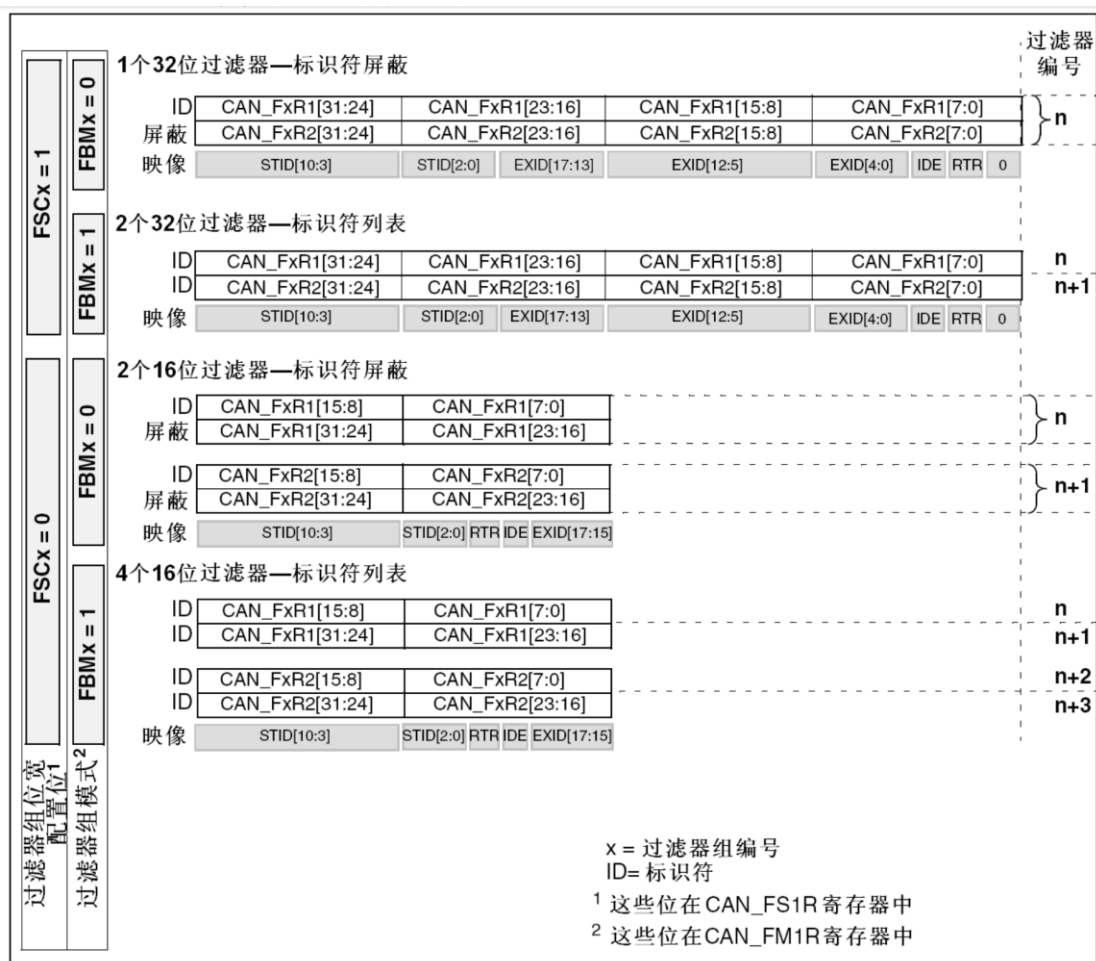
有 2 种过滤模式：

（1）标识符列表模式，它把要接收报文的 ID 列成一个表，要求报文 ID 与列表中的某一个标识符完全相同才可以接收，可以理解为白名单管理。

（2）掩码模式（屏蔽位模式），它把可接收报文 ID 的某几位作为列表，这几位被称为掩码，可以把它理解成关键字搜索，只要掩码（关键字）相同，就符合要求，报文就会被保存到接收 FIFO。

如果使能了筛选器，且报文的 ID 与所有筛选器的配置都不匹配，CAN 外设会丢弃该报文，不存入接收 FIFO。

每个 CAN 提供了 14 个位宽可变的、可配置的过滤器组(13~0)。每个过滤器组 x 由 2 个 32 位寄存器，CAN_FxR1 和 CAN_FxR2 组成。



说明：

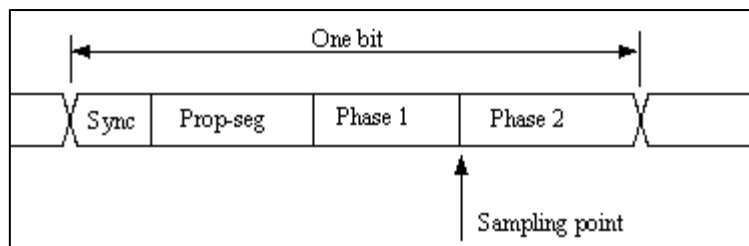
(1) 当工作于 32 位屏蔽位模式时，FR1 保存标识符，FR2 保存屏蔽。FR2 某位是 1 表示来的 ID 的这位必须和 FR1 中对应的位一致，FR2 某位是 0，表示 ID 的这位不关心。

(2) 当工作于 32 位标识符模式时。FR1 和 FR2 分别保存两个标识符。这意味着将来只有两个 ID 会匹配成功。

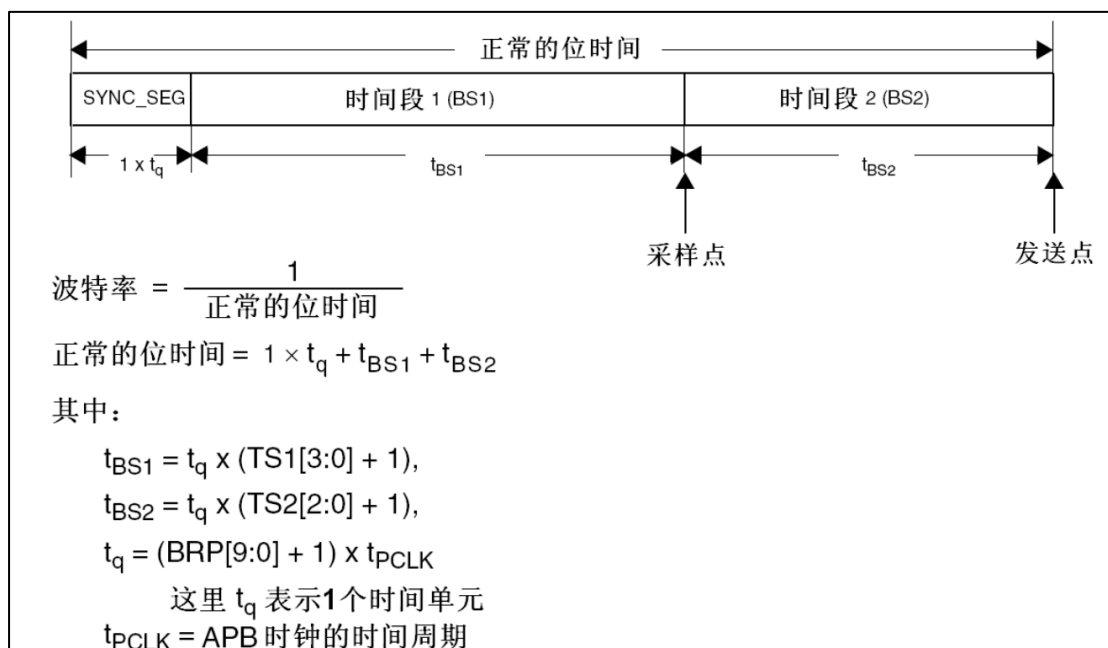
1.2.4.5 STM32 种 CAN 的位时序

STM32 外设定义的位时序与我们前面解释的 CAN 标准时序有一点区别。

标准时序：



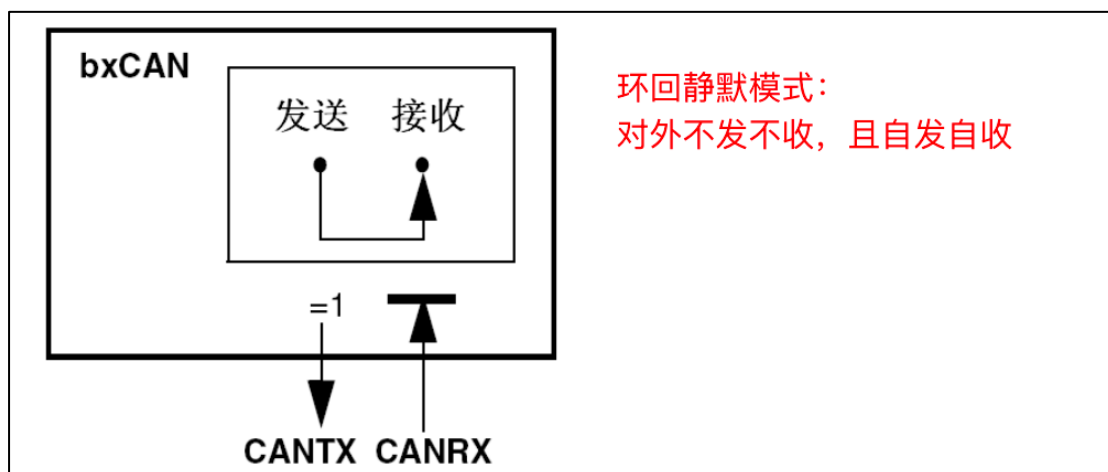
STM32 的位时序：把传播时间段和相位缓冲段 1 做了合并。

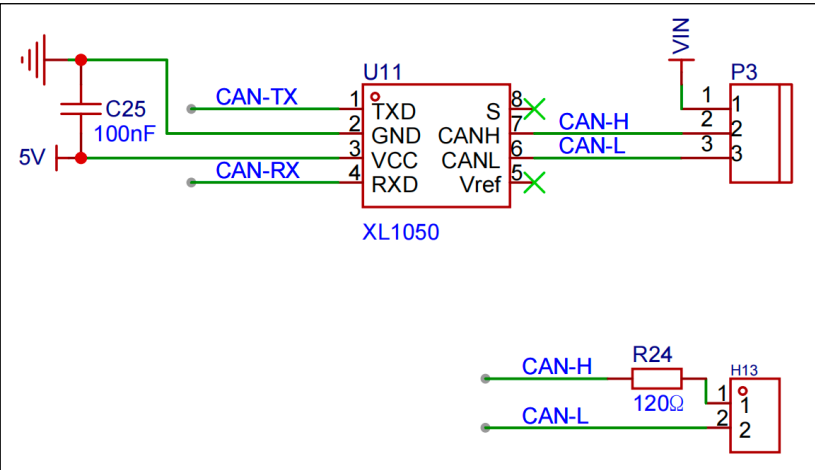
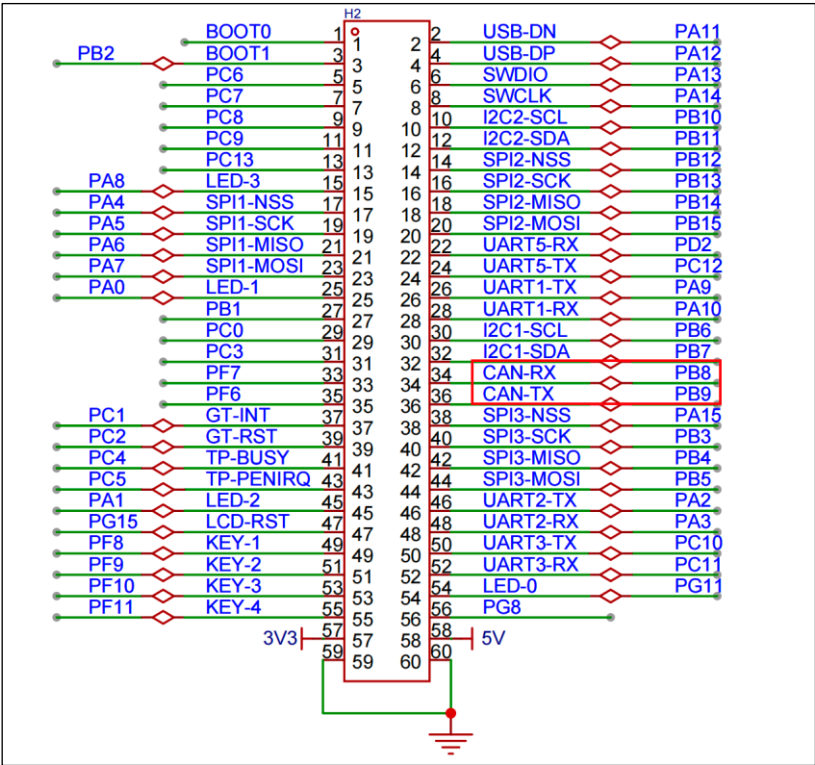


1.3 CAN 通讯案例 1：环回静默模式测试

1.3.1 需求描述

我们使用环回静默模式测试 CAN 能否正常工作。把接收到的报文数据发送到串口输出，看是否可以正常工作。





PB8	I/O	FT	PB8	TIM4_CH3 ⁽⁷⁾ /SDIO_D4	I2C1_SCL/ CAN_RX
PB9	I/O	FT	PB9	TIM4_CH4 ⁽⁷⁾ /SDIO_D5	I2C1_SDA/ CAN_TX

我们没有用 CAN 的默认引脚，而是用的重定向的引脚 PB8 和 PB9。

1.3.2 软件设计（寄存器）



CAN通信案例用到的寄存器详解.docx

1.3.2.1 main.c

```
#include "stm32f10x.h" // Device header
#include "usart.h"
#include "Delay.h"
#include "can.h"
#include "string.h"

int main(void)
{
    usart1_init();
    printf("尚硅谷 CAN 通讯实验：静默回环 寄存器版\r\n");
    CAN_Init();
    printf("CAN 初始化配置完成...\r\n");

    uint16_t stdId = 0x066;
    uint8_t *tData = "abcdefg";
    CAN_SendMsg(stdId, tData, strlen((char *)tData));
    printf("发送完毕...\r\n");

    tData = "123";
    CAN_SendMsg(stdId, tData, strlen((char *)tData));
    printf("发送完毕...\r\n");

    stdId = 0x067;
    tData = "xyz";
    CAN_SendMsg(stdId, tData, strlen((char *)tData));
    printf("发送完毕...\r\n");

    /* 1. 接收数据 */
    RxDataStruct rxDataStruct[8];
    uint8_t rxMsgCount;
    CAN_ReceiveMsg(rxDataStruct, &rxMsgCount);
    printf("接收完毕 rxMsgCount = %d...\r\n", rxMsgCount);

    /* 2. 输出消息 */
    uint8_t i;
    for (i = 0; i < rxMsgCount; i++)
    {
        RxDataStruct msg = rxDataStruct[i];
        printf("stdId = %d, length = %d, msgData = %s\r\n", msg.stdId, msg.length,
msg.data);
    }
    while (1)
    {
```

```
}  
}
```

1.3.2.2 can.h

```
#ifndef __CAN_H  
#define __CAN_H  
#include "stm32f10x.h"  
#include "usart.h"  
#include "string.h"  
  
/**  
 * @description: 存储接收到的数据  
 * @return {*}  
 */  
typedef struct  
{  
    uint16_t stdId;  
    uint8_t data[8];  
    uint8_t length;  
} RxDataStruct;  
  
void CAN_Init(void);  
void CAN_SendMsg(uint16_t stdId, uint8_t *data, uint8_t length);  
void CAN_ReceiveMsg(RxDataStruct rxDataStruct[], uint8_t *msgCount);  
  
#endif
```

1.3.2.3 can.c

```
#include "can.h"  
  
/**  
 * @description: CAN 通讯初始化  
 */  
void CAN_Init(void)  
{  
    /* 1. 开启时钟 CAN 时钟和 GPIO 时钟 */  
    RCC->APB1ENR |= RCC_APB1ENR_CAN1EN;  
    RCC->APB2ENR |= RCC_APB2ENR_IOPBEN;  
    RCC->APB2ENR |= RCC_APB2ENR_AFIOEN;  
  
    /* 2. 重定向 PB8 和 PB9 引脚 10: CAN_RX 映像到 PB8, CAN_TX 映像到 PB9 */  
    AFIO->MAPR |= AFIO_MAPR_CAN_REMAP_1;  
    AFIO->MAPR &= ~AFIO_MAPR_CAN_REMAP_0;
```

```

/* 3. 初始化 GPIO:
    PB9(CAN_Tx): 复用推挽输出 mode=11 cnf=10
    PB8(CAN_Rx): 浮空输入 mode=00 cnf=01
*/
GPIOB->CRH |= GPIO_CRH_MODE9; /* mode = 11 */
GPIOB->CRH |= GPIO_CRH_CNF9_1; /* cnf = 10 */
GPIOB->CRH &= ~GPIO_CRH_CNF9_0;

GPIOB->CRH &= ~GPIO_CRH_MODE8; /* mode = 00 */
GPIOB->CRH &= ~GPIO_CRH_CNF8_1; /* cnf = 01 */
GPIOB->CRH |= GPIO_CRH_CNF8_0;

/* 4. 初始化 CAN */
/* 4.1 进入初始化模式 */
CAN1->MCR |= CAN_MCR_INRQ;
while ((CAN1->MSR & CAN_MSR_INAK) == 0) /* 等待进入初始化模式 */
    ;
/* 4.2 退出睡眠模式 */
CAN1->MCR &= ~CAN_MCR_SLEEP;
while ((CAN1->MSR & CAN_MSR_SLAK) != 0) /* 等待退出睡眠模式 */
    ;

/* 4.3 自动离线管理。 允许自动退出离线状态 */
CAN1->MCR |= CAN_MCR_ABOM;

/* 4.4 自动唤醒管理。 检测到有报文，可以从睡眠模式由硬件自动唤醒。 */
CAN1->MCR |= CAN_MCR_AWUM;

/* 4.5 配置位时序寄存器 */
/* 4.5.1 静默模式 用于调试 */
CAN1->BTR |= CAN_BTR_SILM;
/* 4.5.2 回环模式 用于调试 */
CAN1->BTR |= CAN_BTR_LBKM;
/* 4.5.3 波特率分频器，定义 Tq 的长度。
    配置 35 表示 36 分频，则产生波特率的时钟位 1MHz。
    Tq = 1us
*/
CAN1->BTR &= ~CAN_BTR_BRP; /* 相应的位均置 0 (9:0) */
CAN1->BTR |= 35 << 0;
/* 4.5.4 时间段 1(3*Tq)和时间段 2(6*Tq) */
CAN1->BTR &= ~CAN_BTR_TS1;
CAN1->BTR &= ~CAN_BTR_TS2;
CAN1->BTR |= (3 << 16);
CAN1->BTR |= (6 << 20);

```

```

/* 4.5.5 再同步跳跃宽度 2*Tq*/
CAN1->BTR &= ~CAN_BTR_SJW;
CAN1->BTR |= (2 << 24);

/* 4.6 退出初始化模式 */
CAN1->MCR &= ~CAN_MCR_INRQ;
while ((CAN1->MSR & CAN_MSR_INAK) != 0) /* 等待退出初始化模式 */
    ;

/* 4.7 配置过滤器： 接收所有消息 */
/* 4.7.1 进入过滤器初始化模式 */
CAN1->FMR |= CAN_FMR_FINIT;
/* 4.7.2 过滤器组 0 工作模式： 掩码模式 0:掩码模式 1: 标识符模式 */
CAN1->FM1R &= ~CAN_FM1R_FBM0;
/* 4.7.2 过滤器组 0 为单个 32 位配置 0:2 给 16 位 1:单个 32 位*/
CAN1->FS1R |= CAN_FS1R_FSC0;
/* 4.7.3 给过滤器组 0 分配 FIFO 0:FIF00 1:FIF01. 通过后的报文会放入这个 FIFO 中*/
CAN1->FFA1R &= ~CAN_FFA1R_FFA0;
/* 4.7.4 设置过滤器组 0 标识符寄存器 FR1 */
CAN1->sFilterRegister[0].FR1 = 0x00000000; /* id 每位都是 0 */
/* 4.7.5 设置过滤器组 0 屏蔽位寄存器 FR2 */
CAN1->sFilterRegister[0].FR2 = 0x00000000; /* 屏蔽位是 0，表示不关心 ID 对应的位。都是 0，表示接收所有消息 */
/* 4.7.6 激活过滤器组 0 */
CAN1->FA1R |= CAN_FA1R_FACT0;
/* 4.7.7 退出过滤器初始化模式 */
CAN1->FMR &= ~CAN_FMR_FINIT;
}

/**
 * @description: 发送消息
 * @param {uint16_t} stdId 标准帧 id
 * @param {uint8_t} *data 要发送的数据
 * @param {uint8_t} length 发送的数据的字节数
 */
void CAN_SendMsg(uint16_t stdId,
                 uint8_t *data,
                 uint8_t length)
{
    if (length > 8)
    {
        printf("数据长度不能超过 8 个字节\r\n");
        return;
    }
    /* 1. 等待邮箱 0 为空 (也可以判断其他邮箱) 0: 非空 1: 空*/

```

```

while ((CAN1->TSR & CAN_TSR_TME0) == 0)
    ;

/* 2. 使用标准标识符 0:标准标识符 1: 扩展标识符 */
CAN1->sTxMailBox[0].TIR &= ~CAN_TI0R_IDE;

/* 3. 0: 数据帧 or 1: 远程帧 */
CAN1->sTxMailBox[0].TIR &= ~CAN_TI0R_RTR;

/* 4. 设置标准标识符 */
CAN1->sTxMailBox[0].TIR &= ~CAN_TI0R_STID;
CAN1->sTxMailBox[0].TIR |= (stdId << 21);

/* 5. 设置数据长度 */
CAN1->sTxMailBox[0].TDTR &= ~CAN_TDT0R_DLC;
CAN1->sTxMailBox[0].TDTR |= (length << 0);

/* 6. 设置数据 */
uint8_t i;
CAN1->sTxMailBox[0].TDLR = 0; /* 低位寄存器 */
CAN1->sTxMailBox[0].TDHR = 0; /* 高位寄存器 */
for (i = 0; i < length; i++)
{
    if (i < 4)
    {
        CAN1->sTxMailBox[0].TDLR |= (data[i] << (8 * i));
    }
    else
    {
        CAN1->sTxMailBox[0].TDHR |= (data[i] << (8 * (i - 4)));
    }
}

/* 7. 请求发送数据 */
CAN1->sTxMailBox[0].TIR |= CAN_TI0R_TXRQ;
}

/**
 * @description:
 * @param {uint16_t} *stdId 读取数据的标准 id
 * @param {uint8_t} *data 读取到的数据
 * @param {uint8_t} *length 读取到的数据的长度
 */
void CAN_ReceiveMsg(RxDataStruct rxDataStruct[], uint8_t *msgCount)

```

```

{
    /* 1. 获取 FIFO0 中的报文数 */
    *msgCount = (CAN1->RF0R & CAN_RF0R_FMP0) >> 0;

    uint8_t i, j;
    for (i = 0; i < *msgCount; i++)
    {
        RxDataStruct *msg = &rxDataStruct[i];
        /* 2. 读取标准标识符 id */
        msg->stdId = (CAN1->sFIFOMailBox[0].RIR >> 21) & 0x7FF;

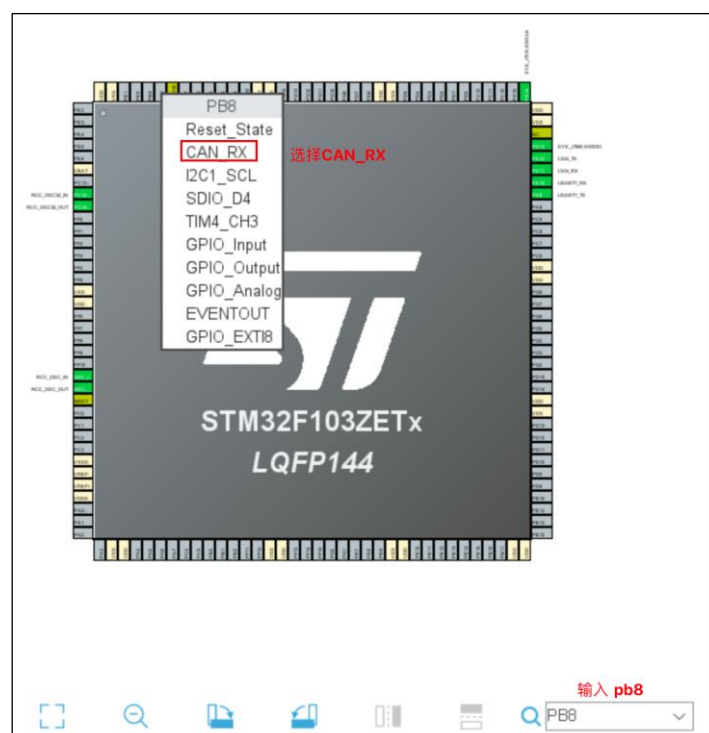
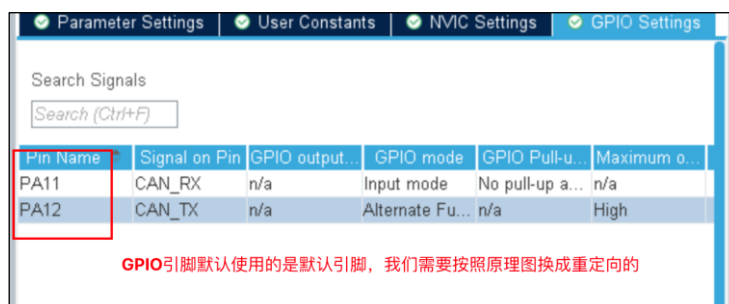
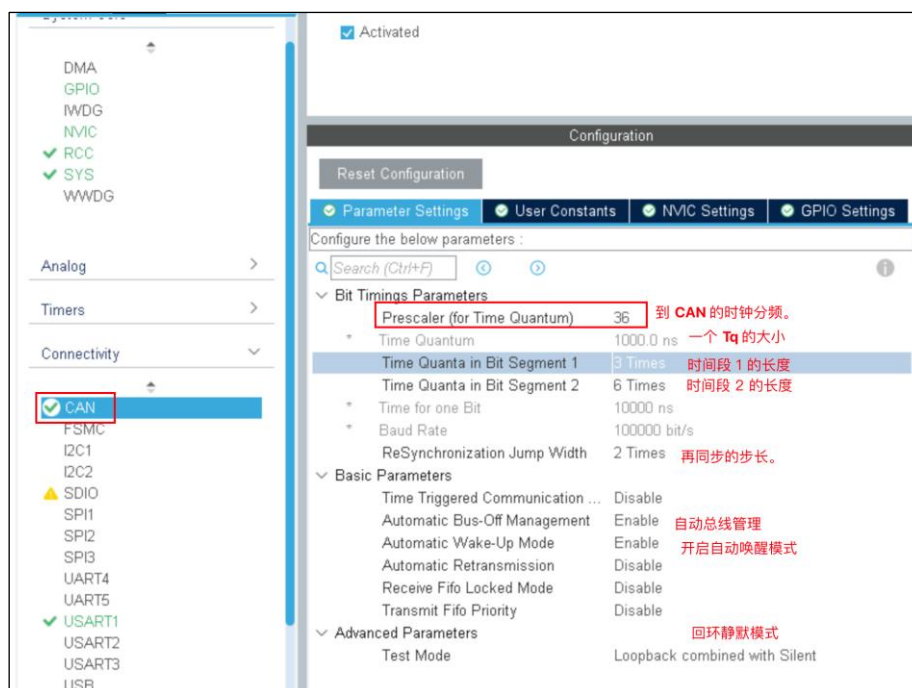
        /* 3. 读取数据长度 */
        msg->length = (CAN1->sFIFOMailBox[0].RDTR >> 0) & 0x0F;

        /* 4. 读取数据 */
        memset(msg->data, 0, sizeof((char *)msg->data));
        uint32_t low = CAN1->sFIFOMailBox[0].RDLR;
        uint32_t high = CAN1->sFIFOMailBox[0].RDHR;
        for (j = 0; j < msg->length; j++)
        {
            if (j < 4)
            {
                msg->data[j] = (low >> (8 * j)) & 0xFF;
            }
            else
            {
                msg->data[j] = (high >> (8 * (j - 4))) & 0xFF;
            }
        }
        /* 5. 释放 FIFO 0. 则报文数减 1 */
        CAN1->RF0R |= CAN_RF0R_RFOM0;
    }
}

```

1.3.3 软件设计（HAL 库）

1.3.3.1 STM32CubeMx 设置



Pin Name	Signal on Pin	GPIO output...	GPIO mode	GPIO Pull-u...	Maximum o...
PB8	CAN_RX	n/a	Input mode	No pull-up a...	n/a
PB9	CAN_TX	n/a	Alternate Fu...	n/a	High

1.3.3.2 can.h

在 can.h 中添加如下代码。

```
/* USER CODE BEGIN Prototypes */
typedef struct
{
    uint16_t stdId;
    uint8_t data[8];
    uint8_t length;
} RxDataStruct;

void CAN_Filter_Config(void);
void CAN_SendMsg(uint16_t stdId, uint8_t *data, uint8_t length);
void CAN_ReceiveMsg(RxDataStruct rxDataStruct[], uint8_t *msgCount);
/* USER CODE END Prototypes */
```

1.3.3.3 can.c

在 can.c 中添加如下代码

```
/* USER CODE BEGIN 1 */
/**
 * @description: 配置过滤器
 */
void CAN_Filter_Config()
{
    CAN_FilterTypeDef sFilterConfig;

    sFilterConfig.FilterBank = 0; // 过滤器编号, CAN1 是 0-13,
    CAN2 是 14-27
    sFilterConfig.FilterMode = CAN_FILTERMODE_IDMASK; // 采用掩码模式
    sFilterConfig.FilterScale = CAN_FILTERSCALE_32BIT; // 设置筛选器的尺度, 采用 32 位
    sFilterConfig.FilterIdHigh = 0x0000; // 过滤器 ID 高 16 位, 即 CAN_FxR1
    寄存器的高 16 位
    sFilterConfig.FilterIdLow = 0x0000; // 过滤器 ID 低 16 位, 即 CAN_FxR1
    寄存器的低 16 位
    sFilterConfig.FilterMaskIdHigh = 0x0000; // 过滤器掩码高 16 位, 即
    CAN_FxR2 寄存器的高 16 位
    sFilterConfig.FilterMaskIdLow = 0x0000; // 过滤器掩码低 16 位, 即
    CAN_FxR2 寄存器的低 16 位
    sFilterConfig.FilterFIFOAssignment = CAN_RX_FIFO0; // 设置经过筛选后数据存储到哪个
    接收 FIFO
```

```

        sFilterConfig.FilterActivation = ENABLE;           // 是否使能本筛选器
        sFilterConfig.SlaveStartFilterBank = 14;          // 指定为 CAN1 分配多少个滤波器组

        HAL_CAN_ConfigFilter(&hcan, &sFilterConfig);
    }

/**
 * @description: 发送信息
 * @param {uint16_t} stdId
 * @param {uint8_t} *data
 * @param {uint8_t} length
 */
void CAN_SendMsg(uint16_t stdId,
                 uint8_t *data,
                 uint8_t length)
{
    /* 1. 检测发送邮箱是否可用 */
    while (HAL_CAN_GetTxMailboxesFreeLevel(&hcan) == 0)
        ;

    CAN_TxHeaderTypeDef txHeader;
    txHeader.IDE = CAN_ID_STD; // 标准帧还是扩展帧
    txHeader.RTR = CAN_RTR_DATA; // 帧的类型：数据帧还是远程帧
    txHeader.StdId = stdId; // 标准帧的 id
    txHeader.DLC = length; // 发送的数据长度 单位字节
    uint32_t txMailBox; // 会把这次使用的邮箱存入到这个变量
    /* 2. 发送消息 */
    HAL_CAN_AddTxMessage(&hcan, &txHeader, data, &txMailBox);
}

/**
 * @description: 接收消息
 * @param {RxDataType} *
 */
void CAN_ReceiveMsg(RxDataStruct rxDataStruct[], uint8_t *msgCount)
{
    /* 1. 检测 FIFO0 收到的报文个数 */
    *msgCount = HAL_CAN_GetRxFifoFillLevel(&hcan, CAN_RX_FIFO0);

    /* 2. 遍历出所有消息 */
    uint8_t i;
    CAN_RxHeaderTypeDef rxHeader;
    for (i = 0; i < *msgCount; i++)
    {

```

```

        HAL_CAN_GetRxMessage(&hcan, CAN_RX_FIFO0, &rxHeader, rxDataStruct[i].data);
        rxDataStruct[i].stdId = rxHeader.StdId;
        rxDataStruct[i].length = rxHeader.DLC;
    }
}
/* USER CODE END 1 */

```

1.3.3.4 main.c

```

int main(void)
{

    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();
    MX_CAN_Init();
    MX_USART1_UART_Init();
    /* USER CODE BEGIN 2 */
    /* 1. 配置过滤器 */
    CAN_Filter_Config();
    /* 2. 启动 CAN 总线 */
    HAL_CAN_Start(&hcan);

    /* 3. 发送数据 */
    uint16_t stdId = 0x011;
    uint8_t *tData = "abcdefg";
    CAN_SendMsg(stdId, tData, strlen((char *)tData));
    printf("发送完毕...\r\n");
    tData = "123";
    CAN_SendMsg(stdId, tData, strlen((char *)tData));
    printf("发送完毕...\r\n");

    /* 4. 接收数据 */
    RxDataStruct rxDataStruct[8];
    uint8_t rxMsgCount;
    CAN_ReceiveMsg(rxDataStruct, &rxMsgCount);
    printf("接收完毕 rxMsgCount = %d...\r\n", rxMsgCount);

    /* 5. 输出消息 */
    uint8_t i;
    for (i = 0; i < rxMsgCount; i++)
    {
        RxDataStruct msg = rxDataStruct[i];
        printf("stdId = %d, length = %d, msgData = %s\r\n", msg.stdId, msg.length,
msg.data);
    }
}

```

```
while (1)
{
}
}
```

1.4 CAN 通讯实验 2：双击测试：1 发 1 收

1.4.1 需求描述

使用 2 块开发板实现 CAN 消息的发送和接收。一个发送数据，另外一个接收数据。

1.4.2 硬件设计

需要把 2 块开发的 CAN_High 连起来，CAN_Low 连起来。连接如图所示。

1.4.3 软件设计（寄存器）

复制 CAN 通信案例 1 的寄存器版本工程 **2 次**，一个用于**发送**，一个用于**接收**。

1.4.3.1 can.c

注释掉下面 2 行代码，其他不用做任何变化。

```
/* 4.5 配置位时序寄存器 */
/* 4.5.1 静默模式 用于调试 */
// CAN1->BTR |= CAN_BTR_SILM;
/* 4.5.2 回环模式 用于调试 */
// CAN1->BTR |= CAN_BTR_LBKM;
```

1.4.3.2 main.c 发送

用于发送的工程只**保留发送**的代码即可。

```
#include "stm32f10x.h" // Device header
#include "usart.h"
#include "Delay.h"
#include "can.h"
#include "string.h"

int main(void)
{
    usart1_init();
    printf("尚硅谷 CAN 通讯实验：发送节点 寄存器版\r\n");
    CAN_Init();
    printf("CAN 初始化配置完成...\r\n");

    uint16_t stdId = 123;
    uint8_t *tData = "abcdef";
```

```

while (1)
{
    CAN_SendMsg(stdId, tData, strlen((char *)tData));
    printf("发送完毕...\r\n");
    Delay_s(3);
}
}

```

1.4.3.3 main.c 接收

用于接收的工程只保留接收的代码即可。

```

#include "stm32f10x.h" // Device header
#include "usart.h"
#include "Delay.h"
#include "can.h"
#include "string.h"

int main(void)
{
    usart1_init();
    printf("尚硅谷 CAN 通讯实验：接收节点 寄存器版\r\n");
    CAN_Init();
    printf("CAN 初始化配置完成...\r\n");

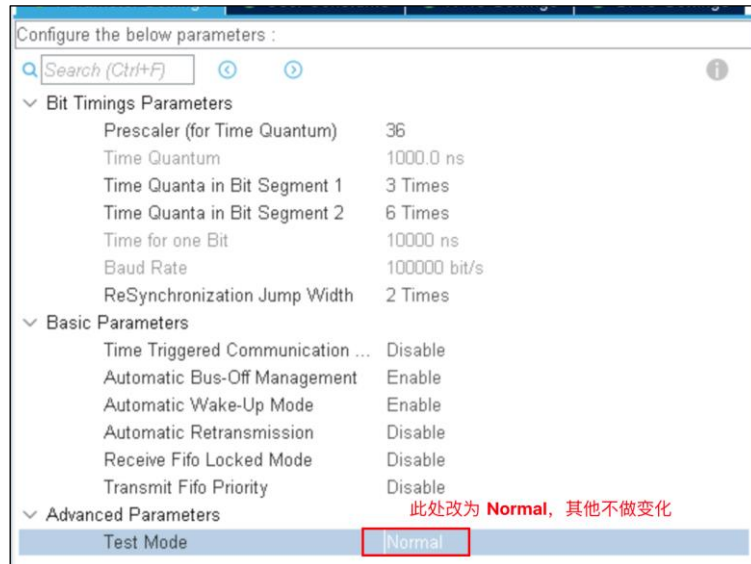
    RxDataStruct rxDataStruct[8];
    uint8_t rxMsgCount;
    while (1)
    {
        /* 1. 接收数据 */
        CAN_ReceiveMsg(rxDataStruct, &rxMsgCount);
        /* 2. 输出消息 */
        uint8_t i;
        for (i = 0; i < rxMsgCount; i++)
        {
            RxDataStruct msg = rxDataStruct[i];
            printf("stdId = %d, length = %d, msgData = %s\r\n", msg.stdId,
msg.length, msg.data);
        }
    }
}

```

1.4.4 软件设计（HAL 库）

1.4.4.1 CubeMx 设置

拷贝 CAN 通信实验 1 的 HAL 库版本工程 2 次。一个用于发送，一个用于接收。重新打开每个工程，把 Test Mode 改成 Normal 即可，其他不用改变。然后重新生成代码。



1.4.4.2 main.c 发送

```
int main(void)
{
    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();
    MX_CAN_Init();
    MX_USART1_UART_Init();
    /* USER CODE BEGIN 2 */
    /* 1. 配置过滤器 */
    CAN_Filter_Config();
    /* 2. 启动 CAN 总线 */
    HAL_CAN_Start(&hcan);
    printf("尚硅谷 CAN 发送实验...\r\n");

    while (1)
    {
        uint16_t stdId = 0x011;
        uint8_t *tData = "abcdefg";
        CAN_SendMsg(stdId, tData, strlen((char *)tData));
        printf("发送完毕...\r\n");
        HAL_Delay(300);
    }
}
```

1.4.4.3 main.c 接收

```
int main(void)
{
    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();
    MX_CAN_Init();
    MX_USART1_UART_Init();
    /* USER CODE BEGIN 2 */
    /* 1. 配置过滤器 */
    CAN_Filter_Config();
    /* 2. 启动 CAN 总线 */
    HAL_CAN_Start(&hcan);
    printf("尚硅谷 CAN 接收 实验...\r\n");
    /* 4. 接收数据 */
    RxDataStruct rxDataStruct[8];
    uint8_t rxMsgCount;
    while (1)
    {
        CAN_ReceiveMsg(rxDataStruct, &rxMsgCount);
        /* 5. 输出消息 */
        uint8_t i;
        for (i = 0; i < rxMsgCount; i++)
        {
            RxDataStruct msg = rxDataStruct[i];
            printf("stdId = %d, length = %d, msgData = %s\r\n", msg.stdId,
msg.length, msg.data);
        }
    }
}
```

第 2 章 物联网通信之以太网通讯

2.1 以太网通讯基础知识

2.1.1 什么是以太网

以太网（Ethernet）是一种计算机局域网技术。IEEE 组织的 IEEE 802.3 标准制定了以太网的技术标准，它规定了包括**物理层的连线、电子信号和介质访问控制**的内容。以太网是目前应用最普遍的**局域网技术**，取代了其他局域网标准如令牌环、FDDI 和 ARCNET。

以太网的标准拓扑结构为总线型拓扑，但目前的快速以太网（100BASE-T、1000BASE-T 标准）为了减少冲突，将能提高的网络速度和使用效率最大化，使用交换机（Switch hub）来进行网络连接和组织。如此一来，以太网的拓扑结构就成了星型；但在逻辑上，以太网仍然使用总线型拓扑和 CSMA/CD（Carrier Sense Multiple Access/Collision Detection，即载波多重存取/碰撞侦测）的总线技术。

经过长期的发展，以太网已成为应用最为广泛的局域网，包括标准以太网（10 Mbit/s）、快速以太网（100 Mbit/s）、千兆以太网（1000 Mbit/s）和万兆以太网（10 Gbit/s）等。IEEE 802.3 规范则是基于以太网的标准制定的，并与以太网标准相互兼容。

2.1.2 互联网和以太网的区别

（1）简单来说，网络按照区域来划分，分为广域网和局域网。这只是按照使用区域大小来划分的。就像省和村的关系。

（2）然后在这个小区域（局域网）里建设网络，就需要使用多种标准技术，其中电气标准中规定用双绞线还是单芯线等，这个电气标准中有以**太网技术、令牌环网技术、ATM 网技术、帧中继技术等**，不要被以太网中这个网字迷惑，把它看成技术，我们是用了 CSMA/CA 技术（别名：以太网技术），使用方便、网络建造简洁，以太网技术就是流传开来。

（3）局域网中物理网络按照以太网技术敷设完毕，还并不能通信，这个时候就需要其他技术标准，我们经常见到的 TCP/IP 技术，TCP/IP 技术可以依托以太网技术、令牌环网技术等上使用，而且我们经常 TCP/IP 与以太网配合使用，所以我们日常中口语中容易将 TCP/IP 与以太网技术混在一起说。其实是不同层级的技术。

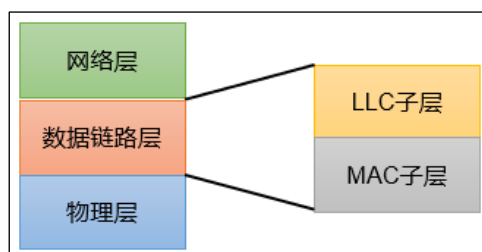
这些所有的东西组合在一起就是互联网了。



互联网是一个范围的概念，以太网只是这个范围内的一种技术。

2.1.3 以太网的层次

以太网采用无源的介质，按广播方式传播信息。它规定了**物理层和数据链路层**协议，规定了物理层和数据链路层的接口以及数据链路层与更高层的接口。



1) 物理层

物理层规定了以太网的基本物理属性，如数据编码、时标、电频等。

物理层位于 OSI 参考模型的最底层，它直接面向实际承担数据传输的物理媒体（即通信通道），物理层的传输单位为比特（bit），即一个二进制位（“0”或“1”）。实际的比特传输必须依赖于传输设备和物理媒体，但是，物理层不是指具体的物理设备，也不是指信号传输的物理媒体，而是指在物理媒体之上为上一层（数据链路层）提供一个传输原始比特流的物理连接。

2) 数据链路层

数据链路层是 OSI 参考模型中的第二层，介于物理层和网络层之间。数据链路层在物理层提供的服务的基础上向网络层提供服务，其最基本的服务是将源设备网络层转发过来的数据可靠地传输到相邻节点的目的设备网络层。

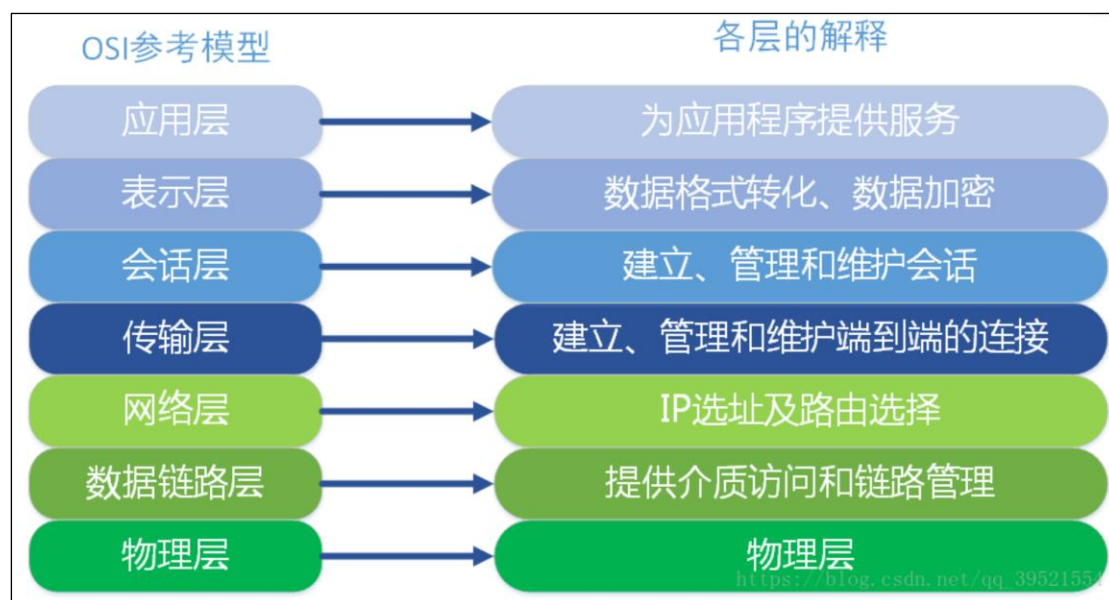
由于以太网的物理层和数据链路层是相关的，针对物理层的不同工作模式，需要提供特定的数据链路层来访问。这给设计和应用带来了一些不便。

为此，一些组织和厂家提出把数据链路层再进行分层，分为媒体接入控制子层（MAC）和逻辑链路控制子层（LLC）。这样不同的物理层对应不同的 MAC 子层，LLC 子层则可以完全独立。

2.1.4 OSI 7 层模型

OSI（Open System Interconnect），即开放式系统互联。一般都叫 OSI 参考模型，是 ISO（国际标准化组织）组织在 1985 年研究的网络互连模型。ISO 为了更好的使网络应用更为普及，推出了 OSI 参考模型。其含义就是推荐所有公司使用这个规范来控制网络。这样所有公司都有相同的规范，就能互联了。

OSI 定义了网络互连的七层框架（物理层、数据链路层、网络层、传输层、会话层、表示层、应用层），即 ISO 开放互连系统参考模型。



每一层实现各自的功能和协议，并完成与相邻层的接口通信。OSI 的服务定义详细说明了各层所提供的服务。

2.1.5 TCP/IP 4 层模型

众所周知，OSI 参考模型是学术上和法律上的国际标准，是完整的权威的网络参考模型。而 **TCP/IP 参考模型** 是事实上的国际标准，即现实生活中被广泛使用的网络参考模型。

OSI 引入了服务、接口、协议、分层的概念，TCP/IP 借鉴了 OSI 的这些概念建力 TCP/IP 模型。

OSI 先有模型，后有协议，先有标准，后进行实践；而 TCP/IP 则相反，先有协议和应用再提出了模型，且是参照的 OSI 模型。

OSI 是一种理论模型，而 TCP/IP 已被广泛使用，成为网络互联事实上的标准。

OSI七层网络模型	TCP/IP四层概念模型	对应网络协议
应用层 (Application)	应用层	HTTP、TFTP, FTP, NFS, WAIS、SMTP
表示层 (Presentation)		Telnet, Rlogin, SNMP, Gopher
会话层 (Session)		SMTP, DNS
传输层 (Transport)	传输层	TCP, UDP
网络层 (Network)	网络层	IP, ICMP, ARP, RARP, AKP, UUCP
数据链路层 (Data Link)	数据链路层	FDDI, Ethernet, Arpanet, PDN, SLIP, PPP
物理层 (Physical)		IEEE 802.1A, IEEE 802.2到IEEE 802.11

2.1.6 一些常见的网络协议

1) IP 协议

IP 协议是 TCP/IP 协议族中最为核心的协议，更确切的说是网络层重要的协议之一。

IP 层接收由更低层（网络接口层例如以太网设备驱动程序）发来的数据包，并把该数据包发送到更高层---TCP 或 UDP 层；相反，IP 层也把从 TCP 或 UDP 层接收来的数据包传送到更低层。IP 数据包是不可靠的，因为 IP 并没有做任何事情来确认数据包是否按顺序发送的或者有没有被破坏，IP 数据包中含有发送它的主机的地址（源地址）和接收它的主机的地址（目的地址）。

2) TCP 协议

TCP 是面向连接的通信协议，通过三次握手建立连接，通讯完成时要拆除连接，由于 TCP 是面向连接的所以只能用于端到端的通讯。

TCP 提供的是一种可靠的数据流服务，采用“带重传的肯定确认”技术来实现传输的可靠性。TCP 还采用一种称为“滑动窗口”的方式进行流量控制，所谓窗口实际表示接收能力，用以限制发送方的发送速度。

3) UDP 协议

UDP 是面向无连接的通讯协议，UDP 数据包括目的端口号和源端口号信息，由于通讯不需要连接，所以可以实现广播发送。

UDP 通讯时不需要接收方确认，属于不可靠的传输，可能会出现丢包现象，实际应用中要求程序员编程验证。

UDP 与 TCP 位于同一层，但 UDP 不管数据包的顺序、错误或重发。

4) HTTP 和 HTTPS 协议

HTTP 协议是 Hyper Text Transfer Protocol（超文本传输协议）的缩写，是用于从万维网（WWW:World Wide Web）服务器传输超文本到本地浏览器的传送协议。

HTTP 是一个基于 TCP/IP 通信协议来传递数据（HTML 文件、图片文件、查询结果等）。

HTTPS 协议是 HyperText Transfer Protocol Secure（超文本传输安全协议）的缩写，是一种通过计算机网络进行安全通信的传输协议。HTTPS 经由 HTTP 进行通信，但利用 SSL/TLS 来加密数据包，HTTPS 开发的主要目的，是提供对网站服务器的身份认证，保护交换资料的隐私与完整性。

2.2 W5500 芯片

要进行通讯，需要相应的硬件支持，在嵌入式应用领域，应用最广泛的一个以太网芯片就是 W5500，素有以太之王的称号。

官方网址：<https://www.w5500.com/index.html>

2.2.1 W5500 芯片介绍

是韩国半导体公司 WIZnet 提供的一款高性价比的以太网芯片。其全球独一无二的**全硬件 TCP/IP 协议栈专利技术**，解决了**嵌入式以太网**的接入问题，简单易用，安全稳定，是物联网设备的首选解决方案。

W5500 集成了 TCP/IP 协议栈，10/100M 以太网数据链路层（MAC）及物理层（PHY），使得用户使用单芯片就能够在他们的应用中拓展网络连接。

久经市场考验的 WIZnet 全硬件 TCP/IP 协议栈支持 TCP，UDP，IPv4，ICMP，ARP，IGMP 以及 PPPoE 协议。W5500 内嵌 32K 字节片上缓存以供以太网包处理。如果你使用 W5500，你只需要一些简单的 Socket 编程就能实现以太网应用。这将会比其他嵌入式以太网方案更加快捷、简便。

用户可以同时使用 **8 个硬件 Socket 独立通讯**。W5500 提供了 SPI（外设串行接口）从而能够更加容易与外设 MCU 整合。而且，W5500 的使用了新的高效 SPI 协议支持 80MHz 速率，从而能够更好的实现高速网络通讯。为了减少系统能耗，W5500 提供了网络唤醒模式（WOL）及掉电模式供客户选择使用。

2.2.2 W5500 芯片特点

- （1）支持硬件 TCP/IP 协议：TCP，UDP，ICMP，IPv4，ARP，IGMP，PPPoE。
- （2）支持 8 个独立端口（Socket）同时通讯。
- （3）支持掉电模式。

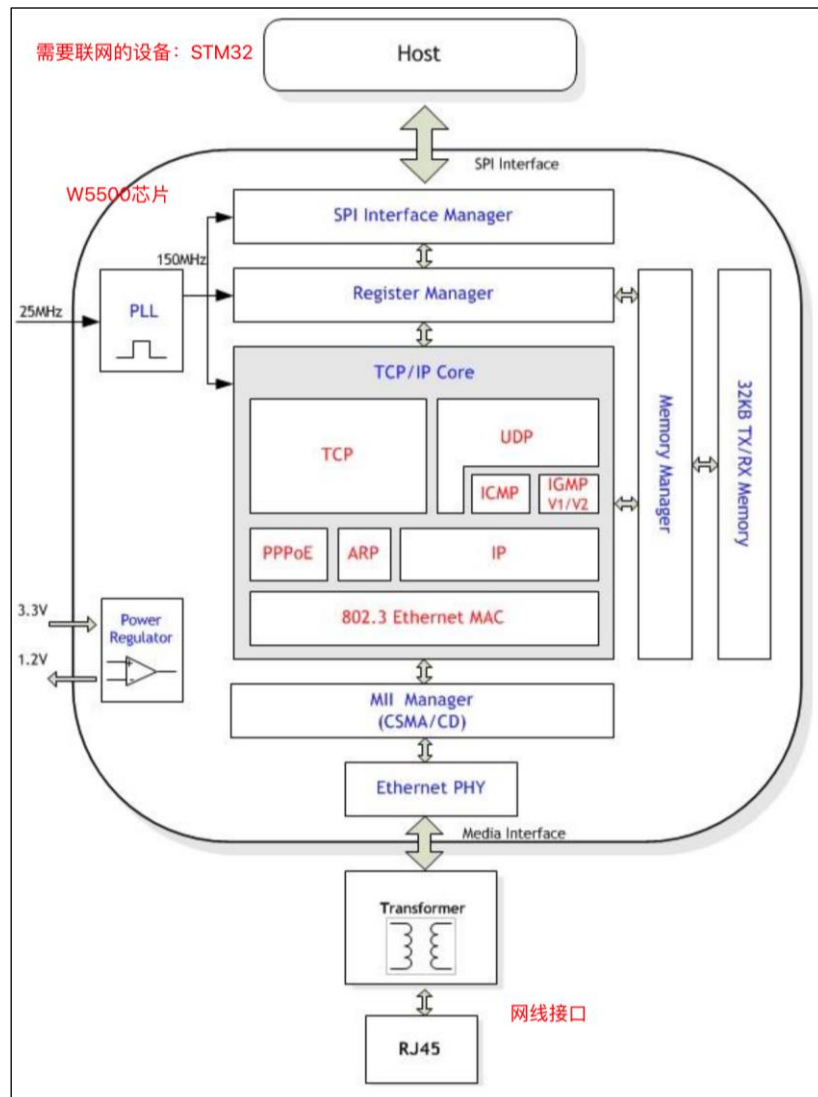
- (4) 支持网络唤醒。
- (5) 支持高速串行外设接口（SPI 模式 0, 3）。
- (6) 内部 32K 字节收发缓存。
- (7) 内嵌 10BaseT/100BaseTX 以太网物理层（PHY）。
- (8) 支持自动协商（10/100-Based 全双工/半双工）。
- (9) 不支持 IP 分片。
- (10) 3.3V 工作电压，I/O 信号口 5V 耐压。
- (11) LED 状态显示（全双工/半双工，网络连接，网络速度，活动状态）。
- (12) LQFP48 无铅封装（7x7mm，间距 0.5mm）

2.2.3 应用目标

W5500 适合于以下嵌入式应用：

- (1) 家庭网络设备：机顶盒、个人录像机、数码媒体适配器。
- (2) 串行转以太网：门禁控制、LED 显示屏、无线 AP 继电器等。
- (3) 并行转以太网：POS/微型打印机、复印机。
- (4) USB 转以太网：存储设备、网络打印机。
- (5) GPIO 转以太网：家庭网络传感器。
- (6) 安全系统：数字录像机、网络摄像机、信息亭。
- (7) 工厂和楼宇自动化控制系统。
- (8) 医疗监测设备。
- (9) 嵌入式服务器。

2.2.4 接入框图



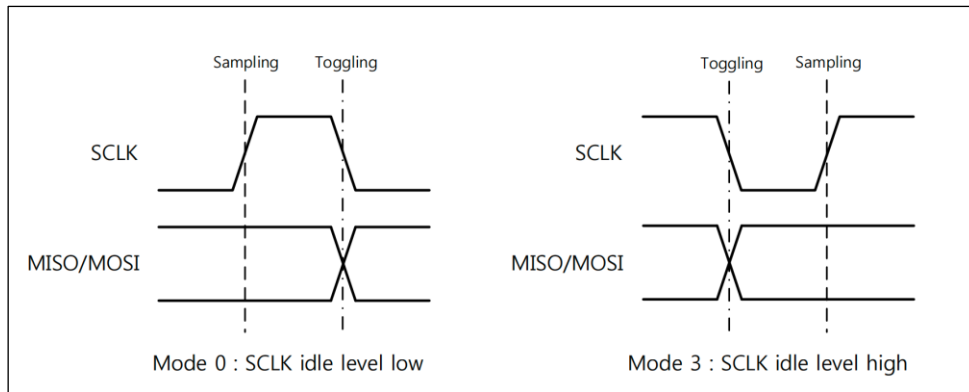
2.2.5 主控芯片与 W5500 交互

1) SPI 连接

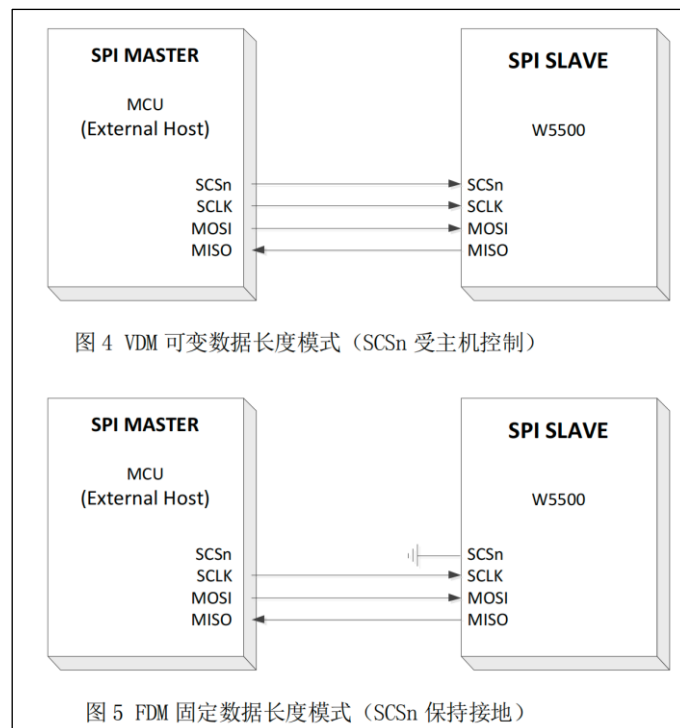
W5500 提供了 SPI (串行外部接口) 作为外设主机接口, 有 SCSn、SCLK、MOSI、MISO 共 4 路信号, 且作为 SPI 从机工作。

在 W5500 中只支持工作模式 0 和 3, 在这两种模式下数据总是在 SCLK 信号的上升沿被锁存, 在 SCLK 信号的下降沿被输出。

MOSI 和 MISO 信号无论是接收或发送, 均遵从最高标志位 (MSB) 到最低标志位 (LSB) 的传输序列。



2) 固定数据长度模式和可变数据长度模式



如果 SPI 工作模式设置为可变数据长度模式 (**VDM**)，SPI 的 SCSn 信号需要由外部主机通过 SPI 帧控制。

在可变数据长度模式下，SCSn 控制 SPI 帧的开始和停止：**SCSn 信号拉低**（高电平到低电平），即代表 W5500 的 **SPI 帧开始**（地址段）；**SCSn 信号拉高**（低电平到高电平），即代表 W5500 的 **SPI 帧结束**（数据段的随机 N 字节数据结尾）；

在我们的电路图设计中，ScSn 接的是片选信号，所以我们应该选择**可变数据长度模式**。

3) W5500 的内部存储器

(1) 1 个普通寄存器 block：这里配置了 W5500 的一些基本信息，如网络配置（IP，MAC 地址，Socket 中断配置等）。

(4) 8 个 Socket 对应的**发送缓冲寄存器** block (共 16k)。

(1) **W5500-RST**: 重置硬件，重置 (Reset) 低电平有效；该引脚需要保持低电平至少 500 us，才能重置 W5500；（正常使用应该高电平，需要重置芯片的时候置为低电平不少 500us）。连接的是 PG7。

(2) W5500-INT: 中断输出 (Interrupt output) 低电平有效; 低电平: W5500 的中断生效。高电平: 无中断; 连接的是 PG6。

(3) W5500-CS 片选引脚。连接的是 PD3

(4) 连接的是 STM32 的 SPI2 外设。

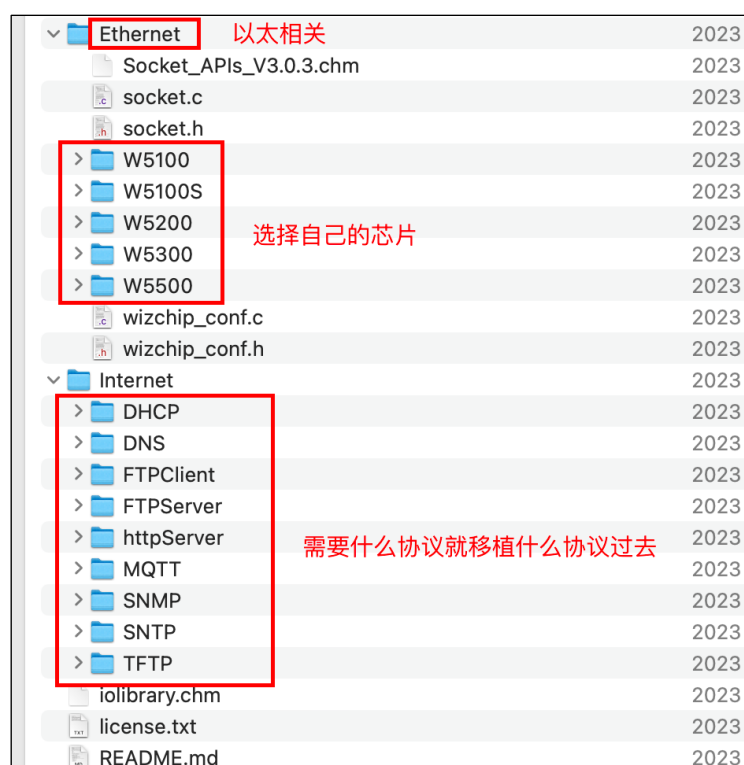
2.3.3 W5500 官方库的移植

官方库地址: https://gitcode.net/mirrors/Wiznet/ioLibrary_Driver

2.3.3.1 下载 W5500 官方库



2.3.3.2 官方库的结构



2.3.3.3 创建 Keil 项目

Copy 以前我们编写的基于寄存器版本的 SPI 程序。

2.3.3.4 移植我们需要的

在 hardware 下创建一个目录 w5500，然后复制我们需要的。暂时先 copy 这么多，以后可以根据需要再添加。

	dhcp.c	dhcp	2023/2/17 21:27	C 文件	30 KB
	dhcp.h		2023/2/17 21:27	H 文件	6 KB
	dns.c	dns	2023/2/17 21:27	C 文件	15 KB
	dns.h		2023/2/17 21:27	H 文件	5 KB
	httpParser.c		2023/2/17 21:27	C 文件	10 KB
	httpParser.h		2023/2/17 21:27	H 文件	6 KB
	httpServer.c	httpserver	2023/9/18 13:41	C 文件	22 KB
	httpServer.h		2023/2/17 21:27	H 文件	4 KB
	httpUtil.c		2023/2/17 21:27	C 文件	2 KB
	httpUtil.h		2023/2/17 21:27	H 文件	1 KB
	socket.c	socket	2023/9/17 18:22	C 文件	29 KB
	socket.h		2023/2/17 21:27	H 文件	28 KB
	w5500.c		2023/2/17 21:27	C 文件	9 KB
	w5500.h		2023/9/18 13:56	H 文件	81 KB
	wizchip_conf.c		2023/9/17 11:21	C 文件	25 KB
	wizchip_conf.h		2023/9/17 10:40	H 文件	26 KB

2.3.3.5 修改 wizchip_conf.h

找到宏定义 _WIZCHIP_，如果不是 W5500，就改成 W5500。

```
#ifndef _WIZCHIP_
#define _WIZCHIP_ W5100, W5100S, W5200, W5300, W5500
#endif
```

修改工作模式为可变数据长度模式（大致在 155 行）。

```
#elif (_WIZCHIP_ == W5500)
#define _WIZCHIP_ID_ "W5500\0"

/**
 * @brief Define interface mode. \n
 * @todo Should select interface mode as chip.
 *      - @ref \_WIZCHIP_IO_MODE_SPI_ \n
 *      -@ref \_WIZCHIP_IO_MODE_SPI_VDM_ : Valid only in @ref \_WIZCHIP_ ==
W5500 \n
 *      -@ref \_WIZCHIP_IO_MODE_SPI_FDM_ : Valid only in @ref \_WIZCHIP_ ==
W5500 \n
 *      - @ref \_WIZCHIP_IO_MODE_BUS_ \n
 *      - @ref \_WIZCHIP_IO_MODE_BUS_DIR_ \n
 *      - @ref \_WIZCHIP_IO_MODE_BUS_INDIR_ \n
 *      - Others will be defined in future. \n\n
 *      ex> <code> #define \_WIZCHIP_IO_MODE_ \_WIZCHIP_IO_MODE_SPI_VDM_ </code>
 *
 */
#endif
// #define _WIZCHIP_IO_MODE_ _WIZCHIP_IO_MODE_SPI_FDM_
```

```
#define _WIZCHIP_IO_MODE_ _WIZCHIP_IO_MODE_SPI_VDM_
```

2.3.3.6 注册函数

(1) 函数说明

wizchip_conf.c 文件中，官方提供了一些接口，待用户补充。

```
void    wizchip_cris_enter(void) {} // 进入临界区(没有上系统,默认即可)
void    wizchip_cris_exit(void) {} // 退出临界区(没有上系统,默认即可)
void    wizchip_cs_select(void) {} // 片选使能
void    wizchip_cs_deselect(void) {} // 片选失能
iodata_t wizchip_bus_readdata(uint32_t AddrSel) { return * ((volatile iodata_t *)((ptrdiff_t) AddrSel)); } // 总线读函数
void    wizchip_bus_writedata(uint32_t AddrSel, iodata_t wb) { *((volatile iodata_t*)((ptrdiff_t)AddrSel)) = wb; } // 总线写函数
uint8_t wizchip_spi_readbyte(void) {return 0;} // 读一个字节
void    wizchip_spi_writebyte(uint8_t wb) {} // 写一个字节
void    wizchip_spi_readburst(uint8_t* pBuf, uint16_t len) {} //按长度读
void    wizchip_spi_writeburst(uint8_t* pBuf, uint16_t len) {} // 按长度写
```

上面的函数只是些空实现，关键的是下面的注册函数。

```
//注册进入/退出临界区函数
void reg_wizchip_cris_cbfunc(void(*cris_en)(void), void(*cris_ex)(void))
//注册 SPI 片选(CS)使能/失能函数
void reg_wizchip_cs_cbfunc(void(*cs_sel)(void), void(*cs_desel)(void))
//注册总线 读/写 函数
void reg_wizchip_bus_cbfunc(iodata_t(*bus_rb)(uint32_t addr), void (*bus_wb)(uint32_t addr, iodata_t wb))
//注册 SPI 按字节 读/写 函数
void reg_wizchip_spi_cbfunc(uint8_t (*spi_rb)(void), void (*spi_wb)(uint8_t wb))
//注册 SPI 按长度 读/写函数
void reg_wizchip_spiburst_cbfunc(void (*spi_rb)(uint8_t* pBuf, uint16_t len), void (*spi_wb)(uint8_t* pBuf, uint16_t len))
```

我们只关注 3 个就可以：注册临界区进入/退出，注册使能/失能，注册字节读/写。临界区进入和退出函数可以不用实现（如果上了嵌入式系统才需要关注）。（其他的根据需要完成也可以）。

首先在 wizchip_conf.c 中添加包含文件 `include "spi.h"`。

(2) 实现片选使能和失能函数

```
void wizchip_cs_select(void)
{
    GPIOD->ODR &= ~GPIO_ODR_ODR3;
}
void wizchip_cs_deselect(void)
{

```

```

    GPIOD->ODR |= GPIO_ODR_ODR3;
}

```

(3) 实现读写字节函数

```

uint8_t wizchip_spi_readbyte(void)
{
    return SPI_SwapByte(0xFF);
}

void wizchip_spi_writebyte(uint8_t wb)
{
    SPI_SwapByte(wb);
}

```

(4) 添加一个封装注册功能的函数

```

void user_register_function(void)
{
    reg_wizchip_cris_cbfunc(wizchip_cris_enter, wizchip_cris_exit);

    reg_wizchip_cs_cbfunc(wizchip_cs_select, wizchip_cs_deselect);

    reg_wizchip_spi_cbfunc(wizchip_spi_readbyte, wizchip_spi_writebyte);
}

```

并在 wizchip_conf.h 中添加函数 user_register_function 的声明。

```

void user_register_function(void);

```

我们在初始化网络的时候可以调用这个函数。

2.3.4 软件设计

2.3.4.1 spi.h

```

#ifndef __SPI_H
#define __SPI_H

#include "usart.h"
#include "stm32f10x.h"
#include "delay.h"

void SPI_Init(void);

void SPI_Start(void);

void SPI_Stop(void);

uint8_t SPI_SwapByte(uint8_t byte_sended);

#endif

```

2.3.4.2 spi.c

```
#include "spi.h"

/* cs: PD3 在使用引脚 SPI 的时候，片选信号我们仍然手动控制*/
#define CS_HIGH() (GPIOC->ODR |= GPIO_ODR_ODR3)
#define CS_LOW() (GPIOC->ODR &= ~GPIO_ODR_ODR3)

/**
 * @description: 使能时钟和 spi 用到引脚实现初始化
 * @return {*}
 */
void SPI_Init(void)
{
    /* 1. 使能 SPI2 时钟 和 GPIOB GPIOC GPIOG 时钟*/
    RCC->APB1ENR |= RCC_APB1ENR_SPI2EN;
    RCC->APB2ENR |= RCC_APB2ENR_IOPBEN;
    RCC->APB2ENR |= RCC_APB2ENR_IOPDEN;
    RCC->APB2ENR |= RCC_APB2ENR_IOPGEN;

    /* 2. GPIOA 的配置 */
    /* 2.1. 设置 PD3 的工作模式 使用通用推挽输出 MODE=11 CNF=00*/
    GPIOC->CRL |= GPIO_CRL_MODE3;
    GPIOC->CRL &= ~GPIO_CRL_CNF3;
    /* 2.2. 配置 CLK(PB13) MOSI(PB15) 为复用推挽输出 MODE=11 CNF=10*/
    GPIOB->CRH |= (GPIO_CRH_MODE13 | GPIO_CRH_MODE15);
    GPIOB->CRH |= (GPIO_CRH_CNF13_1 | GPIO_CRH_CNF15_1);
    GPIOB->CRH &= ~(GPIO_CRH_CNF13_0 | GPIO_CRH_CNF15_0);
    /* 2.3. 配置 MISO(PB14) 使用浮空输入 MODE=00 CNF=01 */
    GPIOB->CRH &= ~GPIO_CRH_MODE14;
    GPIOB->CRH &= ~GPIO_CRH_CNF14_1;
    GPIOB->CRH |= GPIO_CRH_CNF14_0;

    /* 3. SPI2 相关配置 */
    /* 3.1 配置主模式 0=从设备 1=主设备*/
    SPI2->CR1 |= SPI_CR1_MSTR;
    /* 3.2 选择分频系数 000=fpcclk/2, 001=fpcclk/4 ....,我们选择 4 分频 */
    SPI2->CR1 &= ~SPI_CR1_BR;
    SPI2->CR1 |= SPI_CR1_BR_0;
    /* 3.3 时钟极性 0: 空闲状态时, SCK 保持低电平; 1: 空闲状态时, SCK 保持高电平。 */
    SPI2->CR1 &= ~SPI_CR1_CPOL;
    /* 3.4 时钟相位 0: 数据采样从第一个时钟边沿开始; 1: 数据采样从第二个时钟边沿开始。 */
    /*
    SPI2->CR1 &= ~SPI_CR1_CPHA;

    /* 3.5 数据帧格式 0: 8 位数据帧; 1: 16 位数据帧 */

```

```

SPI2->CR1 &= ~SPI_CR1_DFF;
/* 3.6 数据传输顺序 0: 先发送 MSB; 1: 先发送 LSB。 */
SPI2->CR1 &= ~SPI_CR1_LSBFIRST; /* 高位先行 */
/* 3.7 使用软件实现片选（我们自 7 已控制 CS）0: 禁止软件从设备管理; 1: 启用软件从设备管理。 */
SPI2->CR1 |= SPI_CR1_SSM;
/* 3.8 当 SSM=1 时, 这个时候对 SSI 的操作无效, 必须设置为 1 */
SPI2->CR1 |= SPI_CR1_SSI;
/* 4 启动 SPI2 0: 禁止 SPI 设备; 1: 开启 SPI 设备。 */
SPI2->CR1 |= SPI_CR1_SPE;
}
/**
 * @description: SPI 通讯开始。 片选信号置低
 */
void SPI_Start(void)
{
    CS_LOW();
}

/**
 * @description: SPI 通讯结束。片选信号置高
 */
void SPI_Stop(void)
{
    CS_HIGH();
}

/**
 * @description: 使用 SPI 交换一个直接的数据
 * @param {uint8_t} byte_sended 要发送的字节数据
 * @return {uint8_t} 读到字节数据
 */
uint8_t SPI_SwapByte(uint8_t byte_sended)
{
    /* 1. 等待发送缓冲区为空 */
    while (!(SPI2->SR & SPI_SR_TXE))
        ;
    /* 2. 把数据写入到数据寄存器 */
    SPI2->DR = byte_sended;
    /* 3. 等待接收缓冲区非空 */
    while (!(SPI2->SR & SPI_SR_RXNE))
        ;
    /* 4. 返回读到的字节数据 */
    return SPI2->DR;
}

```

```
}
```

2.3.4.3 eth.h

```
#ifndef __ETH_H
#define __ETH_H
#include "stm32f10x.h"
#include "w5500.h"
#include "delay.h"
#include "usart.h"
#include "spi.h"
void ETH_Init(void);

void ETH_Reset(void);

void ETH_SetMac(void);

void ETH_SetIP(void);

#endif
```

2.3.4.4 eth.c

```
#include "eth.h"

// 这些参数务必要参考电脑的 ip 设置
uint8_t mac[6] = {110, 120, 130, 140, 150, 160}; // mac 地址 (随意)
uint8_t ip[4] = {192, 168, 11, 222}; // ip 地址(前三段和电脑保持一致)
uint8_t sub[4] = {255, 255, 255, 0}; // 子网掩码
uint8_t gw[4] = {192, 168, 11, 1}; // 默认网关(和电脑保持一致)
void ETH_Init(void)
{
    /* 0. 初始化 SPI2*/
    SPI_Init();
    // 1: 注册函数
    user_register_function();
    // 2. 复位 w500
    ETH_Reset();
    // 3. 设置 mac 地址
    ETH_SetMac();
    // 4. 设置 ip 地址
    ETH_SetIP();
}

/*
对 w5500 进行复位
*/
```

```

void ETH_Reset(void)
{
    RCC->APB2ENR |= RCC_APB2ENR_IOPGEN;
    /* PG7 重置引脚 使用通用推挽输出 MODE=11 CNF=00*/
    GPIOG->CRL |= GPIO_CRL_MODE7;
    GPIOG->CRL &= ~GPIO_CRL_CNF7;

    // 给 w5500 的芯片设置一个至少 500ms 的低电平就可复位完成
    GPIOG->ODR &= ~GPIO_ODR_ODR7;
    Delay_ms(800); // 延迟 800ms
    GPIOG->ODR |= GPIO_ODR_ODR7;
    printf("reset 执行完毕 \r\n");
}

/*
设置 mac 地址
*/
void ETH_SetMac(void)
{
    // 设置 mac 地址
    // 参数是 6 个字节的 mac 地址组成的数组
    printf("开始设置 mac 地址 \r\n");
    setSHAR(mac);
    printf("完成设置 mac 地址 \r\n");
}

/*
设置 ip 地址
*/
void ETH_SetIP(void)
{
    printf("开始设置 ip 地址 \r\n");
    // 设置 ip 地址
    setSIPR(ip);
    printf("完成设置 ip 地址 \r\n");
    // 设置子网掩码
    printf("开始设置子网掩码 \r\n");
    setSUBR(sub);
    printf("完成设置子网掩码 \r\n");
    // 配置默认网关
    printf("开始设置网关 \r\n");
    setGAR(gw);
    printf("完成设置网关 \r\n");
}

```

```
    printf("ip 地址: %d.%d.%d.%d \r\n", ip[0], ip[1], ip[2], ip[3]);  
}
```

2.3.4.5 main 函数

```
#include "stm32f10x.h" // Device header  
#include "usart.h"  
#include "delay.h"  
#include "string.h"  
#include "eth.h"  
  
uint8_t r_buff[100] = {0};  
int main(void)  
{  
    usart1_init();  
    printf("尚硅谷以太网实验: 测试 IP 连通 \r\n");  
    printf("开始初始化以太网.... \r\n");  
    ETH_Init();  
    printf("初始化以太网完成.... \r\n");  
}
```

2.3.4.6 测试

(1) 接网线

有两种接法

- 让网线插入路由器或者交换机。
- 也可以单片机和电脑直接互联。

设置 IP 的时候务必需要注意。前三段必须一样。

(2) 验证是否能 ping 通

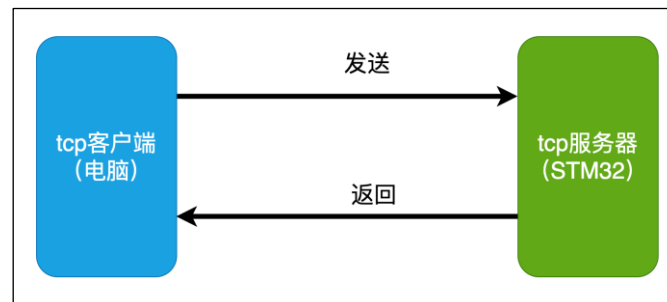
```
PS C:\Users\lzc> ping 192.168.11.222  
  
正在 Ping 192.168.11.222 具有 32 字节的数据:  
来自 192.168.11.222 的回复: 字节=32 时间=9ms TTL=128  
来自 192.168.11.222 的回复: 字节=32 时间=7ms TTL=128  
来自 192.168.11.222 的回复: 字节=32 时间=4ms TTL=128  
来自 192.168.11.222 的回复: 字节=32 时间=64ms TTL=128  
  
192.168.11.222 的 Ping 统计信息:  
    数据包: 已发送 = 4, 已接收 = 4, 丢失 = 0 (0% 丢失),  
    往返行程的估计时间(以毫秒为单位):  
        最短 = 4ms, 最长 = 64ms, 平均 = 21ms  
PS C:\Users\lzc> |
```

2.4 以太网通讯案例 2：搭建 TCP 服务端

2.4.1 需求描述

在 TCP 通讯的时候，客户端必须主动联系服务器，这样才能实现通讯。服务器与客户端之间的连接是一种长连接，一旦连上是不会断开的。

在 STM32 上启动一个 TCP 的服务端，在电脑上用 TCP 客户端去连接服务端.客户端给服务端发送数据后,服务端再原封不动的返回给客户端。



2.4.2 软件设计

2.4.2.1 复制上一个工程

给文件夹也改个名字。

2.4.2.2 main.c

```
#include "stm32f10x.h" // Device header
#include "usart.h"
#include "delay.h"
#include "string.h"
#include "eth.h"
#include "tcp_server.h"

uint8_t r_buff[100] = {0};
int main(void)
{
    usart1_init();
    printf("尚硅谷以太网实验: TCP server \r\n");
    printf("开始初始化以太网.... \r\n");
    ETH_Init();
    printf("初始化以太网完成.... \r\n");

    while (1)
    {
        TCP_Server_Socket0();
    }
}
```

2.4.2.3 tcp_server.h

```
#ifndef __TCP_SERVER_H
```

```

#define __TCP_SERVER_H
#include "usart.h"
#include "w5500.h"
#include "socket.h"

void TCP_Server_Socket0(void);
#endif

```

2.4.2.4 tcp_server.c

```

#include "tcp_server.h"
#include "delay.h"
// 接收数据的缓冲区
uint8_t rBuff[2048] = {0};
int8_t listenStatus;
uint16_t rDataLength;
uint8_t clientIp[4];
uint16_t clientPort;

/**
 * @description: 监听本机的 socket0
 */
void TCP_Server_Socket0(void)
{
    /* 0. socket0 中断使能 */
    // setSIMR(0);
    /* 1. 获取 Socket0 的状态 */
    uint8_t socketStatus = getSn_SR(0);
    switch (socketStatus) // 获取 socket 0 的状态
    {
        /* 2. 表示 socket 0 已经打开, 处于 TCP 工作模式, 可以监听或连接 */
        case SOCK_INIT:

            /*
             * 使用 socket 0 监听来自客户端的请求
             */
            listenStatus = listen(0);
            if (listenStatus == SOCK_OK)
            {
                printf("socket 0 监听成功....\r\n");
            }
            else if (listenStatus == SOCKERR_SOCKINIT)
            {
                printf("socket 0 没有初始化....\r\n");
            }
            else if (listenStatus == SOCKERR_SOCKCLOSED)

```

```

{
    printf("socket 0 已经关闭....\r\n");
}
else
{
    printf("socket 0 未知错误....\r\n");
}
break;
/* 3. 表示连接已建立, 可以接收数据或发送数据了 */
case SOCK_ESTABLISHED:
    /* 获取客户端 ip 和端口 */
    getSn_DIPR(0, clientIp);
    clientPort = getSn_DPORT(0);
    printf("客户端 %d.%d.%d.%d: %d 成功建立连接...\r\n",
        clientIp[0],
        clientIp[1],
        clientIp[2],
        clientIp[3],
        clientPort);
    while (1) // 循环读取数据
    {
        /* 等待接收数据。 Sn_IR(RECV) 中断无论何时, 只要接收到了对方数据, 此位置 1 */
        while ((getSn_IR(0) & Sn_IR_RECV) == 0)
        {
            // 在等待连接期间, socket 状态 != SOCK_ESTABLISHED, 则表示 socket 发生了
意外, 就关闭 socket 0, 并退出函数
            if (getSn_SR(0) != SOCK_ESTABLISHED)
            {
                printf("socket 0 发生意外, 将重新打开 socket 0 ...\r\n");
                close(0);
                return;
            }
        }
        /* 把 Sn_IR_RECV 置 1 (注意是置 1 而不是置 0), 清除接收中断位, 用于下次接收 */
        setSn_IR(0, getSn_IR(0) & Sn_IR_RECV);
        /* 获取接收到数据的长度 */
        rDataLength = getSn_RX_RSR(0);
        if (rDataLength > 0)
        {
            /* 接收数据 */
            recv(0, rBuff, rDataLength);
            printf("接收到的数据长度是: %d, 内容是: %s", rDataLength, rBuff);
            /* 接收到数据返回给发送方 */
            send(0, rBuff, rDataLength);
        }
    }
}

```

```

    }
}

/* 4. socket 处于关闭状态状态，资源已经被释放。 在这里我们可以重新打开 socket */
case SOCK_CLOSED:
/* 并打开 socket
    参数 1 : socket 编号,
    参数 2 : 协议
    参数 3 : 端口
    参数 4 : sock 的 flag(无延时响应) *
    如果打开成功,则返回 socket 编号
*/

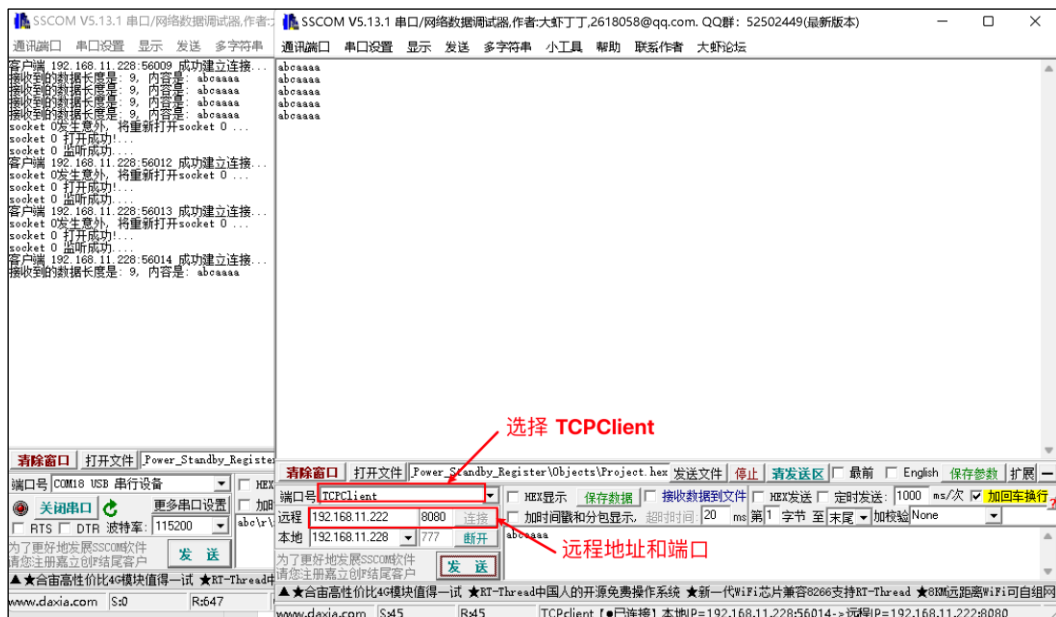
if (socket(0, Sn_MR_TCP, 8080, SF_TCP_NODELAY) == 0)
{
    printf("socket 0 打开成功!... \r\n");
}
else
{
    printf("socket 0 打开失败!... \r\n");
}

break;

default:
// printf("socket 0 的其他状态... %x \r\n", socketStatus);
break;
}
}

```

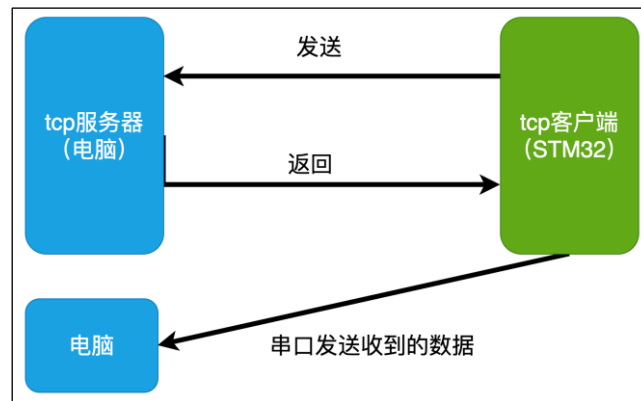
2.4.2.5 测试



2.5 以太网通讯案例 3：搭建 TCP 客户端

2.5.1 需求描述

在 STM32 上启动一个 TCP 的客户端，去连接电脑端的 TCP 服务端。STM32 客户端主动给电脑服务端发送数据。然后电脑再给客户端回数据，再通过串口展示客户端接收到的数据。



2.5.2 软件设计

2.5.2.1 main.c

```
#include "stm32f10x.h" // Device header
#include "usart.h"
#include "delay.h"
#include "string.h"
#include "eth.h"
#include "tcp_client.h"

uint8_t r_buff[100] = {0};
int main(void)
{
    usart1_init();
    printf("尚硅谷以太网实验: TCP server \r\n");
    printf("开始初始化以太网.... \r\n");
    ETH_Init();
    printf("初始化以太网完成.... \r\n");

    while (1)
    {
        TCP_Client_Socket0();
    }
}
```

2.5.2.2 tcp_client.h

```
#ifndef __TCP_CLIENT_H
#define __TCP_CLIENT_H

#include "usart.h"
#include "w5500.h"
#include "socket.h"

void TCP_Client_Socket0(void);

#endif
```

2.5.2.3 tcp_client.c

```
#include "tcp_client.h"
#include "delay.h"
// 接收数据的缓冲区
uint8_t rBuff[2048] = {0};
int8_t connectStatus;
uint16_t rDataLength;
uint8_t clientIp[4];
uint16_t clientPort;
/* 服务器 ip */
uint8_t serverIp[4] = {192, 168, 11, 228};
/* 服务器端口 */
uint32_t serverPort = 8888;

/**
 * @description: 监听本机的 socket0
 */
void TCP_Client_Socket0(void)
{
    /* 0. socket0 中断使能 */
    // setSIMR(0);
    /* 1. 获取 Socket0 的状态 */
    uint8_t socketStatus = getSn_SR(0);
    switch (socketStatus) // 获取 socket 0 的状态
    {
        /* 2. 表示 socket 0 已经打开, 处于 TCP 工作模式, 可以监听或连接 */
        case SOCK_INIT:

            /*
             作为客户端要主动去连接服务器
            */
            connectStatus = connect(0, serverIp, serverPort);
            if (connectStatus == SOCK_OK)
```

```

{
    printf("连接服务器成功....\r\n");
}
else
{
    /* 连接失败就关闭 socket */
    close(0);
    printf("连接服务器失败....\r\n");
}
break;
/* 3. 表示连接已建立，可以接收数据或发送数据了 */
case SOCK_ESTABLISHED:
    send(0, "Hello server, I am Client!\r\n", 28);
    while (1)
    {

        /* 等待接收数据。Sn_IR(RECV) 中断无论何时，只要接收到了对方数据，此位置 1*/
        while ((getSn_IR(0) & Sn_IR_RECV) == 0)
        {
            // 在等待连接期间，socket 状态 != SOCK_ESTABLISHED，则表示 socket 发生了
意外，就关闭 socket 0,并退出
            if (getSn_SR(0) != SOCK_ESTABLISHED)
            {
                printf("socket 0 发生意外，将重新打开 socket 0 ... \r\n");
                close(0);
                return;
            }
        }
        /* 把 Sn_IR_RECV 置 1(注意是置 1 而不是置 0)，清除接收中断位，用于下次接收 */
        setSn_IR(0, getSn_IR(0) & Sn_IR_RECV);
        /* 获取接收到数据的长度 */
        rDataLength = getSn_RX_RSR(0);
        if (rDataLength > 0)
        {
            /* 接收数据 */
            recv(0, rBuff, rDataLength);
            printf("接收到的数据长度是: %d, 内容是: %s", rDataLength, rBuff);
            /* 接收到数据返回给发送方 */
            send(0, rBuff, rDataLength);
        }
    }

    /* 4. socket 处于关闭状态状态，资源已经被释放。 在这里我们可以重新打开 socket */
case SOCK_CLOSED:
    /* 并打开 socket

```

```

        参数 1 : socket 编号,
        参数 2 : 协议
        参数 3 : 端口
        参数 4 : sock 的 flag(无延时响应) *
        如果打开成功,则返回 socket 编号

    */

    if (socket(0, Sn_MR_TCP, 8080, SF_TCP_NODELAY) == 0)
    {
        printf("socket 0 打开成功!... \r\n");
    }
    else
    {
        printf("socket 0 打开失败!... \r\n");
    }

    break;

default:
    // printf("socket 0 的其他状态... %x \r\n", socketStatus);
    break;
}
}

```

2.5.2.4 测试



2.6 以太网通讯案例 4：UDP 通讯

2.6.1 需求描述

电脑和 STM32 之间实现 UDP 通讯。

UDP 是一种面向无连接的协议，一方发出，如果另外一方正在监听端口，则可以接收到，否则数据就会丢失。

2.6.2 软件设计

2.6.2.1 main 函数

```
#include "stm32f10x.h" // Device header
#include "usart.h"
#include "delay.h"
#include "string.h"
#include "eth.h"
#include "udp.h"

uint8_t r_buff[100] = {0};
int main(void)
{
    usart1_init();
    printf("尚硅谷以太网实验: TCP server \r\n");
    printf("开始初始化以太网.... \r\n");
    ETH_Init();
    printf("初始化以太网完成.... \r\n");

    while (1)
    {
        UDP_Socket0();
    }
}
```

2.6.2.2 udp.h

```
#ifndef __UDP_H
#define __UDP_H
#include "usart.h"
#include "w5500.h"
#include "socket.h"

void UDP_Socket0(void);

#endif
```

2.6.2.3 udp.c

```
#include "udp.h"
#include "delay.h"
// 接收数据的缓冲区
```

```

uint8_t rBuff[2048] = {0};
uint32_t rDataLength;
uint8_t fromAddr[4];
uint16_t fromPort;

/**
 * @description: 监听本机的 socket0
 */
void UDP_Socket0(void)
{
    /* 0. socket0 中断使能 */
    setSIMR(0);
    /* 1. 获取 Socket0 的状态 */
    uint8_t socketStatus = getSn_SR(0);
    switch (socketStatus) // 获取 socket 0 的状态
    {
        /* 2. 表示 socket 0 已经打开, 处于 UDP 工作模式下 */
        case SOCK_UDP:
            while (1)
            {
                while ((getSn_IR(0) & Sn_IR_RECV) == 0)
                {
                    // 在等待连接期间, socket 状态 != SOCK_ESTABLISHED, 则表示 socket 发生了
意外, 就关闭 socket 0, 并退出
                    if (getSn_SR(0) != SOCK_UDP)
                    {
                        printf("socket 0 发生意外, 将重新打开 socket 0 ...\r\n");
                        close(0);
                        return;
                    }
                }
                /* 把 Sn_IR_RECV 置 1(注意是置 1 而不是置 0), 清除接收中断位, 用于下次接收 */
                setSn_IR(0, getSn_IR(0) & Sn_IR_RECV);
                /* 获取接收到数据的长度. 比实际的值大 8 */
                rDataLength = getSn_RX_RSR(0);
                if (rDataLength > 0)
                {
                    /* 接收数据包 */
                    recvfrom(0, rBuff, 2048, fromAddr, &fromPort);
                    printf("接收到的数据长度是: %d, 内容是: %s", rDataLength - 8, rBuff);
                    /* 接收到数据返回给发送方: rDataLength 的值比实际值大 8 */
                    sendto(0, rBuff, rDataLength - 8, fromAddr, fromPort);
                }
            }
        }
    }
}

```

```

/* 4. socket 处于关闭状态状态，资源已经被释放。 在这里我们可以重新打开 socket */
case SOCK_CLOSED:

    if (socket(0, Sn_MR_UDP, 8080, 0) == 0)
    {
        printf("socket 0 打开成功!... \r\n");
    }
    else
    {
        printf("socket 0 打开失败!... \r\n");
    }

    break;

default:
    // printf("socket 0 的其他状态... %x \r\n", socketStatus);
    break;
}
}

```

2.6.2.4 测试



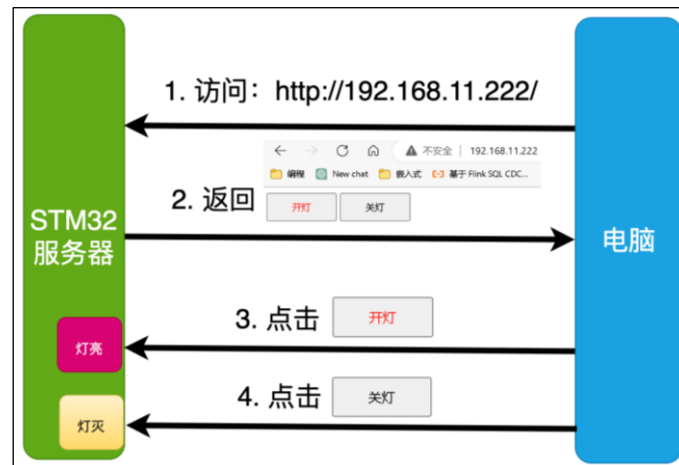
2.7 以太网通讯案例 5：搭建简易版 web 服务器

2.7.1 需求描述

通过网页控制灯的开与关.

实现思路：

- (1) 在 STM32 上开启一个 http 服务器，首页地址 <http://192.168.11.222:80/index.html>
- (2) 当点击开灯按钮的时候，传递参数: <http://192.168.11.222:80/index.html?action=1>
- (3) 当电击关灯按钮的时候，传递参数: <http://192.168.11.222:80/index.html?action=2>
- (4) 收到不同的参数执行不同的操作。



2.7.2 软件设计

2.7.2.1 main 函数

```
#include "stm32f10x.h" // Device header
#include "usart.h"
#include "delay.h"
#include "string.h"
#include "eth.h"
#include "web_server.h"

uint8_t r_buff[100] = {0};
int main(void)
{
    usart1_init();
    printf("尚硅谷以太网实验: TCP server \r\n");
    printf("开始初始化以太网.... \r\n");
    ETH_Init();
    printf("初始化以太网完成.... \r\n");

    WebServer_Init();
    WebServer_Start();
    while (1)
    {

    }
}
```

2.7.2.2 web_server.h

```
#ifndef __UDP_H
#define __UDP_H
#include "usart.h"
#include "w5500.h"
#include "httpserver.h"
#include "httpParser.h"
#include "led.h"
#include "string.h"

void WebServer_Init(void);

void WebServer_Start(void);

#endif
```

2.7.2.3 web_server.c

```
#include "web_server.h"
#include "delay.h"

static uint8_t txBuff[2048] = {0};
static uint8_t rxBuff[2048] = {0};
static uint8_t socketCount = 1;
static uint8_t socketList[] = {0};

static uint8_t *content_name = "index.html";

/* 响应的网页的内容 */
uint8_t content[2048] = "<!doctype html>\n"
    "<html lang=\"en\">\n"
    "<head>\n"
    "    <meta charset=\"GBK\">\n"
    "    <meta name=\"viewport\">\n"
    "        content=\"width=device-width, user-scalable=no,"
initial-scale=1.0, maximum-scale=1.0, minimum-scale=1.0\">\n"
    "    <meta http-equiv=\"X-UA-Compatible\"
content=\"ie=edge\">\n"
    "    <title>尚硅谷嵌入式课程</title>\n"
    "\n"
    "    <style type=\"text/css\">\n"
    "        #open_red{\n"
    "            color: red;\n"
    "            width: 100px;\n"
    "            height: 40px;\n"
```

```

        "\n"
        "\n"
        "        }\n"
        "        #close_red{\n"
        "            color: black;\n"
        "            width: 100px;\n"
        "            height: 40px;\n"
        "        }\n"
        "    </style>\n"
        "</head>\n"
        "<body>\n"
        "<a href=\"/index.html?action=1\"><button id=\"open_red\" >开
灯</button></a>\n"
        "<a href=\"/index.html?action=2\"><button id=\"close_red\" >
关灯</button></a>\n"
        "</body>\n"
        "</html>";

void WebServer_Init(void)
{
    /*
    1. 初始化 http 服务器
    参数 1: 发送缓冲区
    参数 2: 接收缓冲区
    参数 3: 使用的 socket 的个数
    参数 4: 使用的 socket 列表
    */
    httpServer_init(txBuff, rxBuff, socketCount, socketList);

    /* 2. 注册 http 服务器要响应的内容(网页内容) */
    reg_httpServer_webContent(content_name, content);
}

/**
 * @description: 启动 web 服务器
 * @return {*}
 */
void WebServer_Start(void)
{
    while (1)
    {
        httpServer_run(0);
    }
}

```

```

/**
 * @description: http://192.168.11.222/index.html?action=2
                 解析 url 中 action 参数的值
 * @param {uint8_t} *url
 */
uint8_t parse_url_action(uint8_t *url)
{
    uint8_t *params = (uint8_t *)strstr((char *)url, "action=");
    if (params == NULL)
    {
        return '0'; // 没有找到 action
    }
    else
    {
        return (uint8_t)(*(params + 7));
    }
}

/**
 * @description: 根据传入的不同参数执行开灯 or 关灯
 * @param {st_http_request} *parsed_http_request
 * @return {*}
 */
void handle_user_function(uint8_t *url)
{
    uint8_t action = parse_url_action(url);
    if (action == '1')
    { // 是字符 1
        led_on();
    }
    else if (action == '2')
    {
        led_off();
    }
}

```

2.7.2.4 修改 httpServer.c 和 httpServer.h （w5500 官方提供）

需要让框架调用我们上面定义的函数：**handle_user_function()**

在 http_server.c 里面添加一行代码调用 **handle_user_function** 函数。

```

/* 执行用户的逻辑代码 */
handle_user_function(parsed_http_request->URI);

```

大概位置在这里：

```

175 #endif
176 #ifdef _HTTPSERVER_DEBUG_
177     printf("> HTTPSocket[%d] : [State] STATE_HTTP_REQ_DONE\r\n", s);
178 #endif
179 // HTTP 'response' handler; includes send_http_response_header / body function
180 http_process_handler(s, parsed_http_request);
181 /* 执行用户的逻辑代码 */
182 handle_user_function(parsed_http_request->URI);
183 gettime = get_httpServer_timecount();
184 // Check the TX socket buffer for End of HTTP response sends
185 while (getSn_TX_FSR(s) != (getSn_TxMAX(s)))
186 {
187     if ((get_httpServer_timecount() - gettime) > 3)

```

还需要去 httpServer.h 中声明一下这个函数在其他地方定义的

```
extern void handle_user_function(uint8_t *url);
```

```

103 void httpServer_time_handler(void);
104 uint32_t get_httpServer_timecount(void);
105
106 extern void handle_user_function(uint8_t *url);

```

2.7.2.5 测试

- (1) 打开网页: <http://192.168.11.222/>



- (2) 点击开灯测试。
- (3) 点击关灯测试。

第3章 物联网通讯之 WIFI 通讯

3.1 WIFI 基础知识

3.1.1 WiFi 由来

Wi-Fi, 又称“无线网路”, 是 Wi-Fi 联盟的商标, 一个基于 IEEE 802.11 标准的无线局域网技术。“Wi-Fi”常写作“WiFi”或“Wifi”, 但是这些写法并没有被 Wi-Fi 联盟认可。

Wi-Fi 这个术语经常被误以为是指无线保真 (Wireless Fidelity), 类似历史悠久的音频设备分类: 长期高保真 (1930 年开始采用) 或 Hi-Fi (1950 年开始采用)。即便是 Wi-Fi 联盟本身也经常在新闻稿和文件中使用 “Wireless Fidelity” 这个词, Wi-Fi 还出现在 ITAA 的一个论文中。事实上, Wi-Fi 一词是没有任何意义, 也没有全写的。

1999 年, 几家富有远见的公司联合起来组成了一个全球性非营利性协会——无线以太网兼容性联盟 (Wireless Ethernet Compatibility Alliance, WECA), 其目标是使用一种新的无线网络技术, 无论品牌如何, 都能带来最佳的用户体验。在 2000 年, 该小组采用术语 “Wi-Fi” 作为其技术工作的专有名称, 并宣布了正式名称: Wi-Fi Alliance。

3.1.2 WiFi 和 WLAN

WiFi 和 WLAN 经常混淆, 其实他们并不能完全等价。

WLAN: wireless local area network, 无线局域网的全称, 它利用射频技术 Radio Frequency; RF, 使用电磁波构成局域网络, 在空中实现通信。该技术的出现绝不是用来取代有线局域网络, 而是用来弥补有线局域网络之不足。

其实很多时候, 人们将二者混用, 其实实现 WLAN 的方式有很多, wifi 只是实现方式之一。

3.1.3 WiFi 的发展

第一代, 以 IEEE 802.11 原始标准为准, 工作频段为 2.4GHz, 最高速率半双工 2 Mbit/s。

第二代, 以 IEEE 802.11b 为准, 工作频段为 2.4GHz, 最高速率半双工 11 Mbit/s。

第三代, 以 IEEE 802.11a 为准, 工作频段为 5GHz, 最高速率 54 Mbit/s,

第四代, 以 IEEE 802.11n 为准, 称为 “Wi-Fi 4”, 信道宽度 20MHz、40MHz, 工作频段为 2.4GHz 和 5GHz, 最高 4 条空间流, 最大副载波调制 64-QAM, 最高速率半双工 600 Mbit/s。

第五代，以 IEEE 802.11ac 为准，世代名称为“Wi-Fi 5”，信道宽度 20MHz、40MHz、80MHz、80+ 80MHz、160MHz，工作频段为 5GHz，最高 8 条空间流，最大副载波调制 256-QAM，最高速率半双工 6.9 Gbit/s。

第六代，以 IEEE 802.11ax 为准，世代名称为“Wi-Fi 6”，信道宽度 20MHz、40MHz、80MHz、80+ 80MHz、160MHz，工作频段为 2.4GHz 和 5GHz，最高 8 条空间流，最大副载波调制 1024-QAM，最高速率半双工 9.6 Gbit/s。

3.1.4 IEEE 802.11

WIFI 可以看成是 IEEE 802.11 的一种**具体实现**。

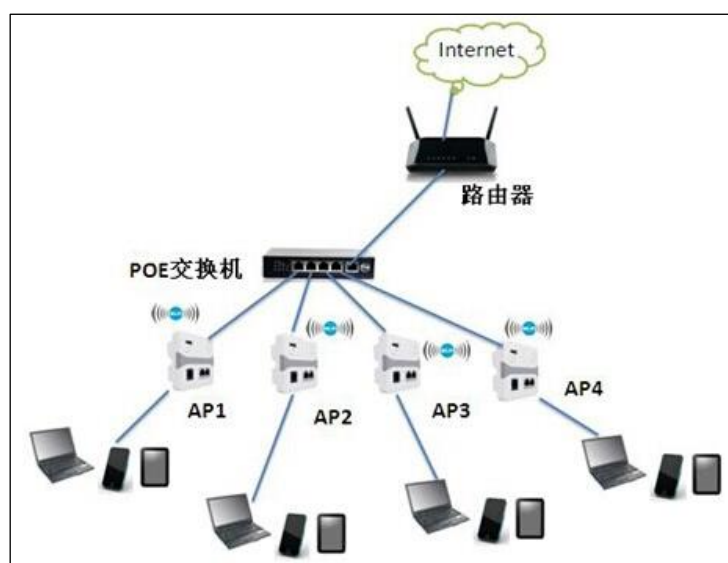
IEEE 802.11 是 IEEE 802 标准委员会（IEEE 802 LAN/MAN Standards Committee）下属的无线局域网工作组，也指代由该组织制定的**无线局域网标准**。

该协议一般与 IEEE 802.2 结合使用，设计目的是为了与以太网无缝互通，经常用于承载 IP 流量。

IEEE 802.11 标准定义一个媒体访问控制（MAC）和几个物理层（PHY）规范，为局域内的固定，便携式和可移动终端（STA）提供无线连接，标准还为监管机构提供了一种标准化方法，对局域网通信的一个或多个频带进行管理。

3.1.5 AP

无线 AP(Wireless Access Point)是“无线接入点”的缩写，单纯性无线 AP 就是一个无线的交换机，仅仅是提供无线信号发射的功能。其工作原理是将网络信号通过双绞线传送过来，经过 AP 产品的编译，将电信号转换成为无线电信号发送出来，形成无线网的覆盖。



家用无线路由器可以看成是一个带路由功能的无线 AP。

3.1.6 WiFi 的频段 5G 和 2.4G

2.4G Wi-Fi 与 5G Wi-Fi 最本质的区别即工作频段（无线电波的频率）不一样，如下表所示，5G Wi-Fi 频段有更高的频率和频宽，可以提供更高的速率和更小的信道干扰。

Wi-Fi	2.4G Wi-Fi	5G Wi-Fi
工作频段	2.4GHz~2.4835GHz	5.150GHz~5.350GHz和5.725GHz~5.850GHz
信道频宽	标准频宽为20MHz，可扩展至40MHz	标准频宽为20MHz，可提高至40/80MHz或更高
速率	可达600MBit/s (802.11n, 4×4 MIMO, 40MHz)	可达1733Mbit/s (802.11ac, 4×4 MIMO, 80MHz)

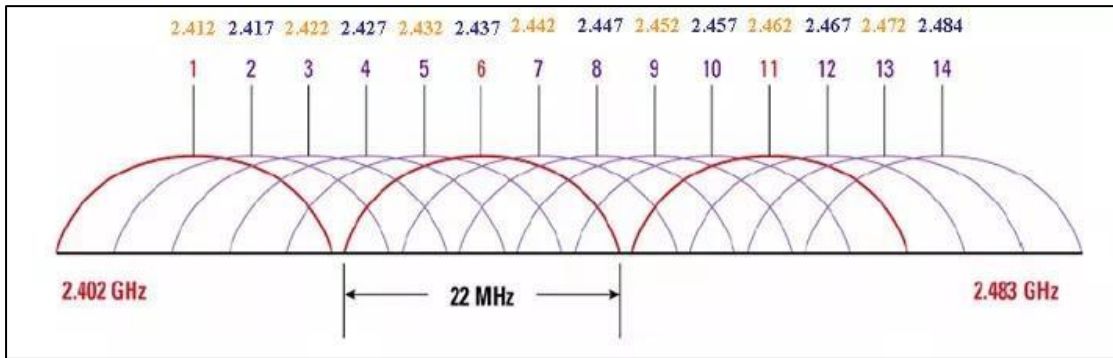
3.1.7 WiFi 的信道

目前用的频段有 2 个 5G 和 2.4G 频段，每个频段又分为若干频道，又称为信道。如果把频段理解为高速公路的话，那么信道可以理解高速公路上的车道。

3.1.8 2.4G 的信道

在标准协议里将 2.4GHz 频段划分出 13 个相互交叠的信道，每个信道的频宽是 20MHz（802.11g、802.11n 每个信道占用 20MHz，802.11b 每个信道占用 22MHz），每个信道都有自己的中心频率。

这 13 个信道可以找出 3 个独立信道，即没有相互交叠的信道。独立信道由于没有频率的交叠区，相邻 AP 使用这 3 个独立信道不会彼此产生干扰。如下图中的 1、6、11 就是三个互不交叠的独立信道。

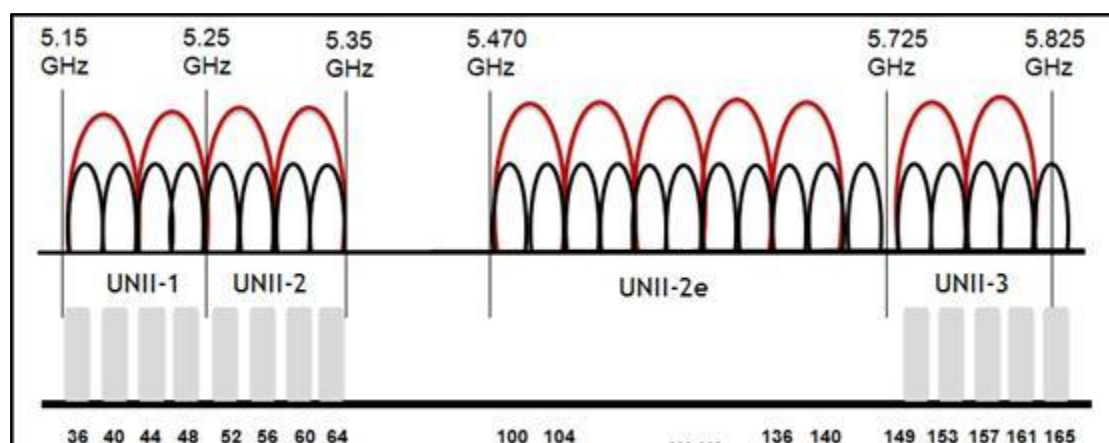


3.1.9 5G Wi-Fi 信道

标准协议将 5GHz 频段分为 24 个 20MHz 宽的信道，且每个信道都为独立信道。这为 WLAN 提供了丰富的信道资源，更多的独立信道也使得信道绑定更有价值，信道绑定是将两个信道绑定成一个信道使用，能提供更大的带宽。如两个 20MHz 的独立信道绑定在一起

可以获得 40MHz 的吞吐量，这好比将两条道路合并成一条使用，自然就提高了道路的通过能力。

下图中，黑色的半圆表示独立信道，红色的半圆表示标准协议推荐的信道绑定。



3.1.10 信道的穿墙能力

5G Wi-Fi 相比 2.4G Wi-Fi **穿墙能力较弱**，这是由电磁波的物理特性决定的：波长越长衰减越少，也更容易绕过障碍物继续传播。

5G 信号频率高、波长短，而 2.4G 信号频率低、波长长，所以 5G 信号穿过障碍物时衰减更大，穿墙能力比 2.4G 信号弱。

3.1.11 一些其他术语

1) STA (Station)

任何一个接入无线 AP 的设备都可以称为一个站点，也就是平时接入路由器的设备。

2) SSID (Service Set Identifier)

每个无线 AP 都应该有一个标示用于用户识别，SSID 就是这个用于用户识别的名字，也就是我们经常说到的 wifi 名。

3) BSSID

每一个网络设备都有其用于识别的物理地址，称作 MAC 地址，一般情况下出厂会有一个默认值，可更改，也有其固定的命名格式，也是设备识别的标识符。BSSID 是针对设备说的，对于 STA 的设备来说，拿到 AP 接入点的 MAC 地址就是这个 BSSID。

4) ESSID

是一个比较抽象的概念，它实际上就和 SSID 相同（本质也是一串字符），只是能如果有好几个无线路由器都叫这个名字，那么我们就相当于把这个 SSID 扩大了，所以这几个无线路由器共同的这个名字就叫 ESSID。

BSSID 就是具体的某个连锁店编号（001）或地址。SSID 就是连锁店的名字或者照片
ESSID 就是连锁店的总公司或者招牌 or 品牌。一般 ssid 和 essid 都是相同的。

3.2 WIFI 模块

SMT32F10x 系统的 MCU 如果想通过 WIFI 联网，必须配备 WFI 模块。（互联型 MCU 内置了 WIFI 外设，不用额外配置 WIFI 模块）。

目前市场上主流的 WiFi 模块有 TI 公司的 CC3200、乐鑫的 ESP8266、联发科的 MT7681。
不同型号的模块在价格、性能和开发难度上都有所不同，下面是对三种型号的 WiFi 模块的对比：

	乐鑫 ESP8266	TICC3200	MT7681
频段	2.4G	2.4G	2.4G
CPU	Tensilica L106（80M）	Arm-Contex-M4（80M）	32-bit RISC MCU
SDK 是否开源	是	是	是
内存	50KB	64KB	64KB
工作电压范围	3.0-3.6V	2.1-3.6V	2.97-3.63V
工作温度	-40-125℃	-40-85℃	-10-70℃
功耗	TX:140mA RX:56mA	TX:229mA RX:59mA	TX:210mA RX:59mA
参考价格	8 元	28 元	16 元

3.2.1 ESP32-C3 模块介绍

我们的开发板选择是乐鑫的芯片：ESP32-C3。相比乐鑫 ESP8266 功能更丰富。
ESP32-C3 是一款安全稳定、低功耗、低成本的物联网芯片，搭载 RISC-V 32 位单核处理器，支持 2.4 GHz **Wi-Fi** 和 **Bluetooth 5**（LE），为物联网产品提供行业领先的射频性能、完善的安全机制和丰富的内存资源。

ESP32-C3 对 Wi-Fi 和 Bluetooth 5（LE）的双重支持降低了设备配网难度，适用于广泛的物联网应用场景。

3.2.2 ESP32-C3 模块的特点

1) 搭载 RISC-V 处理器

ESP32-C3 搭载 RISC-V 32 位单核处理器，时钟频率高达 160 MHz。具有 22 个可编程 GPIO 管脚、内置 400 KB SRAM，支持通过 SPI、Dual SPI、Quad SPI 和 QPI 接口外接多个 flash，满足各类物联网产品功能需求。此外，ESP32-C3 的耐高温特性也使其成为照明和工控领域的理想选择。

2) 行业领先的射频性能

ESP32-C3 集成 2.4 GHz Wi-Fi 和支持长距离的 Bluetooth 5 (LE)，有助于构建覆盖范围更广、射频性能更强的物联网设备。它还支持蓝牙 Mesh (Bluetooth Mesh) 协议和乐鑫 Wi-Fi Mesh，在较高的工作温度下仍能保持卓越的射频性能。

3) 完善的安全机制

ESP32-C3 支持基于 RSA-3072 算法的安全启动和基于 AES-128/256-XTS 算法的 flash 加密功能，确保设备安全连接；创新的数字签名模块和 HMAC 模块确保设备身份安全；支持加密算法的硬件加速器确保设备在本地网络和云上安全传输数据。

4) 成熟的软件支持

ESP32-C3 沿用乐鑫成熟的物联网开发框架 ESP-IDF。ESP-IDF 已成功赋能了数以亿计的物联网设备，并历经了严格的测试和发布周期。开发人员基于其成熟的软件架构，凭借对 API 和工具的熟悉，将更容易构建 ESP32-C3 应用程序或进行程序迁移。ESP32-C3 也支持在**从机模式**下工作，可通过 **ESP-AT 和 ESP-Hosted SDK** 为外部主机 **MCU 提供 Wi-Fi 与 Bluetooth LE 连接功能**。

3.2.3 ESP32-C3 的使用

3.2.3.1 STM32 与 ESP32-C3 连接

ESP-C3 本身就可以作为一个单片机使用。但是我们这里只是把 ESP32-C3 作为一个 wifi/蓝牙模块使用。

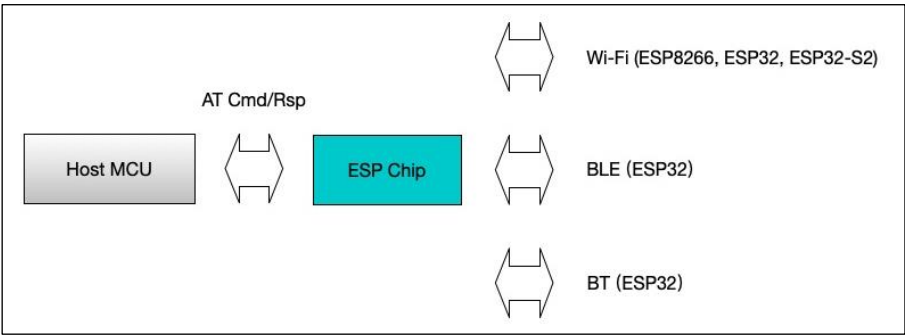
STM32 与 ESP32-C3 使用串口进行通讯。STM32 可以给 ESP32-C3 发送命令，这种命令叫 ESP-AT 指令。

3.2.3.2 ESP-AT 是什么

ESP-AT 是乐鑫开发的可直接用于量产的物联网应用固件，旨在降低客户开发成本，快速形成产品。通过 ESP-AT 指令，您可以快速加入无线网络、连接云平台、实现数据通信以及远程控制等功能，真正的通过无线通讯实现万物互联。

ESP-AT 是基于 ESP-IDF 实现的软件工程。它使 ESP32-C3 模组作为从机，MCU 作为主机。MCU 发送 AT 命令给 ESP32-C3 模组，控制 ESP32-C3 模组执行不同的操作，并接收 ESP32-C3 模组返回的 AT 响应。

ESP-AT 提供了大量功能不同的 AT 命令，如 Wi-Fi 命令、TCP/IP 命令、Bluetooth LE 命令、Bluetooth 命令、MQTT 命令、HTTP 命令、Ethernet 命令等。



AT 命令以 “AT” 开始，代表 Attention，以新的一行 (CR LF) 为结尾。输入的每条命令都会返回 OK 或 ERROR 的响应，表示当前命令的最终执行结果。注意，所有 AT 命令均为串行执行，每次只能执行一条命令。因此，在使用 AT 命令时，应等待上一条命令执行完毕后，再发送下一条命令。如果上一条命令未执行完毕，又发送了新的命令，则会返回 busy p...

默认配置下，MCU 通过 UART 连接至 ESP32-C3 模组、发送 AT 命令以及接收 AT 响应。但是，您也可以根据实际使用情况修改程序，使用其他的通信接口，例如 SDIO。

同样，您也可以基于 ESP-AT 工程，自行开发更多的 AT 命令，以实现更多的功能。

3.2.3.3 ESP32-C3 固件烧录

1) 固件烧录工具下载

下载地址：https://www.espressif.com.cn/zh-hans/support/download/other-tools?keys=&field_type_tid%5B%5D=785



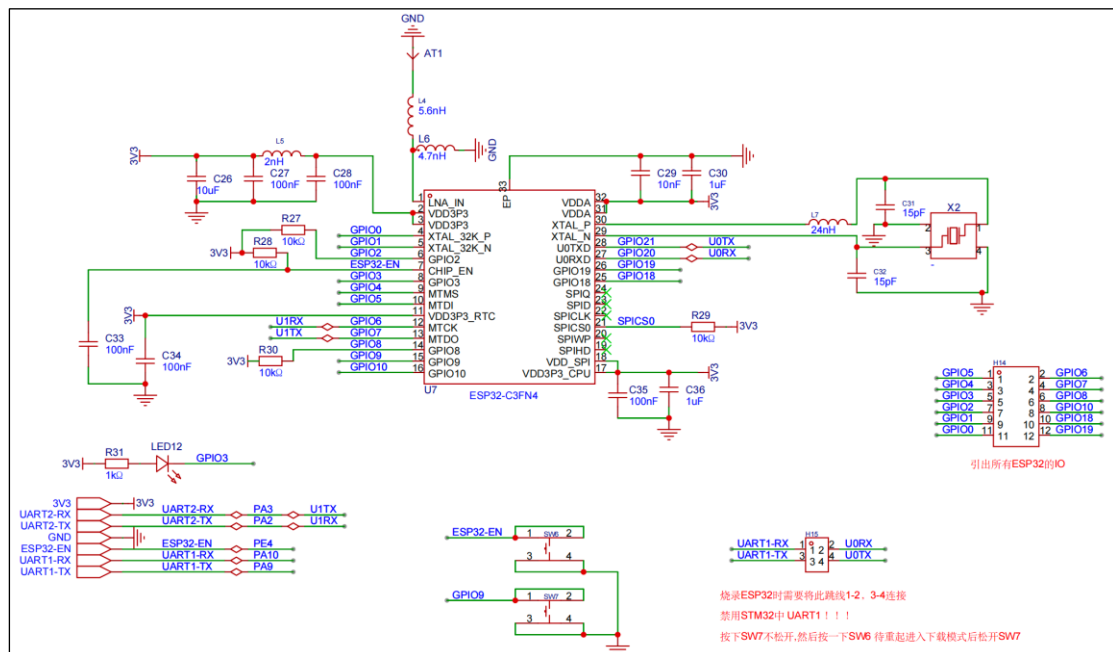
2) 固件下载

下载地址: https://docs.espressif.com/projects/espressif-esp32-c3/latest/esp32c3/AT_Binary_Lists/ESP32-C3_AT_binaries.html

ESP32-C3-MINI-1 系列

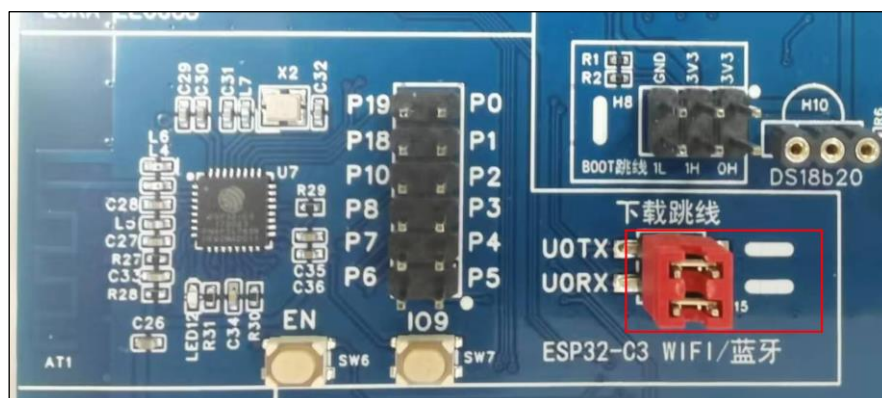
- v3.2.0.0 ESP32-C3-MINI-1-AT-V3.2.0.0.zip (推荐)
- v2.4.2.0 ESP32-C3-MINI-1-AT-V2.4.2.0.zip
- v2.4.1.0 ESP32-C3-MINI-1-AT-V2.4.1.0.zip
- v2.4.0.0 ESP32-C3-MINI-1-AT-V2.4.0.0.zip
- v2.3.0.0 ESP32-C3-MINI-1-AT-V2.3.0.0.zip
- v2.2.0.0 ESP32-C3-MINI-1-AT-V2.2.0.0.zip

3) 硬件连接



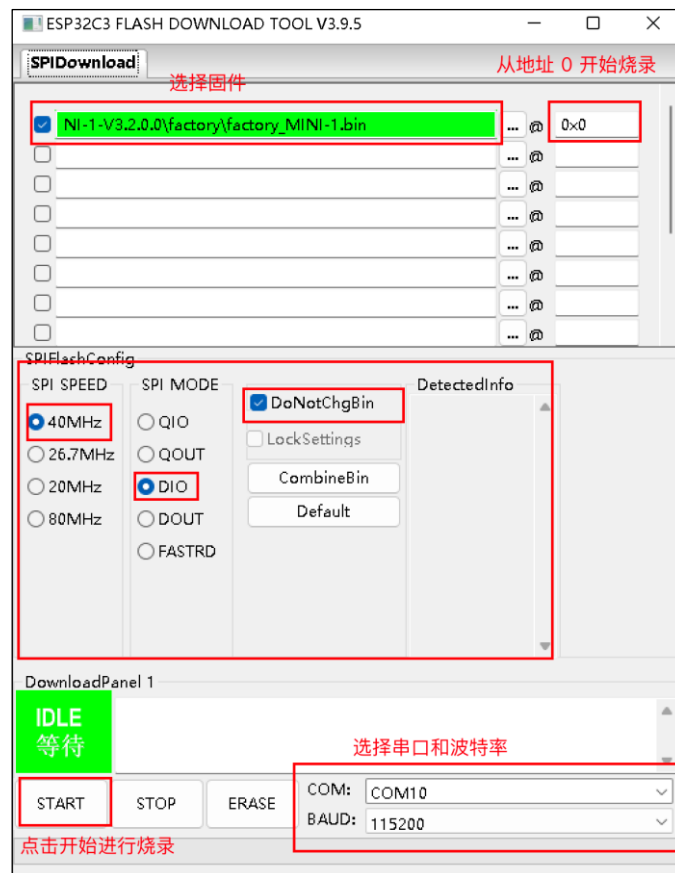
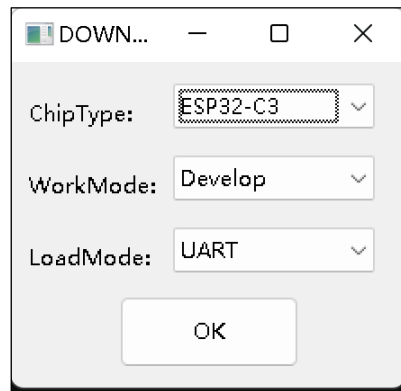
4) 烧录固件

按入下图设置好跳线。

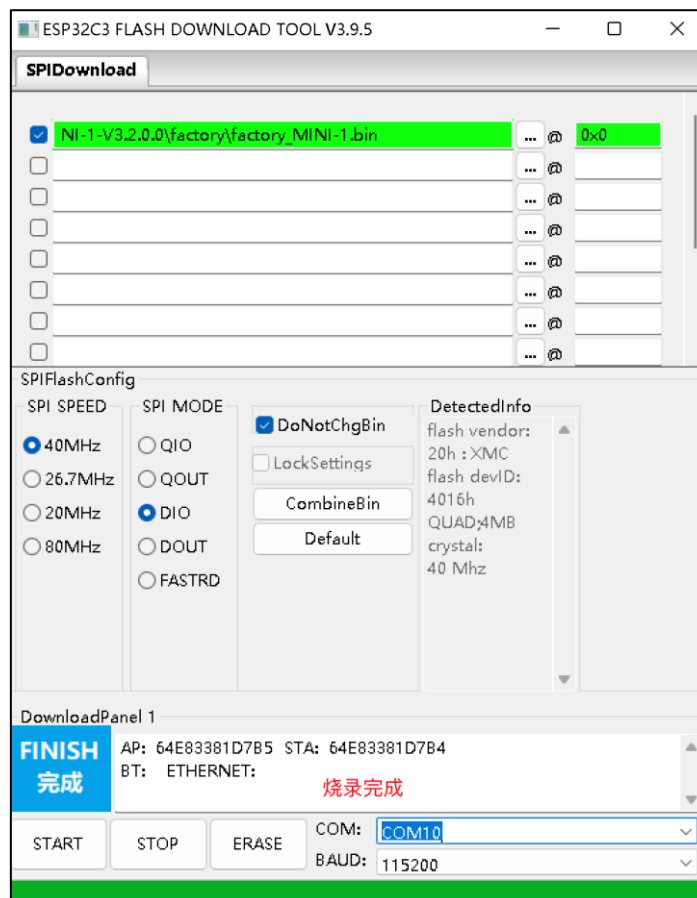
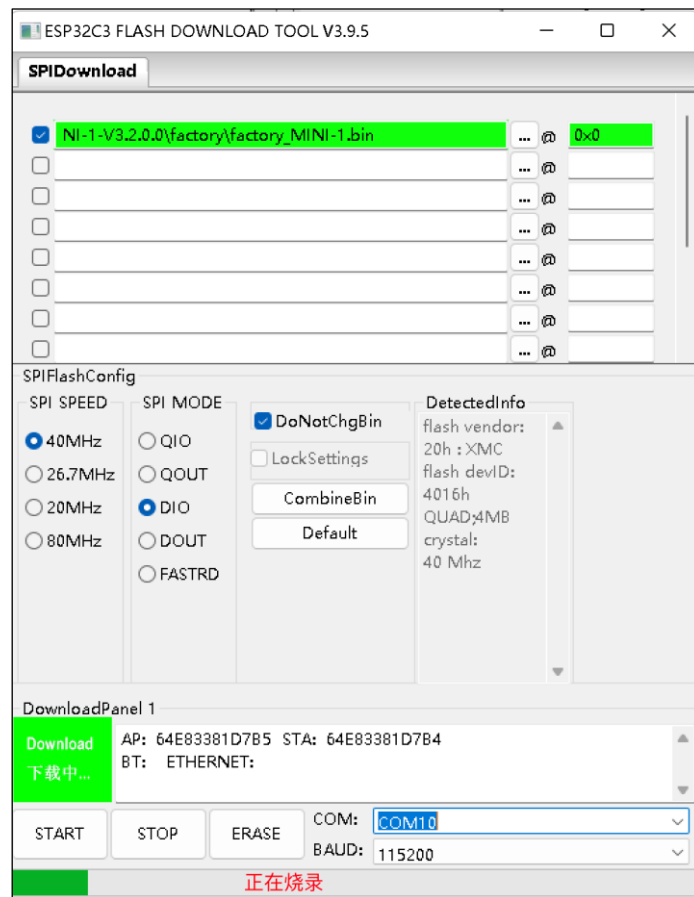


先禁用串口 1 (烧一个程序只要不初始化串口就行)

打开烧录工具。



先按下 sw7 不松开，然后再按下 sw6，然后松开 sw7。这样就使芯片进入下载模式。
然后再按一下 sw7，就会开始下载。



烧录成功后要给 ESP32 芯片重新上电或按下重启键。然后如果输出如下信息，证明下载成功了。



```
SSCOM V5.13.1 串口/网络数据调试器,作者:大虾丁丁,2618058@qq.com. QQ群: 52502449(最新版本)
通讯端口 串口设置 显示 发送 多字符串 小工具 帮助 联系作者 大虾论坛

[0:32mI (35) boot.esp32c3: SPI Speed      : 40MHz [0m
[0:32mI (40) boot.esp32c3: SPI Mode       : DIO [0m
[0:32mI (45) boot.esp32c3: SPI Flash Size : 4MB [0m
[0:32mI (49) boot: Enabling RNG early entropy source... [0m
[0:32mI (55) boot: Partition Table: [0m
[0:32mI (58) boot: ## Label                  Usage      Type ST Offset   Length [0m
[0:32mI (66) boot: 0 otdata                  OTA data   01 00 0000d000 00002000 [0m
[0:32mI (73) boot: 1 phy_init                RF data   01 01 0000f000 00001000 [0m
[0:32mI (81) boot: 2 nvs                     WiFi data 01 02 00010000 0000e000 [0m
[0:32mI (88) boot: 3 at_customize            unknown   40 00 0001e000 00042000 [0m
[0:32mI (96) boot: 4 ota_0                  OTA app    00 10 00060000 001d0000 [0m
[0:32mI (103) boot: 5 ota_1                 OTA app    00 11 00230000 001d0000 [0m
[0:32mI (111) boot: End of partition table [0m
[0:32mI (115) boot: No factory image, trying OTA 0 [0m
[0:32mI (120) esp_image: segment 0: paddr=00060020 vaddr=3c160020 size=32438h (205880) map [0m
[0:32mI (173) esp_image: segment 1: paddr=00092460 vaddr=3fc95c00 size=0386ch ( 14444) load [0m
[0:32mI (177) esp_image: segment 2: paddr=00095cd4 vaddr=40380000 size=0a344h ( 41796) load [0m
[0:32mI (189) esp_image: segment 3: paddr=000a0020 vaddr=42000020 size=155650h (1398352) map [0m
[0:32mI (495) esp_image: segment 4: paddr=001f5678 vaddr=4038a344 size=0b828h ( 47144) load [0m
[0:32mI (507) esp_image: segment 5: paddr=00200ea8 vaddr=50000000 size=00018h (   24) load [0m
[0:32mI (514) boot: Loaded app from partition at offset 0x600000 [0m
[0:32mI (550) boot: Set actual ota_seq=1 in otdata[0] [0m
[0:32mI (550) boot: Disabling RNG early entropy source... [0m
no external 32k oscillator, disable it now.
at param mode: 1
AT cmd port:uart1 tx:7 rx:6 ots:5 rts:4 baudrate:115200
module_name: MINI-1
max tx power=78, ret=0
v3.2.0.0

清除窗口 打开文件 Power_Standby_Register\Objects\Project.hex 发送文件 停止 清发区 最前 English 保存参数 扩展
端口号 COM4 USB 串行设备 [ ] HEX显示 保存数据 [ ] 接收数据到文件 [ ] HEX发送 [ ] 定时发送: 1000 ms/次 [ ] 加回车换行
[ ] 关闭串口 [ ] 更多串口设置 [ ] 加时间戳和分包显示, 超时时间: 20 ms 第1 字节 至 末尾 [ ] 加校验 None
[ ] RTS [ ] DTR 波特率: 115200 3445
为了更好地发展SSCOM软件
请您注册嘉立创结网客户 发送
▲QQ群满了, 没有第二个群. 有需要请在虾坛提问[注册] ★合宙高性价比4G模块 ★RT-Thread中国人的开源免费操作系统 ★8KM远距离WiFi可自
www.daxia.com S:0 R:2350 COM4 已打开 115200bps,8,1,None,None
```

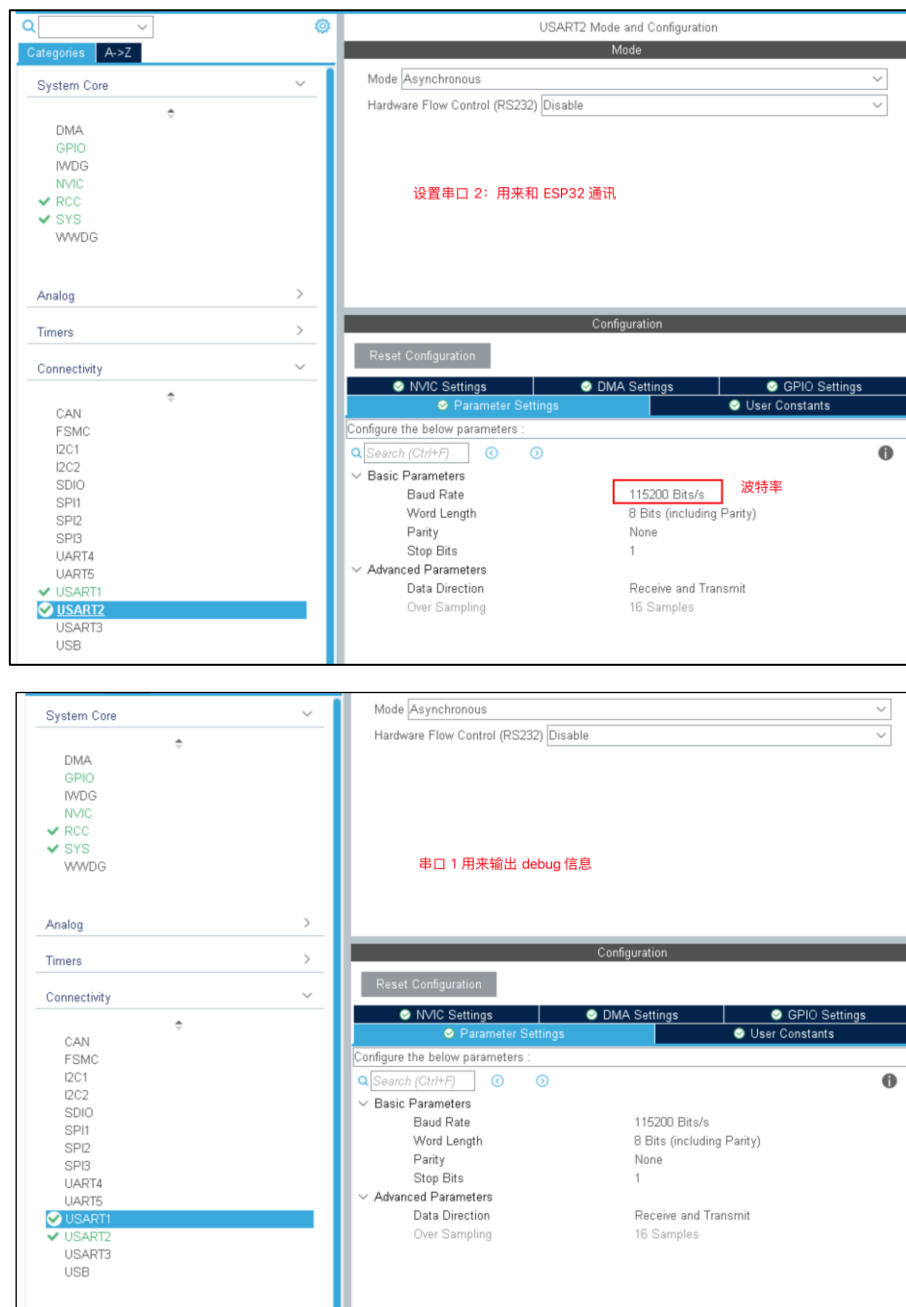
3.3 WIFI 案例 1：测试 AT 指令

3.3.1 需求描述

测试 AT 指令是否能够正常控制 ESP32 的 wifi，比如重启，读取设备信息等。

3.3.2 软件设计

3.3.2.1 STM32CubeMX 设置



3.3.2.2 usart.h 中添加函数声明

```
/* USER CODE BEGIN Prototypes */
#include "stdio.h"
int fputc(int ch, FILE *f);
/* USER CODE END Prototypes */
```

3.3.2.3 usart.c 中添加函数

```
/* USER CODE BEGIN 1 */
int fputc(int ch, FILE *f)
{
    /* 发送一个字节数据到串口 DEBUG_USART */
```

```

        HAL_UART_Transmit(&huart1, (uint8_t *)&ch, 1, 1000);
        return (ch);
    }
/* USER CODE END 1 */

```

3.3.2.4 main.c

```

uint8_t buff[10000];
int main(void)
{
    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();
    MX_USART2_UART_Init();
    MX_USART1_UART_Init();
    ESP32_Init();

    /* 1. 测试 AT 启动 */
    ESP32_Send_CMD("AT\r\n", 4);
    ESP32_ReadResponse(buff, 1000);
    printf("测试AT启动: %s", buff);
    memset(buff, 0, sizeof(buff));

    /* 2. 查看版本信息 */
    ESP32_Send_CMD("AT+GMR\r\n", 8);
    ESP32_ReadResponse(buff, 1000);
    printf("查看版本信息: %s", buff);

    while (1)
    {
        /* USER CODE END WHILE */

        /* USER CODE BEGIN 3 */
    }
    /* USER CODE END 3 */
}

```

3.3.2.5 esp32.h

```

#ifndef __ESP32_H
#define __ESP32_H

#include "usart.h"
#include "string.h"
#include "stdio.h"

void ESP32_Init(void);

```

```

void ESP32_Send_CMD(uint8_t *cmd, uint16_t cmdLength);
void ESP32_ReadResponse(uint8_t responseBuff[], uint16_t size);
#endif

```

3.3.2.6 esp32.c

```

#include "esp32.h"
uint8_t rBuff[1000] = {0};
/**
 * @description: 初始化 ESP32
 */
void ESP32_Init(void)
{
    /* 1. 初始化串口 2(通过串口 2 与 ESP32 进行通讯) */
    MX_USART2_UART_Init();

    /* 2. 重启 ESP32 */
    uint8_t *atCmd = "AT+RST=0\r\n";
    ESP32_Send_CMD(atCmd, strlen((char *)atCmd));
    HAL_Delay(3000);
}

/**
 * @description: 发送 AT 指令
 * @param {uint8_t} *cmd 要发送的 AT 指令(指令必须以 \r\n 结束)
 * @param {uint16_t} cmdLength 指令长度
 */
void ESP32_Send_CMD(uint8_t *cmd, uint16_t cmdLength)
{
    HAL_UART_Transmit(&huart2, cmd, cmdLength, 1000);
    do
    {
        ESP32_ReadResponse(rBuff, 1000);
        printf("%s", rBuff);
    }while(strstr((char *)rBuff, "OK") == NULL); // 一直等到读到 OK
    printf("\r\n===== \r\n");
}

/**
 * @description: 发送 AT 指令后, 用来接收响应
 *
 * 要考虑到接收的响应为非定长数据。
 *
 * @param {uint8_t} responseBuff[] 存储接收的响应的缓冲区。
 * @param {uint16_t} Size 缓冲区的长度
 */

```

```
void ESP32_ReadResponse(uint8_t responseBuff[], uint16_t size)
{
    /* 清空缓冲区 */
    memset(responseBuff, 0, size);
    uint16_t rxLen = 0;
    HAL_UARTEx_ReceiveToIdle(&huart2, responseBuff, size, &rxLen, 10000);
}
```

3.4 WiFi 案例 2：TCP 通讯

3.4.1 ESP32 支持的 WIFI 工作模式

在实现 TCP 的通讯案例之前我们需要先了解下 ESP32 支持的工作模式。

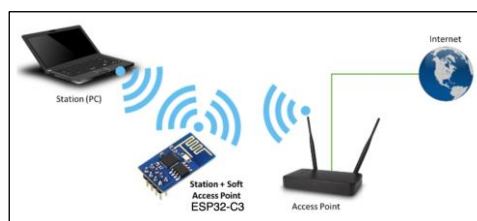
(1) 基站模式。（即 STA 模式或 Wi-Fi 客户端模式），此时 ESP32 连接到接入点（AP）。相当于 ESP32 是手机，连接你家里的路由器。



(2) AP 模式。（即 Soft-AP 模式或接入点模式），此时基站连接到 ESP32。相当于 ESP32 是手机，给别人开热点。



(3) AP-STA 共存模式。（ESP32 既是接入点，同时又作为基站连接到另外一个接入点）。相当于你手机蹭别人的 wifi 网，又给自己人开热点。



(4) 上述模式的各种安全模式（WPA、WPA2 及 WEP 等）。

(5) Scan 扫描接入点（包括主动扫描及被动扫描）。

(6) 使用混杂模式监控 IEEE802.11 Wi-Fi 数据包。

3.4.2 需求描述

使用 AT 指令在 ESP32 上面启动一个 TCP 服务器，从而可以与其他 TCP 客户端实现 TCP 通讯。

3.4.3 软件设计

拷贝 Wifi 案例 1，在上面的基础上进行修改。

3.4.3.1 main.c

```
/* USER CODE BEGIN PFP */
uint8_t rxBuff[200] = {0};
uint16_t realReceiveCount = 0;
uint8_t id = 255;
uint8_t ip[16] = {0};
uint16_t port = 0;

int main(void)
{
    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();
    MX_USART2_UART_Init();
    MX_USART1_UART_Init();
    printf("尚硅谷 wifi TCP 服务器实验...\r\n");

    /* 1. 初始化 WIFI */
    WIFI_Init();
    /* 2. 启动 TCP 服务器 */
    WIFI_StartTcpServer();
    while (1)
    {
        /* 3. 从串口读取数据 */
        realReceiveCount = 0;
        WIFI_ReadTcpData(rxBuff, 200, &id, ip, &port, &realReceiveCount);
        if (realReceiveCount > 0)
        {
            printf("收到数据:对方 id=%d, 对方 ip:port=%s:%d, 字节数=%d, 内容=%s \r\n",
id, ip, port, realReceiveCount, rxBuff);

            // 再发给对方
            WIFI_SendTcpData(id, rxBuff, realReceiveCount);
        }
    }
}
```

3.4.3.2 wifi.h

```
#ifndef __WIFI_H
#define __WIFI_H

#include "esp32.h"

void WIFI_Init(void);

void WIFI_STA_MODE(void);

void WIFI_AP_MODE(void);

void WIFI_StartTcpServer(void);

void WIFI_ReadTcpData(uint8_t rxBuff[],
                      uint16_t maxSize,
                      uint8_t *id,
                      uint8_t *ip,
                      uint16_t *port,
                      uint16_t *realReceveSize);

void WIFI_SendTcpData(uint8_t id,
                      uint8_t *msg,
                      uint16_t size);

#endif
```

3.4.3.3 wifi.c

```
#include "wifi.h"

/**
 * @description: 初始化 WIFI
 * @return {*}
 */
void WIFI_Init(void)
{
    /* 1. 初始化 ESP32 芯片 */
    ESP32_Init();

    // WIFI_AP_MODE();
    WIFI_STA_MODE();
}

/**
```

```

* @description: wifi 设置为 STAT 模式
*/
void WIFI_STA_MODE(void)
{
    /* 1. 设置为基站模式 */
    printf("设置基站模式\r\n");
    uint8_t *cmd = "AT+CWMODE=1\r\n";
    ESP32_Send_CMD(cmd, strlen((char *)cmd));

    /* 2. 指定连接到哪个 AP(wifi 路由器) */
    printf("设置要连接到的 AP\r\n");
    cmd = "AT+CWJAP=\"lzc2022\", \"feiduqiche\"\r\n";
    ESP32_Send_CMD(cmd, strlen((char *)cmd));
}

/**
* @description: wifi 设置为 AP 模式
* @return {*}
*/
void WIFI_AP_MODE(void)
{
    /* 2. 设置 WIFI 为 AP 模式 */
    /* 2.1 设置为 WIFI 模式
        0: 无 Wi-Fi 模式, 并且关闭 Wi-Fi RF
        1: Station 模式
        2: SoftAP 模式
        3: SoftAP+Station 模式
    */
    printf("设置 AP 模式\r\n");
    uint8_t *cmd = "AT+CWMODE=2\r\n";
    ESP32_Send_CMD(cmd, strlen((char *)cmd));

    /* 2.2 设置 AP 的
        - SSID,
        - 密码
        - 信道
        - 加密方式 0: OPEN 2: WPA_PSK 3: WPA2_PSK 4: WPA_WPA2_PSK
        - 广播 ssid 0: 广播 SSID (默认) 1: 不广播 SSID 注意: 如果设置默认值, 就不要添加这
    个参数
    */
    printf("设置 AP 的参数\r\n");
    cmd = "AT+CWSAP=\"atguigu-esp\", \"12345678\", 5, 3\r\n";
    ESP32_Send_CMD(cmd, strlen((char *)cmd));
}

```

```

/* 2.3 设置 AP 的 ip 地址 */
printf("设置 AP 的 IP 地址\r\n");
cmd = "AT+CIPAP=\"192.168.8.1\"\r\n";
ESP32_Send_CMD(cmd, strlen((char *)cmd));
}

/**
 * @description: 开启一个 Tcp 服务器。用来接收 Tcp 客户端的请求
 * @return {*}
 */
void WIFI_StartTcpServer(void)
{
    /* 1. 设置多连接模式。 只有多连接模式才能开启 TCP 服务器 */
    printf("设置多连接模式,用于开启 TCP 服务器\r\n");
    uint8_t *cmd = "AT+CIPMUX=1\r\n";
    ESP32_Send_CMD(cmd, strlen((char *)cmd));

    /*
        2. 建立 TCP 服务器
        - 0: 关闭 TCP 服务区 1: 建立 TCP 服务器
        - TCP 服务器的端口
    */
    printf("建立服务器\r\n");
    cmd = "AT+CIPSERVER=1,8080\r\n";
    ESP32_Send_CMD(cmd, strlen((char *)cmd));

    /*
        3. 设置 IPD(Incoming Packet Data)数据格式
        0: 在 "+IPD" 和 "+CIPRECVDATA" 消息中, 不提示对端 IP 地址和端口信息
        1: 在 "+IPD" 和 "+CIPRECVDATA" 消息中, 提示对端 IP 地址和端口信息
        +IPD,0,4,"192.168.8.3",56241:abcd
    */
    printf("设置 IPD 数据格式\r\n");
    cmd = "AT+CIPDINFO=1\r\n";
    ESP32_Send_CMD(cmd, strlen((char *)cmd));
}

/* 临时缓冲区, 存储 TCP 协议传输过来的所有数据 */
uint8_t tempBuff[1000];

/**
 * @description: 使用阻塞式方式从串口读取 TCP 数据。 利用到了空闲帧
 * @param {uint8_t} rxBuff 存储读取到的实际数据。
 * @param {uint16_t} maxSize 最多读取多少
 * @param {uint8_t} ip 对方 ip 地址

```

```

* @param {uint16_t} port 对方端口
* @param {uint16_t} *realReceveSize 接收的数据量
* @return {*}
*/

void WIFI_ReadTcpData(uint8_t rxBuff[],
                      uint16_t maxSize,
                      uint8_t *id,
                      uint8_t *ip,
                      uint16_t *port,
                      uint16_t *realReceveSize)
{
    uint16_t rxLen = 0;
    /* 清空缓冲区 */
    memset(tempBuff, 0, sizeof(tempBuff));
    memset(rxBuff, 0, maxSize);
    /* 利用空闲帧来接收 TCP 客户端传来的数据 */
    HAL_UARTEx_ReceiveToIdle(&huart2, tempBuff, maxSize, &rxLen, 10000);
    /*
        解析数据
        TCP 客户端传来的数据 ESP32 通过传来的格式是：
        \r\n+IPD,0,4,"192.168.8.3",56241:abcd
            IPD: Incoming Packet Data
            0: 表示来自于哪个连接 ID。
                在多连接模式下，ESP8266/ESP32 可以同时处理多个连接，每个连接有一个唯一的 ID。

            4: 这个数字表示随后的数据长度，单位是字节
            对话的 ip 和端口
            abcd: 实际接收到的数据
    */
    if (strstr((char *)tempBuff, "+IPD") != NULL)
    {
        /*
            解析收到的数据
            %*[\r\n] 跳过任意输入的\r 或\n
            %[^"] 匹配字符串，一直到 "
        */
        sscanf((char *)tempBuff, "%*[\r\n]+IPD,%d,%d,\"%[^\"]\",%d",
              (int *)id,
              (int *)realReceveSize,
              ip,
              (int *)port);

        /* 截取出真正的数据，并存入到临时缓冲区 */
    }
}

```

```

        strtok((char *)tempBuff, ":"); /* 冒号之前的数据 */

        /* 把真正的数据 copy 到用户缓冲区 */
        memcpy(rxBuff, strtok(NULL, ":"), *realReceveSize);
    }
}

/**
 * @description: 通过 TCP 协议给对方发数据
 * @param {uint8_t} id 对方 id
 * @param {uint8_t} *msg 具体信息
 * @param {uint16_t} size 信息长度
 * @return {*}
 */
uint8_t sendCmd[100];
uint8_t responseBuff[100];
void WIFI_SendTcpData(uint8_t id,
                      uint8_t *msg,
                      uint16_t size)
{
    /* 1. 发送 AT 指令 AT+CIPSEND=<link ID>,<length> */
    memset(sendCmd, 0, sizeof(sendCmd));
    sprintf((char *)sendCmd, "AT+CIPSEND=%d,%d\r\n", id, size);
    printf("给对方发送数据\r\n");
    ESP32_Send_CMD(sendCmd, strlen((char *)sendCmd));
    /* 3. 把要发送的数据通过串口发出 */
    HAL_UART_Transmit(&huart2, msg, size, 1000);
}

```

第 4 章 物联网通讯之蓝牙通讯

4.1 蓝牙基础知识

4.1.1 蓝牙概述

蓝牙，是一种利用低功率无线电，支持设备短距离通信的无线电技术，能在包括移动电话、PDAQ、无线耳机、笔记本电脑、相关外设等众多设备之间进行无线信息交换，蓝牙工作在全球通用的 2.4 GHz(2.4 至 2.485 GHz) ISM(即工业、科学、医学)频段，使用 IEEE802.11 协议。

4.1.2 蓝牙产生背景

1998 年 5 月，爱立信、诺基亚、东芝、BM 和英特尔公司等五家著名厂商，在联合开展短程无线通信技术的标准化活动时提出了蓝牙技术，其宗旨是提供一种**短距离、低成本**的无线传输应用技术。芯片霸主 Intel 公司负责半导体芯片和传输软件的开发，爱立信负责无线射频和移动电话软件的开发，IBM 和东芝负责笔记本电脑接口规格的开发。1999 年下半年，著名的业界巨头微软、摩托罗拉、三星、朗讯与蓝牙特别小组的五家公司共同发起成立了蓝牙技术推广组织，从而在全球范围内掀起了一股“蓝牙”热潮。

全球业界即将开发一大批蓝牙技术的应用产品，使蓝牙技术呈现出极其广阔的市场前景，并预示着 21 世纪初将迎来波澜壮阔的全球无线通信浪潮。

4.1.3 蓝牙发展历程

自 1994 年由爱立信推出至今，蓝牙技术已经走过了 20 个岁月，发展为当前的状况。

第一代蓝牙：关于短距离通讯早期的探索，使用的是 BR(Basic Rate)技术，此时蓝牙的理论传输速率，只能达到 721.2Kbps。

第二代蓝牙：新增的 EDR(Enhanced Data Rate)技术，使得蓝牙设备的传输率可达 3Mbps。

第三代蓝牙：核心是 AMP(Generic Alternate MAC/PHY),这是一种全新的交替射频技术，支持动态地选择正确射频，传输速率高达 24Mbps。

第四代蓝牙：主推 Low Energy 低功耗，BLE(Bluetooth Low Energy)低功耗功能。

第五代蓝牙：开启物联网时代大门，在低功耗模式下具备更快更远的传输能力。

4.1.4 蓝牙技术类型

蓝牙协议包括两种技术：BR：Basic Rate 和 LE：Low Energy。

这两种技术都包括搜索（discovery）管理、连接（connection）管理等机制，但它们是相互独立的，不能互通的技术！

厂商如果只实现了一种，那么只能与同样实现该技术的设备互通。如果厂商要确保能和所有的蓝牙设备互通，那么就只能同时实现两种技术，而不去管是否真的需要。

1) 经典蓝牙（BR/EDR）

BR: Basic Rate 是正宗的蓝牙技术，可以包括可选的 EDR（Enhanced Data Rate）技术，以及 AMP（Alternate MAC and PHY layer extension）。

BR: 最早期的蓝牙技术，速度只能达到 721.2Kbps，在那个年代，已为高大上了。•EDR: 随着技术的提升，使用 EDR 技术的蓝牙，理论速率可以达到 2.1Mbps。

AMP: 使用 AMP 技术的蓝牙，理论速率可以达到 54Mbps。蓝牙自身的物理层和 AMP 技术差异明显，BR/EDR 和 AMP 是不能同时使用的。简单的说，就是：BR 和 EDR 是可以同时存在的，但 BR/EDR 和 AMP 只能二选一。

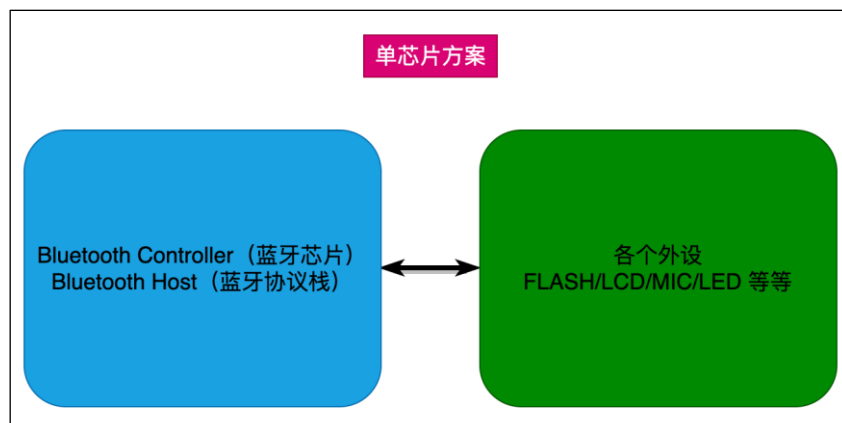
2) 低功耗蓝牙（BLE）

上面所讲的 BR 技术的进化路线，就是传输速率的加快、加快、再加快。但能量是守恒的，你想传的更快，代价就是消耗更多的能量。而有很多的应用场景，并不关心传输速率，反而非常关心功耗。这就是 Bluetooth LE（称作蓝牙低功耗）产生的背景。

低功耗蓝牙与经典蓝牙使用相同的 2.4GHz 无线电频率，因此双模设备可以共享同一个天线。低功耗蓝牙使用的调制系统更简单。BLE 技术相比 BR 技术，差异非常大，或者说就是两种不同的技术，凑巧都加一个“蓝牙”的前缀而已。目前 BLE 主要广泛应用于 IoT（Internet of Things: 物联网）产品领域。

4.1.5 市场上常见蓝牙架构

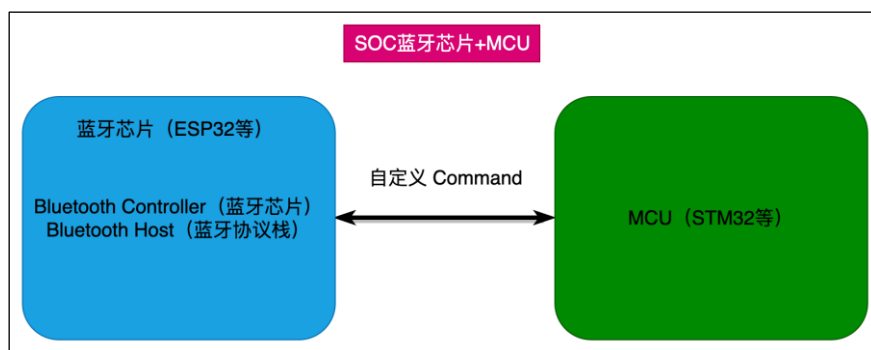
1) SOC 蓝牙单芯片方案



一般是半导体厂商半开源协议栈，把开发的蓝牙协议栈直接烧写到蓝牙芯片中（比如 CSR BC4/5，CSR8670，CSR8675，T1CC2540， NRF51xXx，NRF52XXx，乐鑫 ESP32 等等）。

此类芯片一般可以直接做为 MCU 用，这类产品一般用于消费类电子，集成度很高，调试参数可以直接使用，常见的有蓝牙耳机等产品。

2) SOC 蓝牙+MCU 方案



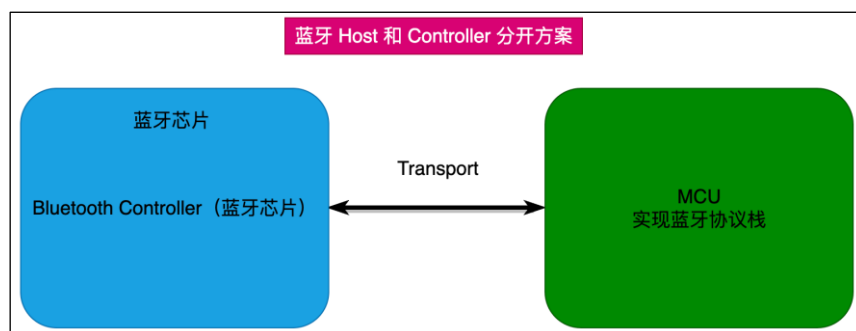
在集成好的蓝牙芯片基础上，通过特定的接口（UART 居多），发送自定义的 command 来达到想要的功能。比如发送 0x01 代表搜索周围设备。

外设个单芯片方案自定义 command 包含 BT Controller（蓝牙芯片）MCU 跟 BT Host（蓝牙协议栈）。

此部分的应用，将蓝牙作为一个外设使用，用于远程通信。例如网上卖的一些蓝牙串口。

比如我们后面应用就是采用的这种方案。STM32 作为 MCU 使用，ESP32 作为蓝牙芯片使用。

3) 蓝牙 host+controller 分开方案



这种应用算是蓝牙最复杂的应用，客户需要使用蓝牙的场景有很多，涉及到的蓝牙协议也有很多，需要将 Host 与 Controller 分开，集成更多的蓝牙协议，比如蓝牙电话（HFP），蓝牙音频（A2DP），蓝牙音乐控制（AVRCP），蓝牙电话本（PBAP）蓝牙短信（MAP）等。

此部分应用，将定制蓝牙的各种服务，实现蓝牙多功能需求。

4) 使用场景

介绍几种使用场景，帮助理解。

(1) 手机。手机的蓝牙复杂应用，注定要用第 3 种方案，也就是蓝牙协议栈（hos）在主芯片中，蓝牙芯片为 HC 架构的

(2) 蓝牙音响，蓝牙耳机此种应用。一般用单芯片方案就能搞定，好处在于开发便捷。

(3) 蓝牙手表。手表要看功能复杂性，如果仅仅有时间显示，传感器交互，蓝牙，那么可以选择单芯片方案（也就是方案），如果有网络等比较复杂的功能就要使用 MCU+蓝牙芯片方案。

(4) 蓝牙手环，蓝牙心率带等。基本上是单芯片方案

4.1.6 蓝牙协议栈

蓝牙协议栈极其复杂，官方协议说明书大概有 800 多页。我们只做一个大致了解。芯片厂家一般都封装好了，我们直接使用即可。

4.1.6.1 蓝牙芯片架构

蓝牙的核心系统，由一个 Host 和一个或多个 Controller 组成。

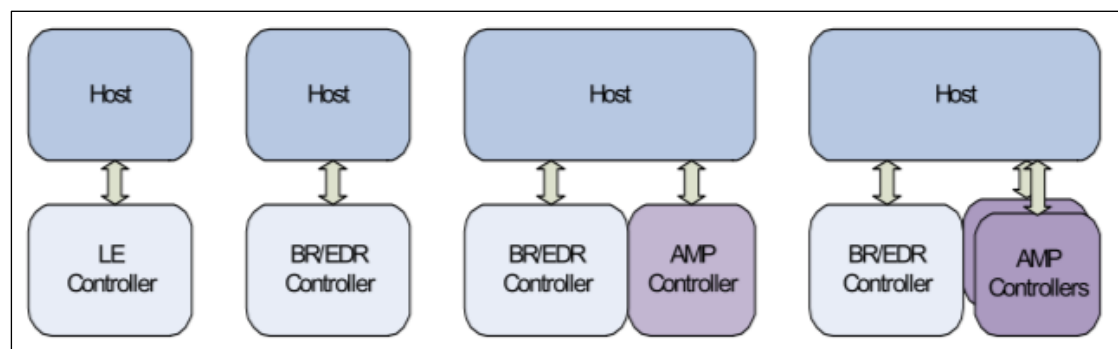
(1) BT Host：逻辑实体，在 HCI（Host Controller Interface）的上层。

(2) BT Controller：逻辑实体，在 HCI（Host Controller Interface）的下层。

根据 Host 与 Controller 的组成关系，常见的蓝牙芯片也分为以下几种：

(1) 单模蓝牙芯片：单一传统蓝牙的芯片，单一低功耗蓝牙的芯片。即（1 个 Host 结合 1 个 Controller）

(2) 双模蓝牙芯片：同时支持传统蓝牙和低功耗蓝牙的芯片。即（1 个 Host 结合多个 Controller）



蓝牙协议是通信协议的一种，一般而言，我们把某个协议的实现代码称为协议栈（protocol stack），BLE 协议栈就是实现低功耗蓝牙协议的代码。

4.1.6.2 BLE 低功耗蓝牙协议栈框架

要实现一个 BLE 应用，首先需要支持 **BLE 射频** 的芯片，然后还需要提供一个与此芯片配套的 **BLE 协议栈**，最后在协议栈上**开发自己的应用**。

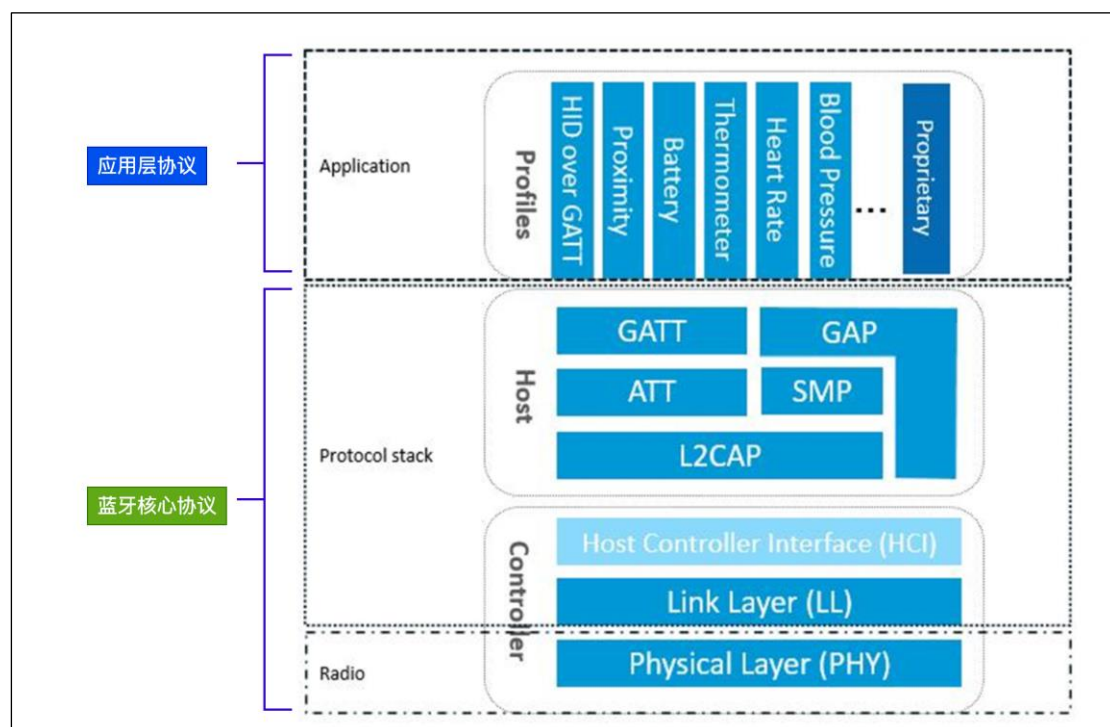
可以看出 BLE 协议栈是连接芯片和应用的桥梁，是实现整个 BLE 应用的关键。

简单来说，BLE 协议栈主要用来对你的应用数据进行层层封包，以生成一个满足 BLE 协议的空中数据包，也就是说，把应用数据包裹在一系列的帧头（header）和帧尾（tail）中。

蓝牙协议规定了两个层次的协议，分别为**蓝牙核心协议**（Bluetooth Core）和**蓝牙应用层协议**（Bluetooth Application）。

蓝牙核心协议关注对蓝牙核心技术的描述和规范，它只提供基础的机制，并不关心如何使用这些机制；

蓝牙应用层协议，是在蓝牙核心协议的基础上，根据具体的应用需求，百花齐放，定义出各种各样的策略，如 FTP、文件传输、局域网等等。



而蓝牙核心协议（Bluetooth Core）又包含 BLE Controller 和 BLE Host 两部分。Controller 负责定义 RF、Baseband 等偏硬件的规范，并在这之上抽象出用于通信的逻辑链路（Logical Link）；Host 负责在逻辑链路的基础上，进行更为友好的封装，这样就可以屏蔽掉蓝牙技术的细节，让 Bluetooth Application 更为方便的使用。



4.1.6.3 物理层 (Physical Layer, PHY)

PHY 层主要负责在物理信道上发送和接收信息包。Bluetooth LE 使用 40 个射频信道。频率范围：2402 MHz 到 2480 MHz。

4.1.6.4 链路层 (Link Layer, LL)

LL 层主要负责创建、修改和释放逻辑链路（以及，如果需要，它们相关的逻辑传输），以及与设备之间的物理链路相关的参数的更新。它控制链路层状态机处于准备、广播、监听/扫描、发起连接、已连接五种状态之一。

4.1.6.5 主机控制接口层 (Host Controller Interface, HCI)

HCI 层向主机和控制器提供一个标准化的接口。该层可以由软件 API 实现或者使用硬件接口 UART、SPI、USB 来控制。

4.1.6.6 通用访问配置文件层 (Generic access profile, GAP)

GAP 层代表所有蓝牙设备通用的基本功能，例如传输、协议和应用程序配置文件使用的模式和访问程序。GAP 服务包括设备发现、连接模式、安全、身份验证、关联模型和服务发现。

4.1.6.7 逻辑链路控制及自适应协议层 (Logical Link Control and Adaptation Protocol, L2CAP)

L2CAP 层负责对主机和协议栈之间交换的数据进行协议复用能力、分段和重组操作。

4.1.6.8 安全管理层（Security Manager, SM）

SMP 层用于生成加密密钥和身份密钥。SMP 还管理加密密钥和身份密钥的存储，并负责生成随机地址并将随机地址解析为已知设备身份。

4.1.6.9 属性协议层（Attribute protocol, ATT）

简单来说，ATT 层用来定义用户命令及命令操作的数据，比如读取某个数据或者写某个数据。BLE 引入了 attribute 概念，用来描述一条一条的数据。Attribute 除了定义数据，同时定义该数据可以使用的 ATT 命令，因此这一层被称为 ATT 层。

4.1.6.10 通用属性配置文件层（Generic Attribute profile, GATT）

GATT 层表示属性服务器和可选的属性客户端的功能。该配置文件描述了属性服务器中使用的服务、特征和属性的层次结构。该层提供用于发现、读取、写入和指示服务特性和属性的接口。

4.2 ESP32-C3 中的蓝牙功能

ESP32-C3 支持 Bluetooth 5（LE）。下载好固件之后（我们前面下载的固件已经支持 wifi 和蓝牙了），STM32 仍然可以通过 AT 指令操作蓝牙。

4.2.1 BLE 角色划分

在 Bluetooth LE 协议栈中不同的层级有不同的角色划分。这些角色划分互不影响。

LL: 设备可以划分为 主机 和 从机，从机广播，主机可以发起连接。

GAP: 定义了 4 种特定角色：广播者、观察者、外围设备 和 中心设备。

GATT: 设备可以分为 服务端 和 客户端

4.2.2 BLE 的地址

分为公共地址和随机地址。

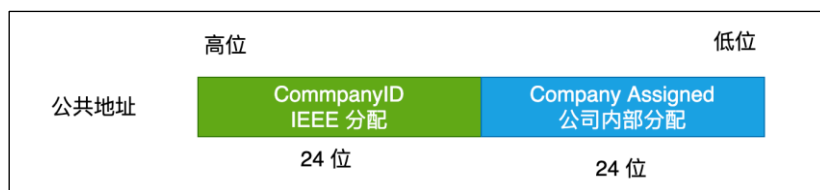
4.2.2.1 公共地址

BLE 的公共地址，就类似于我们日常的身份证号码，是全球唯一的且不可改变的。

为了保证 BLE 公共地址的全球唯一性，其需要向 IEEE 购买，然后 IEEE 组织就会对应地分配公共地址给买家。

其是全球唯一的且在 BLE 设备的整个生命周期，都不会改变。

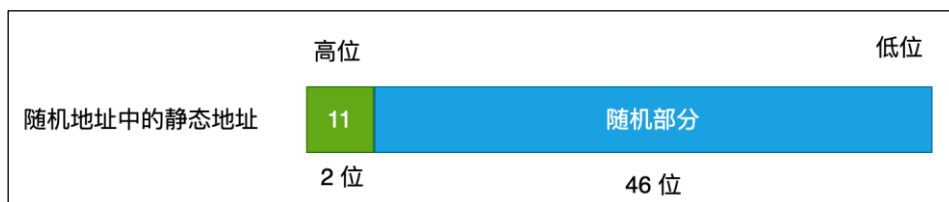
总长度为 6 个字节。



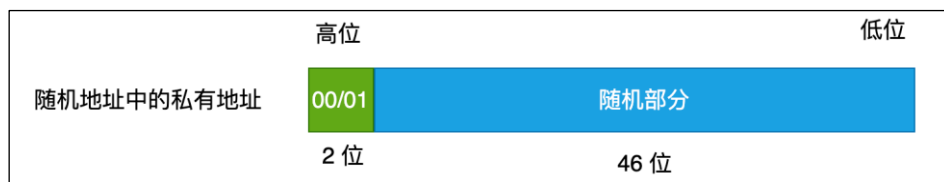
4.2.2.2 随机地址

除了公共地址类型之外，还有一个随机地址类型，提高额外的隐私保护，其又分为静态地址（Static Address）和私有地址（Private Address），它们之间主要通过最高的 2 位有效位来区分。

（1）静态地址。总长度也是 48 位，但是最高位的 2 位必须是 0b11。随机部分至少有一个位是 0 和 1。也就是说不能全部是 0 或全部是 1。设备重启之前不会改变。



（2）私有地址。总长度也是 48 位。最高位是 00/01。随机部分至少有一个位是 0 和 1。也就是说不能全部是 0 或全部是 1。设备重启之前不会改变。



4.2.2.3 使用地址

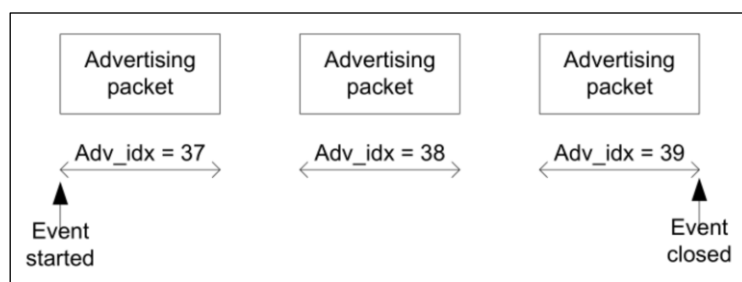
公共地址：该类型的地址是全球唯一且固定的，理论上重复的概率基本为 0，因为它们是由 IEEE 组织分配给买家的，适用于为保证 BLE 设备地址的唯一性，方便其他人辨别。

随机地址：说白了就是为了厂商白嫖，因为地址是免费的。当然也提高了隐私。随着隐私和安全性的关注增加，越来越多的蓝牙设备开始使用随机地址来减少被追踪的风险。这在蓝牙低功耗（BLE）技术中尤为常见。

4.2.3 广播

广播是指从机（服务器）每经过一个时间间隔发送一次广播数据包，这个时间间隔称为广播间隔，这个广播动作叫做广播事件，只有当从机处于广播状态时，主机（客户端）才能发现该从机（服务器）。

在每个广播事件中，广播包会分别在 37、38 和 39 三个信道上依次广播，如下图所示。



广播数据包一般包含可读的设备名称，设备是否可连接等信息。

4.2.4 扫描

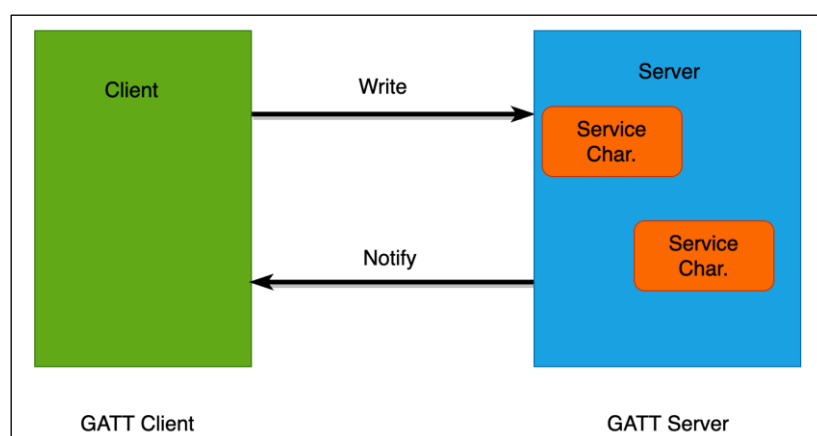
扫描是主机监听从机广播数据包和发送扫描请求的过程，主机通过扫描，可以获取到从机的广播包以及扫描回应数据包，主机可以对已扫描到的从机设备发起连接请求，从而连接从机设备并通信。

4.2.5 通讯

主从之间的通信是通过 GATT 的 Profile 来完成的，Profile 可以理解为配置，数据格式等。

从机作为 GATT 的 Server 端，用来定义和存储 Profile。Profile 包含一个或者多个 Service，每个 Service 又包含一个或者多个 Characteristic，Characteristic 是主从通信的最小单元。

主机作为 GATT 的 Client 端，用来发现和获取从机的 Service 和 Characteristic，从而与之通信。



关于服务 Service 和特征值 Characteristic 的概念。每个服务和特征值都有自己的唯一标识 UUID，标准 UUID 为 128 位，蓝牙协议栈中一般采用 16 位，也就是两个字节的 UUID 格式。

一个从机设备包括一个或者多个服务；一个服务中又可以包括一条或者多条特征值，每个特征值都有自己的属性 Property，属性的取值有：可读 Read，可写 Write 以及通知 Notify。

4.3 蓝牙通讯案例：透传模式下收发数据

4.3.1 需求描述

使用蓝牙模块的透传模式，实现与其他蓝牙客户端的通讯。

4.3.2 软件设计

4.3.2.1 main.c

```
uint8_t rxBuff[100];
uint16_t rxRealLength = 0;
int main(void)
{
    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();

    MX_USART2_UART_Init();
    MX_USART1_UART_Init();
    Bluetooth_Init();
    while (1)
    {
        Bluetooth_ReadData(rxBuff, &rxRealLength);
        if (rxRealLength > 0)
        {
            printf("蓝牙收到数据长度: %d, 数据内容: %s \r\n", rxRealLength, rxBuff);
            /* 数据原封不动返回 */
            Bluetooth_SendData(rxBuff, rxRealLength);
            rxRealLength = 0;
        }
    }
}
```

4.3.2.2 bluetooth.h

```
#ifndef __WIFI_H
#define __WIFI_H
#include "esp32.h"

void Bluetooth_Init(void);
void Bluetooth_ReadData(uint8_t rxBuff[], uint16_t *rxRealLength);
uint8_t Bluetooth_HandleConnChange(void);
void Bluetooth_SendData(uint8_t txBuff[], uint16_t length);
#endif
```

4.3.2.3 bluetooth.c

```
#include "Bluetooth.h"

uint8_t responseBuff[1000] = {0};

/**
 * @description: 初始化蓝牙
 * ESP32-C3 与手机建立 SPP 连接，以及在 UART-Bluetooth LE 透传模式下传输数据
 * 透传必须是当有客户端连接上之后，才能给这个客户端实现透传模式下传输数据
 * @return {*}
 */
void Bluetooth_Init(void)
{
    /* 1. 初始化 ESP32 */
    ESP32_Init();

    /* 2. 设置蓝牙角色 0: 注销 Bluetooth LE 1: client 角色 2: server 角色*/
    printf("设置角色\r\n");
    uint8_t *cmd = "AT+BLEINIT=2\r\n";
    ESP32_Send_CMD(cmd, strlen((char *)cmd));

    /* 3. 服务端创建服务*/
    printf("服务端创建服务\r\n");
    cmd = "AT+BLEGATTSSRVCRE\r\n";
    ESP32_Send_CMD(cmd, strlen((char *)cmd));

    /* 4. 服务端开启服务*/
    printf("服务端开启服务\r\n");
    cmd = "AT+BLEGATTSSRVSTART\r\n";
    ESP32_Send_CMD(cmd, strlen((char *)cmd));

    /* 5. 设置设备名称 */
    printf("设置设备名称\r\n");
    cmd = "AT+BLENNAME=\"atguigu-bt\"\r\n";
    ESP32_Send_CMD(cmd, strlen((char *)cmd));

    /*
     6. 设置蓝牙广播参数
        AT+BLEADVPARAM=<adv_int_min>,<adv_int_max>,<adv_type>,<own_addr_type>,<channel_map>

        a:  adv_int_min adv_int_max 广播的最小间隔和最大间隔。 等于该值乘上 0.625ms
        b:  adv_type 广播类型 0: 通用广播
    */
}
```

```

        c: own_addr_type          Bluetooth LE 地址类型  0: 公共地址 1: 随机地址
    址

        d: channel_map            广播信道    1: ADV_CHNL_37 2: ADV_CHNL_38
                                         4: ADV_CHNL_39 7: ADV_CHNL_ALL

    */
    printf("设置广播参数\r\n");
    cmd = "AT+BLEADVPARAM=50,50,0,0,7,0,,\r\n";
    ESP32_Send_CMD(cmd, strlen((char *)cmd));

    /*
    7. 设置广播数据并开始广播
        AT+BLEADVDATAEX=<dev_name>,<uuid>,<manufacturer_data>,<include_power>
    */

    printf("自动设置广播数据\r\n");
    cmd = "AT+BLEADVDATAEX=\"atguigu-bt\", \"A123\", \"0102030405\", 1\r\n";
    ESP32_Send_CMD(cmd, strlen((char *)cmd));

    printf("开始广播");
    cmd = "AT+BLEADVSTART\r\n";
    ESP32_Send_CMD(cmd, strlen((char *)cmd));

    /*
    8. 配置 Bluetooth LE SPP(Serial Port Profile)
        选择支持 notify 或者 indicate 的 characteristic 作为写通道发送数据，
        选择支持写操作的 characteristic 作为读通道接收数据。
    */

    printf("配置 BLESPPP\r\n");
    cmd = "AT+BLESPPCFG=1,1,7,1,5\r\n";
    ESP32_Send_CMD(cmd, strlen((char *)cmd));

    /*
    9. 透传模式下, Wi-Fi、socket、Bluetooth LE 或 Bluetooth 状态改变时会打印提示信息
    */
    printf("设置系统提示信息\r\n");
    cmd = "AT+SYSMSG=4\r\n";
    ESP32_Send_CMD(cmd, strlen((char *)cmd));
}

uint8_t tempBuff[200] = {0};
/**
 * @description: 透传模式下读取数据。

```

```

* @param {uint8_t} rxBuff 存放读取的数据
* @param {uint8_t} *rxRealLength 读取的数据的实际长度
* @return {*}
*/
void Bluetooth_ReadData(uint8_t rxBuff[], uint16_t *rxRealLength)
{
    uint16_t rxLen = 0;
    /* 清空临时缓冲区, 准备接收数据 */
    memset(tempBuff, 0, sizeof(tempBuff));
    HAL_UARTEx_ReceiveToIdle(&huart2, tempBuff, sizeof(tempBuff), &rxLen, 10000);
    if (rxLen == 0)
    {
        return;
    }
    /*
        处理连接变化。比如有设备连接, 连接断开等 (会涉及到 wifi 和蓝牙)。
        如果是连接信息变化, 则直接返回, 因为不是蓝牙透传的数据。
    */
    if (Bluetooth_HandleConnChange())
    {
        return;
    }

    /* 清空用户缓冲区数据 */
    memset(rxBuff, 0, sizeof(rxBuff));
    /* 把临时缓冲区数据 copy 到用户缓冲区 */
    memcpy(rxBuff, tempBuff, rxLen);

    /* 实际收到的字节数 */
    *rxRealLength = rxLen;
    rxBuff[*rxRealLength] = 0;
}

/**
* @description: 处理连接信息的变化
* @return {*} 1:有变化 0: 没有变化
*/
uint8_t Bluetooth_HandleConnChange(void)
{
    /*
        如果是连接变更信息, 如建立连接, 退出连接等

        +STA_CONNECTED:"32:50:2a:c0:f1:11"  wifi 连接信息变更, 直接返回
        +DIST_STA_IP:"22:e9:fd:98:c5:66","192.168.4.2"
    */

```

```

+STA_DISCONNECTED:"52:d8:5a:c7:b8:8e"

+BLESETPHY:"bc:d0:74:67:f7:64",2,2

+BLECONN:0,"53:4f:61:d6:2f:12" 蓝牙连接, 开启透传模式
+BLEDISCONN:0,"53:4f:61:d6:2f:12" 建立连接后, 蓝牙会自动关闭广播, 所以广播连接
后, 要重新广播
*/
/* 如果是 wifi 连接信息变更, 则不做任何处理 */
if (strstr((char *)tempBuff, "+STA_CONNECTED") != NULL ||
    strstr((char *)tempBuff, "+DIST_STA_IP") != NULL ||
    strstr((char *)tempBuff, "+STA_DISCONNECTED") != NULL)
{
    printf("wifi 连接信息发生变化...\r\n");
    return 1;
}
/* 如果时 BLE 建立连接, 则进入透传模式 */
if (strstr((char *)tempBuff, "+BLECONN") != NULL)
{
    printf("有 BLE 客户端连接, 马上开启 BLE 透传模式...\r\n");
    /* 进入透传模式 */
    printf("使能 BLESPP(开启 BLE 透传)\r\n");
    uint8_t *cmd = "AT+BLESPP\r\n";
    ESP32_Send_CMD(cmd, strlen((char *)cmd));

    /* 启动透传后, 有一个额外的字节数据, 消耗掉 */
    uint8_t tmp;
    HAL_UART_Receive(&huart2, &tmp, 1, 10);
    return 1;
}
/* 如果时 BLE 断开连接, 则退出透传模式, 然后重新广播 */
if (strstr((char *)tempBuff, "+BLEDISCONN") != NULL)
{
    printf("BLE 连接断开, 将退出透传模式...\r\n");
    /* 退出透传模式 */
    HAL_UART_Transmit(&huart2, "+++", 3, 100);
    HAL_Delay(2000); /* 等待 2s 后再发送 AT 指令 */

    /* 重新广播 */
    printf("重新开始广播\r\n");
    uint8_t *cmd = "AT+BLEADVSTART\r\n";
    ESP32_Send_CMD(cmd, strlen((char *)cmd));
    return 1;
}

```

```

    return 0;
}

/**
 * @description: 透传模式下，通过蓝牙向外发送数据
 * @param {uint8_t} txBuff
 * @param {uint16_t} length
 * @return {*}
 */
void Bluetooth_SendData(uint8_t txBuff[], uint16_t length)
{
    HAL_UART_Transmit(&huart2, txBuff, length, 100);
}

```

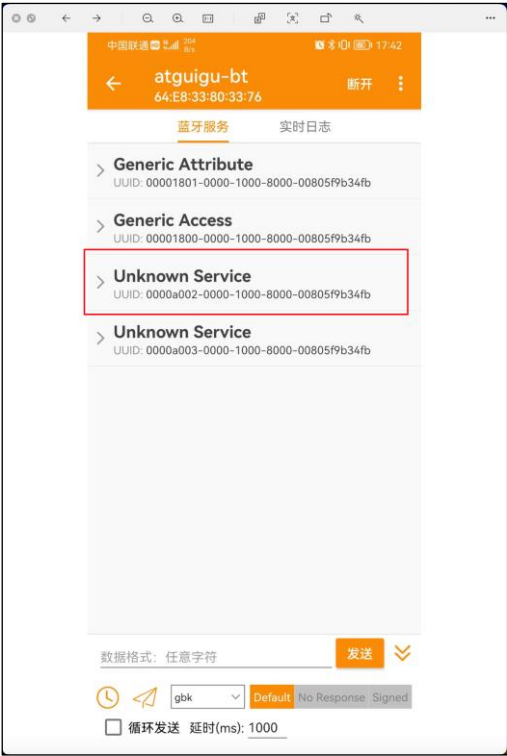
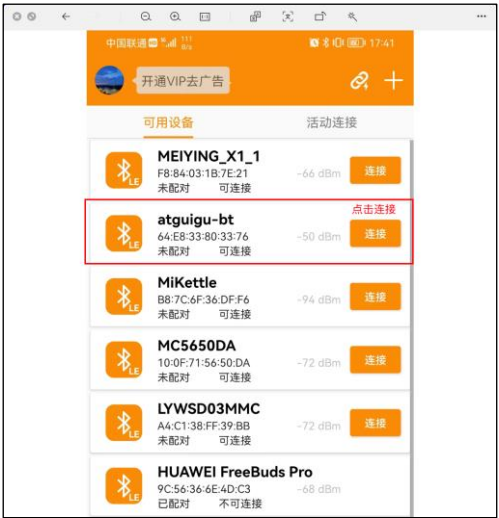
4.3.3 测试

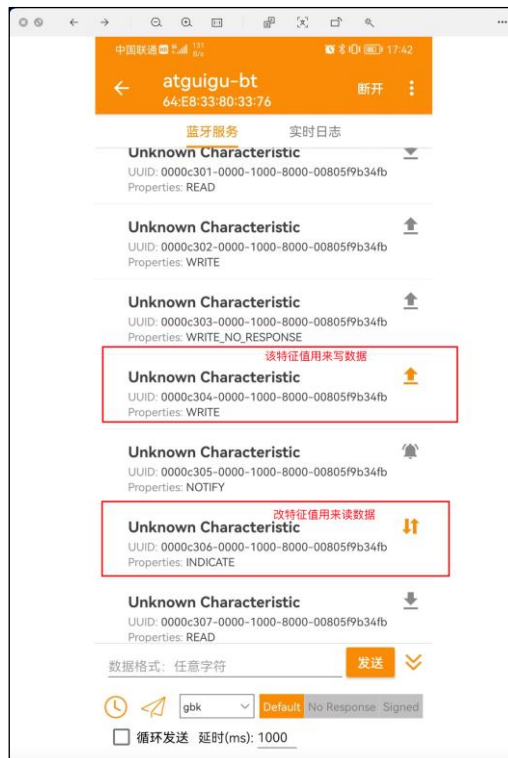
4.3.3.1 在手机上下载并安装蓝牙调试助手

在手机应用市场搜索“蓝牙调试助手”并下载安装。



4.3.3.2 连接进行通信测试





```
OK
服务端开启服务: AT+BLEGATSSRVSTART
OK
设置设备名称: AT+BLENAME="atgui-gu-bt"
OK
设置广播参数: AT+BLEADVPARAM=50,50,0,0,7,0,,
OK
自动设置广播数据: AT+BLEADVDATAEX="atgui-gu-bt","A123","0102030405",1
OK
开始广播: AT+BLEADVSTART
OK
配置BLESPP: AT+BLESPPCFG=1,1,7,1,5
OK
设置系统提示信息: AT+SYSMSG=4
OK
有BLE客户端连接, 马上开启BLE透传模式...
使能 BLESPP(开启 BLE 透传): AT+BLESPP
OK
蓝牙收到数据长度: 3, 数据内容: abc
蓝牙收到数据长度: 6, 数据内容: abc123
蓝牙收到数据长度: 2, 数据内容: cd
蓝牙收到数据长度: 16, 数据内容: 尚硅谷嵌入式课程
```

第 5 章 物联网通讯之 LoRa 通讯

5.1 LoRa 通讯基础知识

5.1.1 常见的 3 种通讯协议

现有的技术提供了面向不同场景的连接基础，目前我们了解的三类主要的协议。

（1）远距离高速率的传输协议

典型协议包括蜂窝网络通信技术，如 3G、4G、5G 相关技术等，这是我们目前移动通信使用的典型技术。

（2）近距离高速率传输技术，

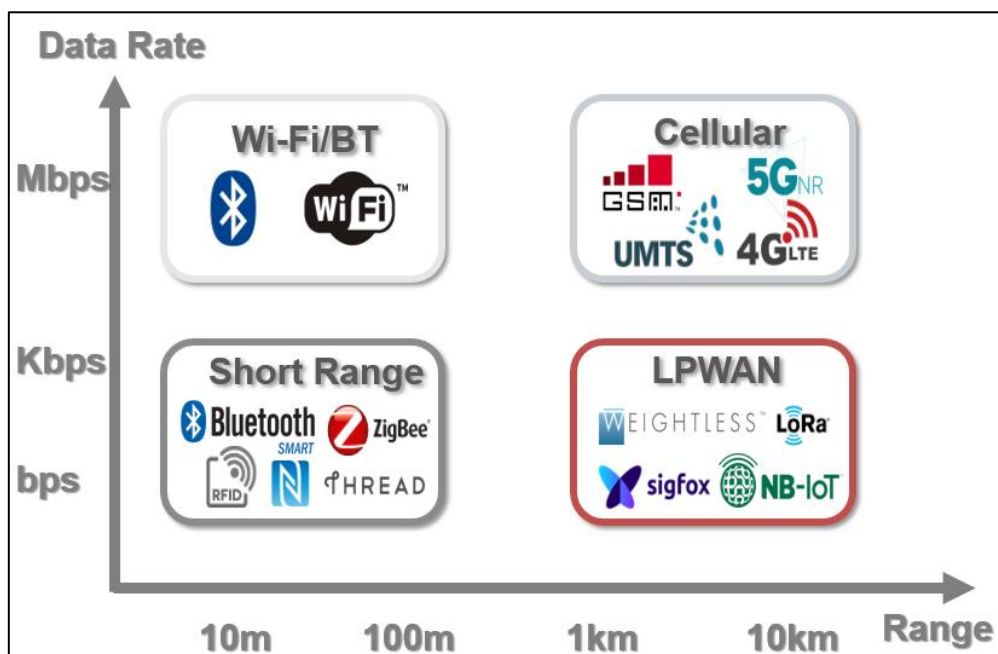
WiFi、传统蓝牙等，这些技术传输距离在几十到几百米级别，主要用在家庭环境和日常应用中，使用非常广泛。

（3）近距离低功耗传输技术

传统物联网中 ZigBee、RFID、低功耗蓝牙。

上面三类技术大都要求较高的信噪比，对障碍的穿透性较小，无法在复杂环境中实现远距离低功耗传输。

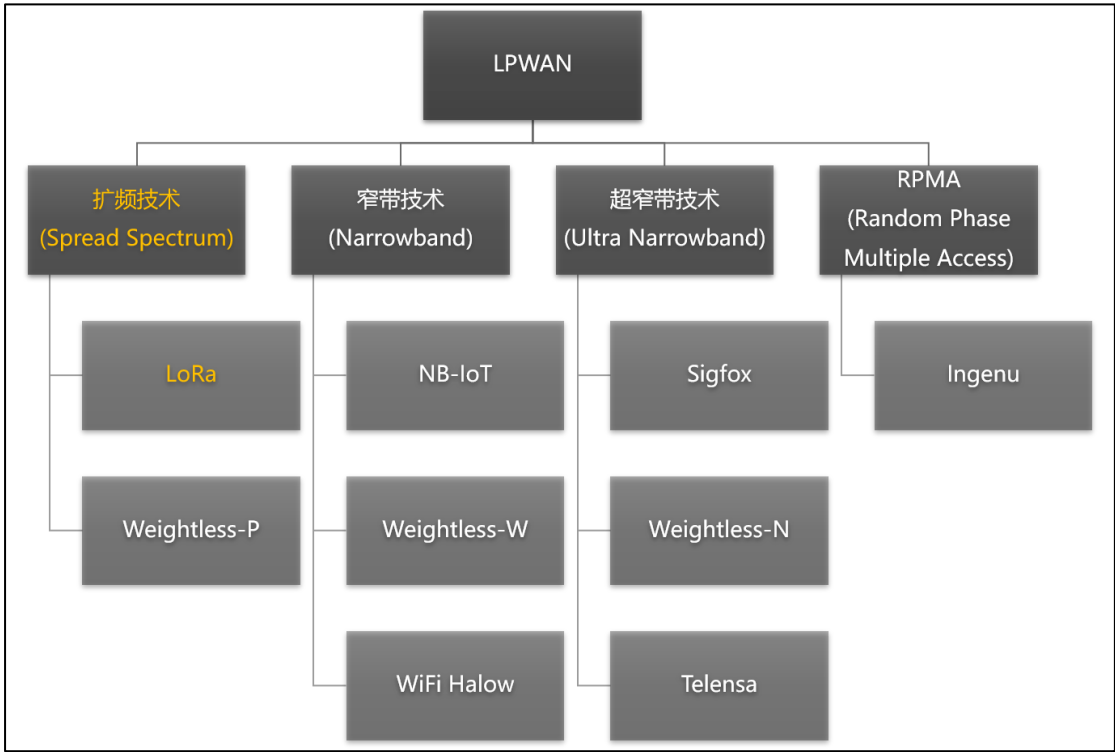
低功耗广域网技术填补这一技术空白。



5.1.2 低功耗广域网

LPWAN（Low Power Wide Area Network）指的是低功耗广域网，其特点在于极低功耗，长距离以及海量连接，适用于物联网万物互联的场景。

LPWAN 不只是一种技术，而是代表了一族有着各种形式的低功耗广域网技术，如下图所示。其中 LoRa 使用的是一种扩频技术，NB-IoT 使用的是窄带技术，这是两种有代表性的低功耗广域网技术。



低功耗广域网发展成为了适合大规模物联网应用场景的连接技术。LPWAN 兼具短距离无线网络低功耗和蜂窝网络超大覆盖范围的优点，覆盖范围广、通信能耗低。因此对于分布在大范围区域内的低功耗物联网设备来说，LPWAN 是最佳的连接选择。

在 LPWAN 网络中，这些物联网设备可以随意部署或移动，因此 LPWAN 可以满足智慧城市中的诸多应用，如智能化计量，家庭自动化，可穿戴电子，物流，环境监测等。这些应用需要交换数据量少，交换的频率也不高。

LPWAN 应用场景包括但不限于智能交通、工厂、农业、采矿等领域。由于 LPWAN 具有传统的蜂窝网络 and 传统无线技术所不具备的特点（例如相比于蜂窝网络来说功耗更低，相比于传统无线网覆盖更广），同时由于其独特的设计，使得 LPWAN 通常能在更低的信噪比下工作，因此能够适合在复杂环境中的联网。

现有的 LPWAN 技术，按工作频段不同，主要可以分为授权频段（License Band）和非授权频段（Unlicense Band）两类。

(1) 采用授权频段技术的为 3GPP (3rd Generation Partnership Project) 主导的 NB-IoT (Narrow Band IoT), 其采用现有蜂窝网络的基础硬件, 通过升级来实现, 主要投入为电信营运商及相关设备厂商。

(2) 至于非授权频段, 就呈现了遍地开花的状况, 大部分不属于电信领域的 ICT 厂商, 主要的代表技术 **LoRa**、SigFox 等。它们都采用了 ISM 频段 (Industrial Scientific Medical Band), 这是一种各国开放给工业、科学及医学机构使用的频段。它们无须许可证及费用, 只需要遵守一定的发射功率 (一般低于 1W), 不要对其他频段造成干扰即可。

5.1.3 LoRa 简介

LoRa 官网: <https://www.semtech.cn/lora>



LoRa 是 Long Range Communication 的简称, 狭义上的 LoRa 指的是一种物理层的信号调制方式, 是 Semtech (先科) 公司定义的一种基于 Chirp 扩频技术的物理层调制方式, 可达到 -148 dBm 的接收灵敏度, 以偏小的数据速率 (0.3-50kbps) 换取更高的通讯距离 (市内 3km, 郊区 15km) 和低功耗 (电池供电在特定条件下可以工作长达 10 年)。

从系统角度看, LoRa 也指由终端节点、网关、网络服务器、应用服务器所组成的一种网络系统架构: LoRa 定义了不同设备在系统中的分工与作用, 规定了数据在系统中流动与汇聚的方式。

从应用角度看, LoRa 为物联网应用提供了一种低成本、低功耗、远距离的数据传输服务: LoRa 在使用 **10mW 射频** 输出功率的情况下, 可以提供超过 **15km** 传输距离, 从而支持大量广域低功耗物联网应用。

5.1.4 LoRa 的特点

1) 远距离

在乡村地区能以长达 30 英里的距离为间隔连接设备, 可穿透建筑物密集的市区或进深较长的室内环境。

2) 低功耗

仅需极少能源, 电池使用寿命长达 10 年, 将电池更换成本降到最低。

3) 安全

采用端到端 AES128 加密、双向认证等技术，实现完整性保护和保密性。

4) 标准化

具有 LoRaWAN 网络的设备互操作性和全球可用性，可在任何地方快速部署物联网应用。

5) 地理定位

无需 GPS 即可实现跟踪应用，具有其他技术难以企及的低功率效益。

6) 移动性

移动中也能保证通过设备顺畅通信，摆脱功率消耗的限制。

7) 高性能

每个基站可处理逾百万条消息，完全满足公共网络运营商为大型市场提供服务的需求。

8) 低成本

减少了基础设施的投资、电池更换支出和最终运营费用

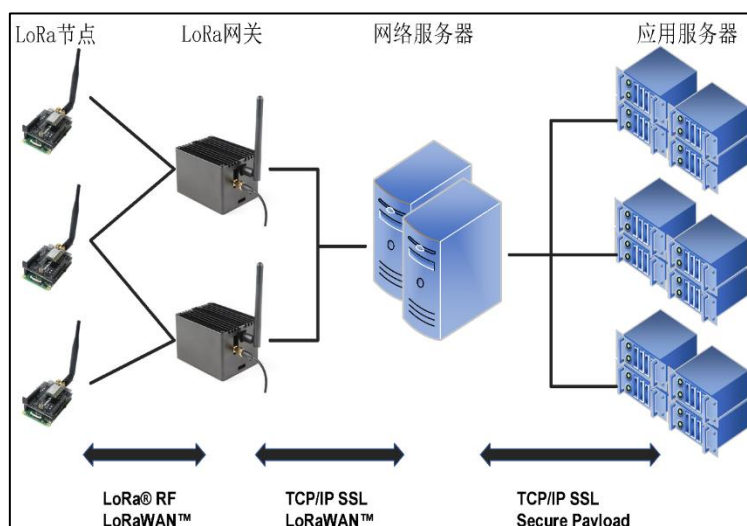
5.1.5 LoRa 应用

相比于 Wi-Fi、蓝牙、ZigBee 等传统无线局域网，LoRa 可以实现更远距离的通信，有效扩展了网络的覆盖范围；而相比于移动蜂窝网络，LoRa 具有更低的硬件部署成本和更长的节点使用寿命，单个 LoRa 节点可以在电池供电的情况下连续工作数年。LoRa 具有低数据率、远距离和低功耗的性质，因此非常适合与室外的传感器及其他物联网设备进行通信或数据交互。

考虑到 LoRa 在覆盖距离、部署成本等方面的巨大优势，近年来 LoRa 在全球范围内进行了大量的应用部署，在智能仪表（如智能水表、智能电表）、智慧城市、智能交通数据采集、野生动物监控等众多物联网场景中都可以看到 LoRa 的应用。

例如 LoRa 通信模块与传统的水质传感器进行连接，从而使用户可以数十公里外远程监控饮用水在输送过程中的水质变化情况。而在荷兰的 KPN 项目中，工程人员通过广泛部署 LoRa 网关，实现 LoRa 网络全覆盖，为智慧运输、智能农业、智慧路灯等具体应用提供了通信支持。

5.1.6 LoRa 组网架构

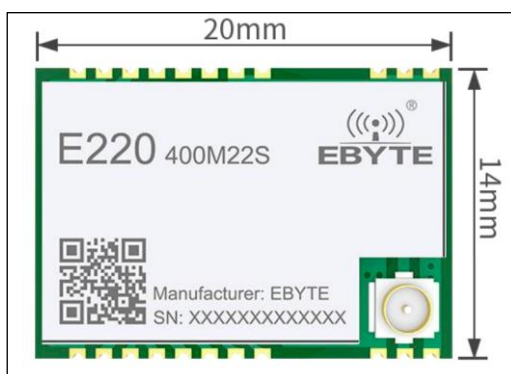


现在常用的 LoRa 架构由**节点**、**网关**及**服务器**所组成，各部分的关系如图所示。 LoRa 节点与网关之间采用单跳直接连接，这一阶段的物理层使用**线性扩频调制**（Chirp Spreading Spectrum，CSS。一种无线信号调制方法，它通过改变信号的频率来扩展信号的带宽），MAC 层通常使用 LoRaWAN 协议。

网关收到数据包后，对数据包信号进行解码，并将解码结果通过蜂窝或有线网络传输给网络服务器，这一阶段使用传统的 TCP/IP 进行传输，同时网络服务器与网关之间的交互仍然遵守 LoRaWAN 协议。网络服务器汇总多个网关的数据，过滤重复的数据包，执行安全检查，并根据内容将数据发送至不同的应用服务器，供用户读取和使用，这一阶段也使用 TCP/IP 和 SSL 进行传输和加密。

5.2 LoRa 模块 E220-400M22S

5.2.1 简介



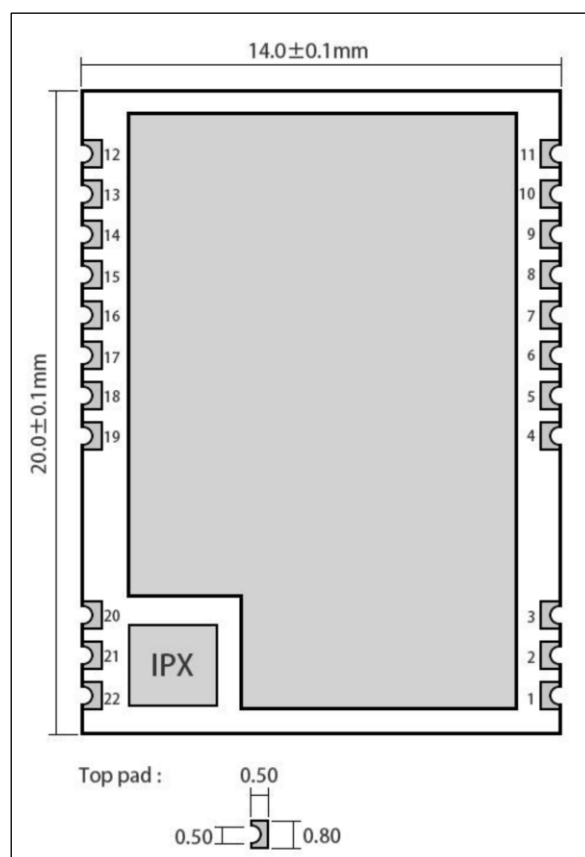
E220-400M22S 是 EBYTE（亿佰特）公司是基于 Semtech 公司生产的全新一代 LoRa"射频频芯片 **LLCC68** 为核心自主研发的超小体积并适用于 433MHz、470MHz 贴片式 LoRa 无线模块，使用工业级 32Mz 晶振。由于其采用原装进口的 **LLCC68** 为模块核心，与上一代 LoRa

收发器相比，抗干扰性能与通信距离得到了进一步提升。由于其采用全新的 LoRa “调制技术，在抗干扰性能、通信距离都远超现在的 SK、GFSK 调制方式的产品。该模块主要针对智能家庭、无线抄表、科研和医疗以及中远距离无线通信设备。该产品可覆盖 410-493MHz 超宽适用频率范围。由于该模块是纯射频收发模块需要使用 MCU 驱动或使用专用的 SPI 调试工具。

5.2.2 应用场景

- (1) 家庭安防报警及远程无钥匙进入
- (2) 智能家居以及工业传感器等
- (3) 无线报警安全系统
- (4) 楼宇自动化解决方案
- (5) 无线工业级遥控器
- (6) 医疗保健产品
- (7) 高级抄表架构（AMT）
- (8) 汽车行业应用。

5.2.3 引脚说明



引脚序号	引脚名称	引脚方向	引脚用途
1	GND		地线，连接到电源参考地
2	GND		地线，连接到电源参考地
3	GND		地线，连接到电源参考地
4	GND		地线，连接到电源参考地
5	DIO3		可配置的通用 IO 口（详见 LLCC68 手册）
6	RXEN	输入	射频开关接收控制脚，连接外部单片机 IO，高电平有效
7	TXEN	输入	射频开关发射控制脚，连接外部单片机 IO 或 DIO2，高电平有效
8	DIO2	输入/输出	可配置的通用 IO 口（详见 LLCC68 手册）
9	VCC		供电电源，范围 1.8V~3.7V（建议外部增加陶瓷滤波电容）
10	GND		地线，连接到电源参考地
11	GND		地线，连接到电源参考地
12	GND		地线，连接到电源参考地
13	DIO1	输入/输出	可配置的通用 IO 口（详见 LLCC68 手册）
14	BUSY	输出	用于状态指示（详见 LLCC68 手册）
15	NRST	输入	芯片复位触发输入脚，低电平有效
16	MISO	输出	SPI 数据输出引脚
17	MOSI	输入	SPI 数据输入引脚

18	SCK	输入	SPI 时钟输入引脚
19	NSS	输入	模块片选引脚，用于开始一个 SPI 通信
20	GND		地线，连接到电源参考地
21	ANT		射频接口，邮票孔
22	GND		地线，连接到电源参考地
关于模块的引脚定义、软件驱动及通信协议详见 SEMTECH 官方 LLCC68 Datasheet			

5.2.4 LoRa 通讯的几个关键参数

1) 载波频率

LoRa 的工作频率。范围在 410MHz-493MHz。典型值 433MHz，470MHz，490MHz。

2) 发射功率

发射功率越大，信号能传输的距离就越远。我们的模块最大发射功率可以导 22dbm。

dBm 是分贝毫瓦（decibel-milliwatts）的缩写，是一个基于对数的单位，用于表示功率相对于 1 毫瓦（mW）的级别。

3) 空中速率

LoRa 的“空中速率”（也称为“空中数据速率”或“无线数据速率”）指的是 LoRa 系统在无线空中接口上实现的数据传输速率。这个速率是指在 LoRa 通信中，数据以多快的速度通过空中从一个设备传输到另一个设备。空中速率受多种因素影响，包括扩频因子（Spreading Factor, SF）、带宽（Bandwidth）、编码率（Coding Rate）等。

4) 扩频因子（SF）

是 LoRa（Long Range）通信技术中的一个关键参数，它影响着 LoRa 信号的传输范围、数据速率和能耗。

LoRa 技术通常提供从 SF7 到 SF12 的扩频因子选择。SF7 提供最高的数据传输速率和最短的传输距离。SF12 则提供最远的传输距离，但数据传输速率最低。

5) 带宽 (BW)

带宽是指可用于 LoRa 传输的频率范围。带宽越宽，可传输的数据量越大，因此空中速率会提高。

6) 编码率 (CR)

编码率是指数据冗余度的级别。增加编码率可以提高传输的可靠性，但会稍微降低空中速率。

7) 同步字 (Sync Word)

LoRa 通信中的同步字 (Sync Word) 是用来区分和识别特定 LoRa 网络的一种机制。它是一个预定义的字节序列，用于帮助 LoRa 接收器区分目标网络的信号和其他 LoRa 信号或背景噪声。在 LoRa 通信中，同步字对于确保数据包的正确识别和网络的隔离非常重要。

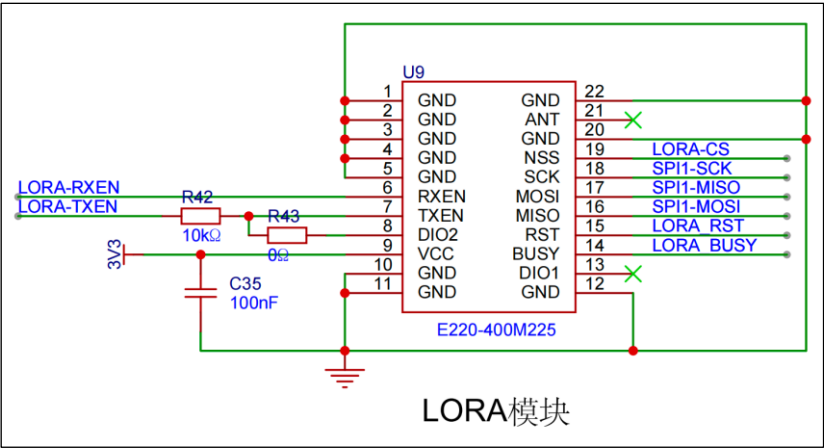
同步字的值通常可以自定义，允许用户根据需求设置不同的值以创建独立的 LoRa 网络。所有在同一个 LoRa 网络中的设备应该使用相同的同步字。

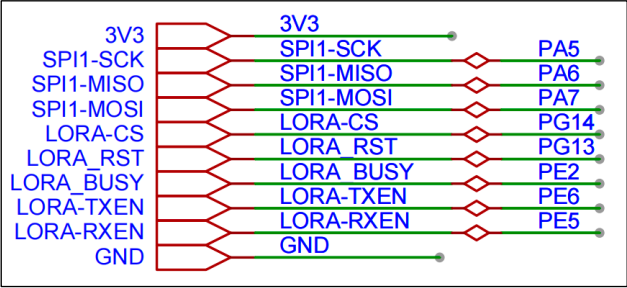
8) 数据包前导码

LoRa 数据包中的前导码 (Preamble) 是 LoRa 通信协议中非常重要的一个组成部分。前导码位于数据包的开始部分，是一系列预定义的模式，用于帮助接收器准确地同步到发送方的信号。

前导码的长度可以根据通信需求进行配置。长的前导码可以提高接收器同步的准确性，特别是在信号弱或环境噪声大的情况下。然而，过长的前导码可能会增加整个数据包的传输时间，从而影响通信的效率。

5.2.5 硬件电路设计





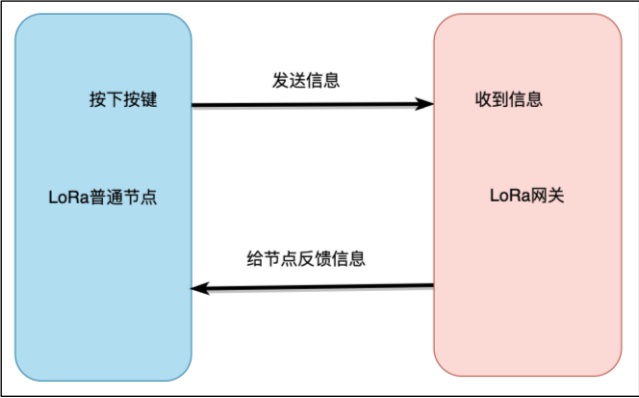
5.3 LoRa 通讯实验 1：LoRa 普通节点

5.3.1 需求描述

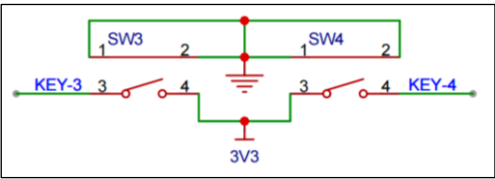
我们需要两个支持 LoRa 的开发板来完成实验。一个普通节点的角色，一个网关角色。

在普通节点上，当我们按下按键时，给网关发送数据。网关收到数据后给普通节点回馈信息。

把他们收到的信息分别在串口打印输出。



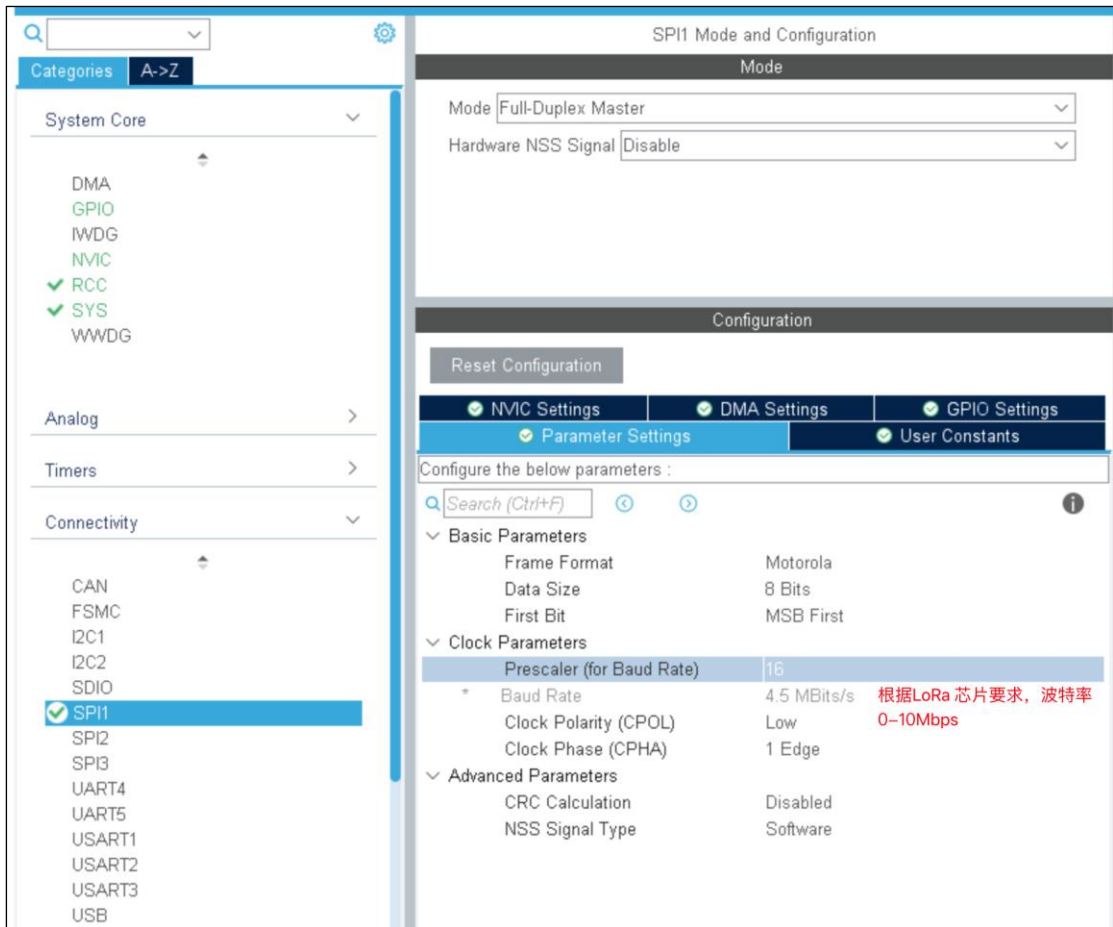
5.3.2 硬件电路设计



55	PF15	FSMC-A9
54	PF14	FSMC-A8
53	PF13	FSMC-A7
50	PF12	FSMC-A6
49	PF11	KEY-4
22	PF10	KEY-3
21	PF9	KEY-2
20	PF8	KEY-1
19	PF7	
18	PF6	
15	PF5	FSMC-A5
14	PF4	FSMC-A4
13	PF3	FSMC-A3
12	PF2	FSMC-A2
11	PF1	FSMC-A1
10	PF0	FSMC-A0

5.3.3 软件设计

5.3.3.1 创建项目



Search Signals

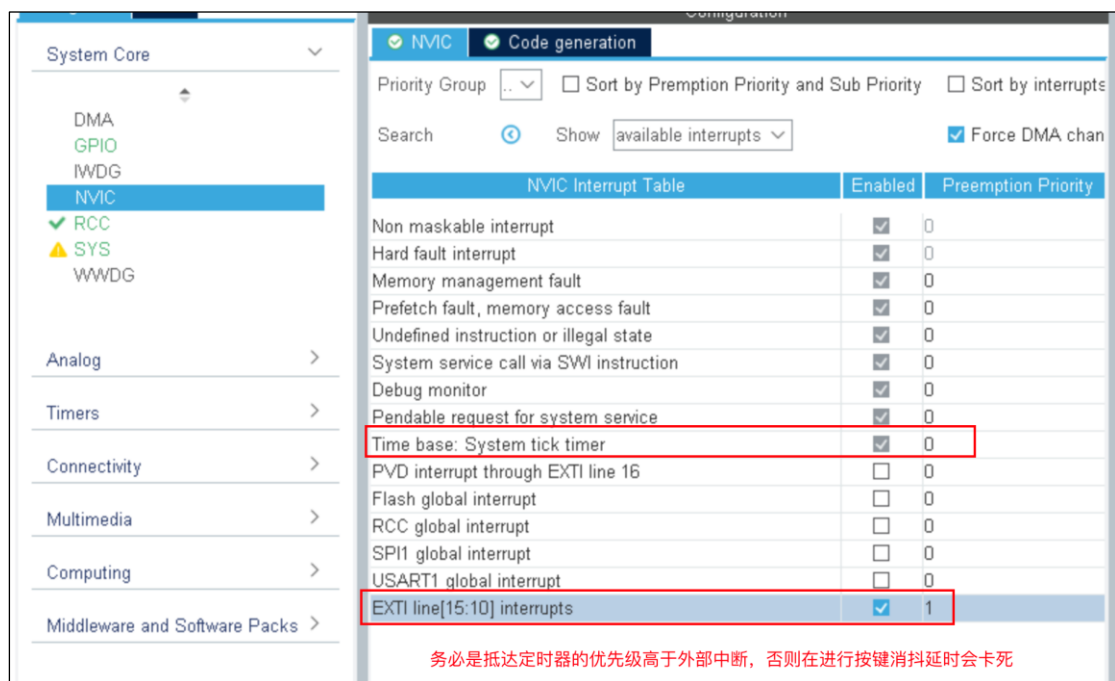
Search (Ctrl+F) 其他 GPIO 引脚。注意 PE2 是浮空输入 PF10 是外部中断复用 ☐ Show only Modified Pins

Pin Name	Signal on Pin	GPIO output...	GPIO mode	GPIO Pull-u...	Maximum o...	User Label	Modified
PE2	n/a	n/a	Input mode	No pull-up a...	n/a	LoRa_BUSY	✓
PE5	n/a	Low	Output Push...	No pull-up a...	Low	LoRa_RxEN	✓
PE6	n/a	Low	Output Push...	No pull-up a...	Low	LoRa_TxEN	✓
PF10	n/a	n/a	External Inte...	No pull-up a...	n/a	KEY	✓
PG13	n/a	Low	Output Push...	No pull-up a...	Low	LoRa_RST	✓
PG14	n/a	Low	Output Push...	No pull-up a...	Low	LoRa_CS	✓

Group By Peripherals

GPIO	RCC	SPI	SYS	USART	NVIC
NVIC Interrupt Table					
EXTI line[15:10] interrupts					
Enabled					
Preemption Priority					
Sub Priority					

使能中断

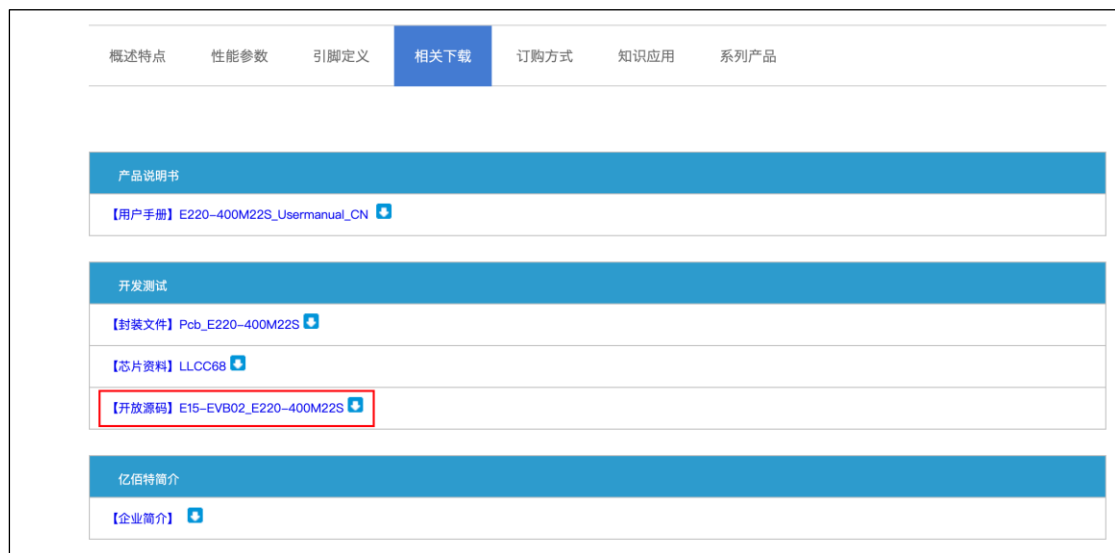


5.3.3.2 移植官方驱动

1) 下载亿佰特官方驱动

首先下载亿佰特官方提供的驱动。下载地址：

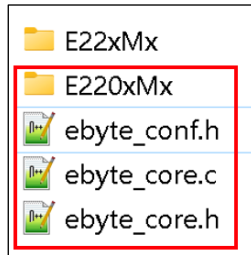
<https://www.ebyte.com/product-view-news.html?id=1355>



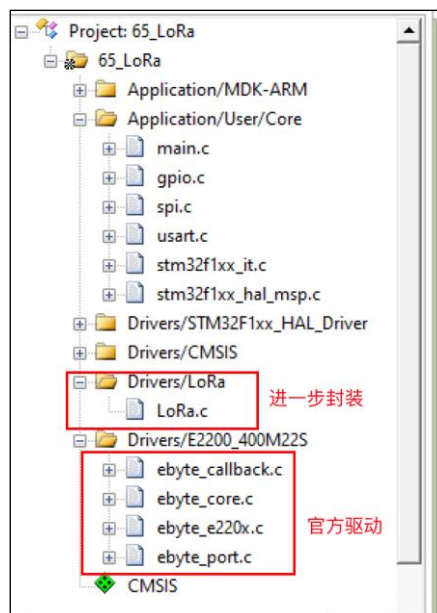
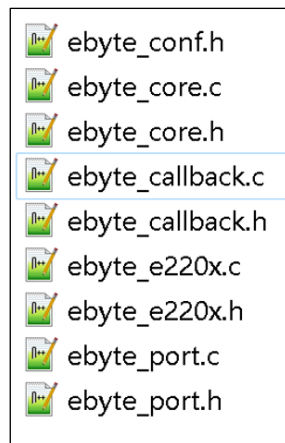
2) copy 必须的文件

把官方驱动文件 Copy 到我们的项目中。

 3_Ebyte_WirelessModule_Drivers



Copy 成功之后的文件，一共 9 个，为了方便，都放入到同一个文件夹（E220_400M225）下：



3) 驱动文件说明

ebyte_core.c/h 必须包含在用户工程中。功能为顶层公共接口库，简化底层调用，今后其他产品将陆续接入该层。用户只需简单的调用高度封装的 API,即可立即开始进行无线传输，省去了底层繁琐的配置。

ebyte_conf.h 必须包含在用户工程中。功能为公共配置，用于帮助驱动库自动切换内部函数接口。需要根据自己的产品来修改宏定义。

ebyte_callback.c/h 接口输出层。功能为提供发送/接收完成后的回调调函数，用户在回调中可实现自己的逻辑。

ebyte_e22x.c/h E22x 产品驱动库，一般无需修改，默认即可。

ebyte_port.c/h 接口输入层。由用户提供具体目标硬件平台的 **I0** 操作，由驱动库内部自行调用完成驱动逻辑。

4) 修改 ebyte_conf.h

添加包含文件和根据我们用的产品型号配置产品型号。

```
#include "main.h" // 添加包含文件
/*= !!!配置产品型号 单选 =====*/
// #define EBYTE_E07_900M10S
// #define EBYTE_E10_400M20S
// #define EBYTE_E10_900M20S
// #define EBYTE_E22_400M22S 添加注释
// #define EBYTE_E22_900M22S
// #define EBYTE_E49_400M20S
// #define EBYTE_E49_900M20S
#define EBYTE_E220_400M22S // 去掉注释
// #define EBYTE_E220_900M22S
/*=====*/

/* SPI 通信 由软件控制 CS(NSS)信号 0:禁用 1:启用 */
#define EBYTE_PORT_SPI_CS_SOFTWARE 1

/* 如何检测模块发送时的状态 0:异步中断通知 1:同步阻塞 */
#define EBYTE_RF_TRANSMIT_CHECK_MODE 1
```

5) 修改 ebyte_port.c

```
#include "ebyte_port.h"
#include "spi.h"

/*= !!!配置目标硬件平台头文件 =====*/

/*=====*/

/* !
 * @brief 配置目标硬件平台 SPI 接口收发函数
 *
 * @param send EBYTE 驱动库上层调用需要传输的数据 1 Byte
 * @return SPI 接收的数据 1 Byte
 * @note 当目标平台采用硬件 SPI_NSS 时，请配置 ebyte_conf.h 文件 关闭驱动库的软件控制 SPI_CS(NSS)
```

```

    *      当目标平台采用软件 SPI_NSS 时，还需额外配置 Ebyte_Port_SpiCsIoControl() 具体请查
    阅函数说明
    */
uint8e_t Ebyte_Port_SpiTransmitAndReceivce(uint8e_t send)
{
    uint8e_t result = 0;

    /* !必须提供：SPI 接口 */
    HAL_SPI_TransmitReceive(&hspi1, &send, &result, 1, 1000);
    return result;
}

/* !
 * @brief (可选)配置目标硬件平台 SPI_CS(NSS)函数
 *
 * @param cmd EBYTE 驱动库上层期望的控制指令
 *      @arg 0: 期望 CS(NSS)引脚输出低电平逻辑信号 EBYTE 模块 SPI 接口为低电平选中
 *      @arg 1: 期望 CS(NSS)引脚输出高电平逻辑信号
 * @note 当目标平台采用硬件 SPI_NSS 时，由配置文件定义 此函数将无效
 *      当目标平台采用软件 SPI_NSS 时，请配置此函数
 */
void Ebyte_Port_SpiCsIoControl(uint8e_t cmd)
{
    if (cmd == 1)
    {
        /* !可选：SPI CS 控制 E22xx 高电平未选中 */
        HAL_GPIO_WritePin(LoRa_CS_GPIO_Port, LoRa_CS_Pin, GPIO_PIN_SET);
    }
    else
    {
        /* !可选：SPI CS 控制 E22xx 低电平选中 */
        HAL_GPIO_WritePin(LoRa_CS_GPIO_Port, LoRa_CS_Pin, GPIO_PIN_RESET);
    }
}

/* !
 * @brief 配置目标硬件平台复位引脚 NRST 控制函数
 *
 * @param cmd EBYTE 驱动库上层期望的控制指令
 *      @arg 0: 期望 NRST 引脚输出低电平逻辑信号
 *      @arg 1: 期望 NRST 引脚输出高电平逻辑信号
 * @note EBYTE 模块 NRST 为低电平复位
 */
void Ebyte_Port_RstIoControl(uint8e_t cmd)

```

```

{
    if (cmd == 1)
    {
        /* !必须提供：NRST 控制 IO 输出高电平 */
        HAL_GPIO_WritePin(LoRa_RST_GPIO_Port, LoRa_RST_Pin, GPIO_PIN_SET);
    }
    else
    {
        /* !必须提供：目标硬件电路对应 NRST 控制 IO 输出低电平 */
        HAL_GPIO_WritePin(LoRa_RST_GPIO_Port, LoRa_RST_Pin, GPIO_PIN_RESET);
    }
}

/* !
 * @brief (可选)配置目标硬件平台发送使能引脚 TXEN 控制函数
 *
 * @param cmd EBYTE 驱动库上层期望的控制指令
 *          @arg 0: 期望 TXEN 引脚输出低电平逻辑信号
 *          @arg 1: 期望 TXEN 引脚输出高电平逻辑信号
 * @note EBYTE 模块 TXEN 为高电平生效
 *       当目标平台采用模块 DIO2 自动控制 TXEN 时，此函数将无效
 */
void Ebyte_Port_TxenIoControl(uint8_t cmd)
{
    if (cmd == 1)
    {
        /* !可选：目标硬件电路对应 TXEN 控制 IO 输出高电平 */
        HAL_GPIO_WritePin(LoRa_TxEN_GPIO_Port, LoRa_TxEN_Pin, GPIO_PIN_SET);
    }
    else
    {
        /* !可选：目标硬件电路对应 TXEN 控制 IO 输出低电平 */
        HAL_GPIO_WritePin(LoRa_TxEN_GPIO_Port, LoRa_TxEN_Pin, GPIO_PIN_RESET);
    }
}

/* !
 * @brief 配置目标硬件平台接收使能引脚 RXEN 控制函数
 *
 * @param cmd EBYTE 驱动库上层期望的控制指令
 *          @arg 0: 期望 RXEN 引脚输出低电平逻辑信号
 *          @arg 1: 期望 RXEN 引脚输出高电平逻辑信号
 * @note EBYTE 模块 RXEN 为高电平生效
 */

```

```

void Ebyte_Port_RxenIoControl(uint8e_t cmd)
{
    if (cmd == 1)
    {
        /* !必须提供：目标硬件电路对应 RXEN 控制 IO 输出高电平 */
        HAL_GPIO_WritePin(LoRa_RxEN_GPIO_Port, LoRa_RxEN_Pin, GPIO_PIN_SET);
    }
    else
    {
        /* !必须提供：目标硬件电路对应 RXEN 控制 IO 输出低电平 */
        HAL_GPIO_WritePin(LoRa_RxEN_GPIO_Port, LoRa_RxEN_Pin, GPIO_PIN_RESET);
    }
}

/* !
 * @brief 配置目标硬件平台接收使能引脚 BUSY 控制函数
 *
 * @note EBYTE 模块 BUSY 为高电平表示忙
 */
uint8e_t Ebyte_Port_BusyIoRead(void)
{
    uint8e_t result = 0;

    /* !必须提供：目标硬件电路对应 BUSY IO 状态读取 */
    HAL_GPIO_ReadPin(LoRa_BUSY_GPIO_Port, LoRa_BUSY_Pin);
    return result;
}

/* !
 * @brief 配置目标硬件平台延时函数
 *
 * @param time 期望延时毫秒
 * @note 请注意时序，模块初始化时将调用此函数，请注意中断是否会影响到此函数
 */
void Ebyte_Port_DelayMs(uint16e_t time)
{
    /* !必须提供：延时函数 */
    HAL_Delay(time);
}

```

6) 修改 ebyte_callback.c

```

#include "ebyte_callback.h"

/*= !!!配置目标硬件平台头文件 =====*/
#include "ebyte_core.h"

```

```

#include "stm32f1xx_hal.h"

#include "stdio.h"

/*= !!!配置目标硬件变量 =====*/

/*=====*/

/* !
 * @brief 发送完成回调接口 由客户实现自己的发送完成逻辑
 *
 * @param state 上层回调提供的状态码 客户请根据示例注释找到对应区域
 *
 * @note E22x 模块可提供的状态码
 *
 *      IRQ_TX_DONE                = 0x0001,
 *      IRQ_RX_DONE                = 0x0002,
 *      IRQ_PREAMBLE_DETECTED     = 0x0004,
 *      IRQ_SYNCWORD_VALID        = 0x0008,
 *      IRQ_HEADER_VALID          = 0x0010,
 *      IRQ_HEADER_ERROR          = 0x0020,
 *      IRQ_CRC_ERROR              = 0x0040,
 *      IRQ_CAD_DONE               = 0x0080,
 *      IRQ_CAD_ACTIVITY_DETECTED = 0x0100,
 *      IRQ_RX_TX_TIMEOUT         = 0x0200,
 */

void Ebyte_Port_TransmitCallback(uint16e_t state)
{
    printf("发送状态码=0x%x\r\n", state);
    /* 发送完成后切换到接收模式 */
    Ebyte_RF.EnterReceiveMode(0);

    /* 发送：正常完成 */
    if (state & 0x0001)
    {
        // To-do 实现自己的逻辑
        printf("数据发送正常...\r\n");
    }
    /* 发送：异常超时 */
    else if (state & 0x0200)
    {
        // To-do 实现自己的逻辑
        printf("数据发送异常超时...\r\n");
    }
    /* 发送：未知错误 */
    else

```

```

{
    /* 发送状态标识不正确，请检查软硬件
       常见问题 1:SPI 通信不正确 2:模块供电不足 */
    printf("数据发送出现未知错误，请检查硬件...\r\n");
    while (1)
        ;
}
}

/* !
 * @brief 接收完成回调接口 由客户实现自己的发送完成逻辑
 *
 * @param state 上层回调提供的状态码 客户请根据示例注释找到对应区域
 *
 * @note E22x 模块可提供的状态码
 *
 *      IRQ_TX_DONE                = 0x0001,
 *      IRQ_RX_DONE                = 0x0002,
 *      IRQ_PREAMBLE_DETECTED      = 0x0004,
 *      IRQ_SYNCWORD_VALID         = 0x0008,
 *      IRQ_HEADER_VALID           = 0x0010,
 *      IRQ_HEADER_ERROR           = 0x0020,
 *      IRQ_CRC_ERROR              = 0x0040,
 *      IRQ_CAD_DONE               = 0x0080,
 *      IRQ_CAD_ACTIVITY_DETECTED  = 0x0100,
 *      IRQ_RX_TX_TIMEOUT          = 0x0200,
 */
void Ebyte_Port_ReceiveCallback(uint16_t state, uint8_t *buffer, uint8_t length)
{
    printf("接收状态码....0x%x\r\n", state);
    /* 接收：正常 */
    if (state & 0x0002)
    {
        // To-do 实现自己的逻辑
        printf("接收到数据，长度=%d，内容=%s\r\n", length, buffer);
    }
    /* 接收：异常超时 */
    else if (state & 0x0200)
    {
        // To-do 实现自己的逻辑
        printf("数据接收出现异常超时\r\n");
    }
    /* 接收：未知错误 */
    else

```

```

{
    /* 发送状态标识不正确，请检查软硬件
       常见问题 1:SPI 通信不正确 2:模块供电不足 */
    printf("数据接收出现未知错误，请检查硬件...\r\n");
    while (1)
        ;
}
}

```

7) 注意

官方的驱动有两个地方有 bug

(1) 在 `ebyte_callback.c` 中，状态码判断的地方不应 `&=` 而是应该直接 `&`。否则如果第一个 if 是 false，则 state 直接就成了 0。

(2) 设置同步字的命令错了。`ebyte_e200x.c`

```

1248 void Ebyte_E220x_SetLoraSyncWord( uint16e_t syncWord )
1249 {
1250     Ebyte_E220x_WriteRegister( REG_LR_SYNCWORDBASEADDRESS, ( syncWord >> 8 ) & 0xFF );
1251     Ebyte_E220x_WriteRegister( REG_LR_SYNCWORDBASEADDRESS + 1, syncWord & 0xFF );
1252 }
1253

```

改为

```

void Ebyte_E220x_SetLoraSyncWord( uint16e_t syncWord )
{
    Ebyte_E220x_WriteRegister( REG_LR_SYNCWORD, ( syncWord >> 8 ) & 0xFF );
    Ebyte_E220x_WriteRegister( REG_LR_SYNCWORD + 1, syncWord & 0xFF );
}

```

```

23 #define REG_LR_SYNCWORDBASEADDRESS fsk 调制下的同步字命令 0x06C0
24 #define REG_LR_SYNCWORD lora 调整下的同步字命令 0x0740

```

	CRC LSB polynomial Value [1]	0x06BF	0x21	mode
SyncWord[0]	0x06C0	-	-	1st byte of the Sync Word in FSK mode
SyncWord[1]	0x06C1	-	-	2nd byte of the Sync Word in FSK mode
SyncWord[2]	0x06C2	-	-	3rd byte of the Sync Word in FSK mode
SyncWord[3]	0x06C3	-	-	4th byte of the Sync Word in FSK mode
SyncWord[4]	0x06C4	-	-	5th byte of the Sync Word in FSK mode
SyncWord[5]	0x06C5	-	-	6th byte of the Sync Word in FSK mode
SyncWord[6]	0x06C6	-	-	7th byte of the Sync Word in FSK mode
SyncWord[7]	0x06C7	-	-	8th byte of the Sync Word in FSK mode
Node Address	0x06CD	0x00		Node Address used in FSK mode
Broadcast Address	0x06CE	0x00		Broadcast Address used in FSK mode
IQ Polarity Setup	0x0736	0x0D		Optimize the inverted IQ operation
LoRa Sync Word MSB	0x0740	0x14		Differentiate the LoRa® signal for Public or Private Network
LoRa Sync Word LSB	0x0741	0x24		Set to 0x3444 for Public Network Set to 0x1424 for Private Network

5.3.3.3 main.c

```
#include "main.h"
#include "spi.h"
#include "usart.h"
#include "gpio.h"

#include "stdio.h"
#include "LoRa.h"
#include "string.h"
void SystemClock_Config(void);
uint8_t msg[100] = {0};
uint32_t count = 0;
uint8_t isKeyPressed = 0;
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    if (GPIO_Pin == KEY_Pin)
    {
        /*
            延时消抖。需要注意的是，一定要使滴答定时器的中断优先级高于外部中断的优先级
            否则会卡死
        */
        HAL_Delay(200);
        if (HAL_GPIO_ReadPin(KEY_GPIO_Port, KEY_Pin))
        {
            isKeyPressed = 1;
        }
    }
}
int main(void)
{
    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();
    MX_SPI1_Init();
    MX_USART1_UART_Init();
    printf("尚硅谷 LoRa 普通节点通讯实验...\r\n");
    LoRa_Init();

    uint32_t name = Ebyte_RF.GetName();
    printf("设备名称=0x%x\r\n", name);

    while (1)
    {
        LoRa_StartPollTask();
    }
}
```

```

        if (isKeyPressed)
        {
            isKeyPressed = 0;
            memset(msg, 0, sizeof((char *)msg));
            /* 给 LoRa 网关发送数据 */
            count++;
            sprintf((char *)msg, "一个普通的 LoRa 节点,我要开始计数了... %d", count);
            LoRa_Send(msg, strlen((char *)msg));
        }
    }
}

```

5.3.3.4 LoRa.h

```

#ifndef __LORA_H
#define __LORA_H

#include "ebyte_core.h"

void LoRa_Init(void);
void LoRa_StartPollTask(void);
void LoRa_Send(uint8_t *buff, uint8_t size);

#endif

```

5.3.3.5 LoRa.c

```

#include "LoRa.h"

/**
 * @description: 初始化 LoRa
 */
void LoRa_Init(void)
{
    /* 1. 初始化 LoRa */
    Ebyte_RF.Init();
}

/**
 * @description: LoRa 的周期执行的函数，建议放在主循环中
 * @return {*}
 */
void LoRa_StartPollTask(void)
{
    Ebyte_RF.StartPollTask();
}

```

```

/**
 * @description: 发送数据
 * @param {uint8_t} *buff 要发送的数据
 * @param {uint8_t} size 要发送的数据字节数
 * @return {*}
 */
void LoRa_Send(uint8_t *buff, uint8_t size)
{
    /* 1. 发送数据 注意超时时间必须是 0*/
    Ebyte_RF.Send(buff, size, 0);
}

```

5.4 LoRa 通讯案例 2：LoRa 网关

5.4.1 软件设计

5.4.1.1 SMT32CubeMX 配置

与普通节点的项目相比，配置方面只需要去掉按键相关的配置即可。其余保持不变。

5.4.1.2 ebyte_callback.c

```

#include "ebyte_callback.h"

/*= !!!配置目标硬件平台头文件 =====*/
#include "ebyte_core.h"
#include "stm32f1xx_hal.h"

#include "stdio.h"
#include "string.h"
/*= !!!配置目标硬件变量 =====*/

/*=====*/

/* !
 * @brief 发送完成回调接口 由客户实现自己的发送完成逻辑
 *
 * @param state 上层回调提供的状态码 客户请根据示例注释找到对应区域
 *
 * @note E22x 模块可提供的状态码
 *
 *      IRQ_TX_DONE           = 0x0001,
 *      IRQ_RX_DONE          = 0x0002,
 *      IRQ_PREAMBLE_DETECTED = 0x0004,
 *      IRQ_SYNCWORD_VALID   = 0x0008,
 *      IRQ_HEADER_VALID     = 0x0010,
 *      IRQ_HEADER_ERROR     = 0x0020,
 *      IRQ_CRC_ERROR        = 0x0040,

```

```

*          IRQ_CAD_DONE                      = 0x0080,
*          IRQ_CAD_ACTIVITY_DETECTED         = 0x0100,
*          IRQ_RX_TX_TIMEOUT                 = 0x0200,
*/
void Ebyte_Port_TransmitCallback(uint16e_t state)
{
    printf("发送状态码=0x%x\r\n", state);
    /* 发送完成后切换到接收模式 */
    Ebyte_RF.EnterReceiveMode(0);

    /* 发送：正常完成 */
    if (state & 0x0001)
    {
        // To-do 实现自己的逻辑
        printf("数据发送正常...\r\n");
    }
    /* 发送：异常超时 */
    else if (state & 0x0200)
    {
        // To-do 实现自己的逻辑
        printf("数据发送异常超时...\r\n");
    }
    /* 发送：未知错误 */
    else
    {
        /* 发送状态标识不正确，请检查软硬件
        常见问题 1:SPI 通信不正确 2:模块供电不足 */
        printf("数据发送出现未知错误，请检查硬件...\r\n");
        while (1)
        {
            ;
        }
    }
}

/* !
* @brief 接收完成回调接口 由客户实现自己的发送完成逻辑
*
* @param state 上层回调提供的状态码 客户请根据示例注释找到对应区域
*
* @note E22x 模块可提供的状态码
*          IRQ_TX_DONE                      = 0x0001,
*          IRQ_RX_DONE                      = 0x0002,
*          IRQ_PREAMBLE_DETECTED           = 0x0004,
*          IRQ_SYNCWORD_VALID              = 0x0008,
*          IRQ_HEADER_VALID                = 0x0010,

```

```

*          IRQ_HEADER_ERROR                = 0x0020,
*          IRQ_CRC_ERROR                   = 0x0040,
*          IRQ_CAD_DONE                     = 0x0080,
*          IRQ_CAD_ACTIVITY_DETECTED        = 0x0100,
*          IRQ_RX_TX_TIMEOUT                = 0x0200,
*/
void Ebyte_Port_ReceiveCallback(uint16_t state, uint8_t *buffer, uint8_t length)
{
    printf("接收状态码....0x%x\r\n", state);
    /* 接收：正常 */
    if (state & 0x0002)
    {
        // To-do 实现自己的逻辑
        buffer[length] = 0;
        printf("接收到 LoRa 节点发来的数据，长度=%d，内容=%s\r\n", length, buffer);

        /* 给节点发送数据 */
        uint8_t *msg = "本网关已经接收到你的数据了，请继续干活...\r\n";
        Ebyte_RF.Send(msg, strlen((char *)msg), 0);
    }
    /* 接收：异常超时 */
    else if (state & 0x0200)
    {
        // To-do 实现自己的逻辑
        printf("数据接收出现异常超时\r\n");
    }
    /* 接收：未知错误 */
    else
    {
        /* 发送状态标识不正确，请检查软硬件
           常见问题 1:SPI 通信不正确 2:模块供电不足 */
        printf("数据接收出现未知错误，请检查硬件...\r\n");
        while (1)
        {
            ;
        }
    }
}

```

5.4.1.3 main.c

```

int main(void)
{
    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();
    MX_SPI1_Init();
    MX_USART1_UART_Init();
}

```

```

printf("尚硅谷 LoRa 网关通讯实验...\r\n");

/* 初始化 LoRa 模块 */
LoRa_Init();

uint32_t name = Ebyte_RF.GetName();
printf("设备名称=0x%x\r\n", name);

while (1)
{
    LoRa_StartPollTask();
}

```

5.4.2 LoRa 普通节点给网关发送数据测试

先下载网关程序，然后网关会等待节点发送数据。

